



**Politecnico
di Torino**

Politecnico di Torino

Ingegneria Aerospaziale

A.A. 2023/2024

Fluidodinamica Computazionale dei sistemi propulsivi

Docente

Andrea Ferrero

Studente

Jafar Mohaddes Zadeh

308343

Indice

Introduzione	4
Generazione di una Mesh	5
1.1 Software gmsh	5
1.1.1 Griglie strutturate	6
1.1.2 Griglie non strutturate	6
1.2 File .geo	7
1.2.1 Definizione della geometria	7
1.2.2 Definizione caratteristiche della mesh	8
Codice CFD	12
2.1 Strutture	12
2.2 Variabili	14
2.3 Main	15
2.3.1 Subroutine input	16
2.3.2 Subroutine leggi_gmsh	18
2.3.3 Subroutine processa_elementi	20
2.3.4 Subroutine init	23
2.3.5 Subroutine compute_dt	26
2.3.6 Subroutine compute_fluxes	27
2.3.7 Subroutine integ_time	33
Bump - mesh strutturata	34
3.1 Inizializzazione	34
3.1.1 Caso 750 nodi	35
3.1.2 Caso 3000 nodi	36
3.1.3 Caso 6750 nodi	37
3.1.4 Caso 12000 nodi	38
3.2 Analisi di convergenza	39
3.2.1 Lax-Friedrichs vs. Roe	41
3.2.2 Valutazione dell'ordine di convergenza	42
Paletta di turbina LS59	45
4.1 Inizializzazione	46
4.1.1 Campo di Mach e Velocità	48

4.1.2	Campo di Entropia	52
4.2	Analisi di convergenza	55
4.3	Confronto con dati sperimentali	57
4.4	Analisi con ANSYS Fluent	60
4.4.1	Realizzazione geometria	60
4.4.2	Realizzazione mesh	61
4.4.3	Analisi Fluent	62
Presa doppia rampa		66
5.1	Inizializzazione	66
5.1.1	Campo di Mach	68
5.1.2	Campo di Entropia	71
5.2	Analisi di convergenza	74
5.3	Confronto con dati sperimentali	76
5.4	Analisi con ANSYS Fluent	78
5.4.1	Realizzazione geometria	78
5.4.2	Realizzazione mesh	79
5.4.3	Analisi Fluent	80
Ottimizzazione di Forma		84
6.1	Procedura	84
6.2	Software modeFRONTIER	85
6.3	Risultati	88

Introduzione

Lo scopo di questo documento è riportare il lavoro svolto nell'ambito delle esercitazioni del corso **Fluidodinamica computazionale dei sistemi propulsivi**. In queste esercitazioni si è affrontato lo sviluppo di un codice CFD tramite l'impiego del linguaggio di programmazione *Fortran 90* e il confronto dello stesso con il software commerciale ANSYS Fluent. Vengono proposti in questo elaborato tutti gli step che hanno portato alla realizzazione del codice e le sue applicazioni su alcuni casi studio di interesse.

Generazione di una Mesh

Prima dello sviluppo del codice CFD ci si pone il problema di creare un file di input allo stesso, in particolare è necessario, attraverso un software esterno, generare un dominio computazionale sul quale viene generata una mesh, nei cui nodi verrà valutata la soluzione delle equazioni. Dopo aver scelto il modello matematico-fisico descrittivo del problema in esame, è necessario discretizzare le equazioni per passare da un dominio continuo ad uno discreto. La scelta di questi punti è ciò che determina la griglia.

1.1 Software gmsh

Per la generazione di una mesh ci si affida al software **gmsh**, il quale è open source ed ha al suo interno diversi ambienti disponibili tra cui un CAD, un generatore di mesh e un post processor. La discretizzazione nello spazio per mezzo di una griglia introduce un errore, la tendenza infatti sarebbe quella di avere una griglia molto fitta per tendere al caso continuo, questa strada però non è percorribile a causa dell'altissimo costo computazionale associato. Se si volesse ridurre il costo computazionale attraverso una griglia molto povera di punti, si incapperebbe ugualmente in delle problematiche in quanto si perderebbe in termini di risoluzione spaziale. La soluzione quindi sta nel popolare la griglia di un'alta quantità di punti laddove necessario, ovvero dove ci si aspetta la presenza di forti gradienti. I forti gradienti, nei casi di interesse, possono trovarsi ad esempio sulle pareti solide in cui si sviluppa uno strato limite o a valle di un urto.

Prima di discutere in dettaglio come implementare questo tipo di logica nella definizione della nostra griglia computazionale, è bene soffermarsi su quali tipi di griglie è possibile realizzare, in particolare ne esistono di due tipi:

- Griglie strutturate.
- Griglie non strutturate.

1.1.1 Griglie strutturate

Le griglie strutturate sono delle griglie in cui ogni nodo di calcolo si trova all'intersezione di due linee coordinate.

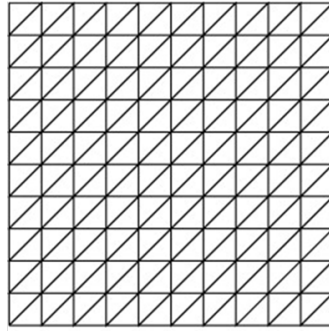


Figura 1.1: Griglia strutturata

Uno dei vantaggi di questa tipologia di griglia è la facilità con la quale è possibile definire i vicini di un nodo j –esimo, infatti ogni elemento può essere indicato con due indici. Questa caratteristica elimina la necessità di una matrice di connectivity e quindi rendere il codice di calcolo più prestante. Un'altra definizione di griglia strutturata è:

Detto N il numero di dimensioni, la griglia è detta strutturata se posso identificare N indici che scorrendo di un'unità identificano i vicini.

Il codice risulta più prestante in caso di griglie strutturate anche a causa di come lavorano i calcolatori: quando vengono richieste le informazioni di un elemento il computer si aspetta che l'user richieda in seguito delle informazioni sugli elementi associati agli indici precedenti e successivi, e quindi le estrae in automatico. Per questo tipo di griglia quindi la logica implementata nei calcolatori consente alte prestazioni. Il difetto di queste griglie, però, è che per domini molto complessi non è possibile realizzarle, inoltre se si è interessati a conoscere la soluzione in un particolare punto non basta aggiungere quel punto, ma a causa delle proprietà della griglia stessa (linee coordinate) bisogna aggiungere molti punti, i quali aumentano il costo computazionale complessivo.

1.1.2 Griglie non strutturate

Per griglie non strutturate, invece, i punti non sono più definiti come intersezione di linee coordinate, e questo infatti riduce la complessità di generazione della griglia, introduce la possibilità di introdurre liberamente nodi singoli dove necessario ed è possibile gestire geometrie più complesse. Come si può immaginare non è più possibile sfruttare una notazione a due indici come nel caso precedente e di conseguenza è necessario definire una matrice di connectivity, che il software dovrà interrogare quando svolgerà le operazioni di calcolo richieste, rendendo il processo meno prestante.

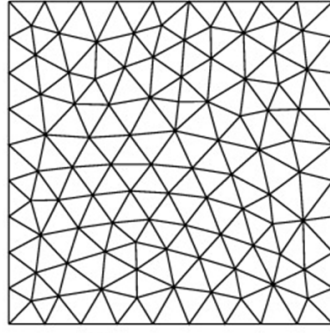


Figura 1.2: Griglia non strutturata

Nelle figure 1.1 e 1.2 vengono rappresentati degli elementi triangolari, ma le griglie possono avere una geometria poligonale diversa.

1.2 File .geo

Il software **gmsh** legge un file di testo con estensione *.geo*, il quale contiene tutte le informazioni necessarie affinché il software generi una geometria con le caratteristiche richieste e delle specifiche per la mesh.

1.2.1 Definizione della geometria

Per prima cosa si definiscono dei punti che verranno successivamente connessi da delle linee, per farlo si usa la sintassi del comando *Point* :

$$Point(i) = \{x, y, z, lc\}$$

Dove:

- i rappresenta il tag univoco associato al punto.
- x, y, z invece sono le coordinate nello spazio.
- lc rappresenta la lunghezza caratteristica dell'elemento in prossimità di quel punto. Si può decidere di assegnare la stessa lunghezza caratteristica per tutti i punti e per infittire la griglia definire successivamente delle proprietà, oppure si può agire direttamente su questo parametro nella definizione dei punti. La seconda strategia risulta onerosa quando i punti sono molti.

Dopo aver definito i punti che descrivono il corpo, si procede alla definizione degli elementi linea che uniscono due punti attraverso il comando *Line*, la cui sintassi è:

$$Line(i) = \{n, m\}$$

Dove:

- i è il tag univoco associato alla linea.
- n è il tag univoco del punto dal quale la linea parte.
- m è il tag univoco del punto su cui la linea finisce.

Come si può intuire dalla sintassi, la linea ha una certa direzione definita dai punti di partenza e arrivo. Questa direzione ha una sua importanza quando si definisce una linea chiusa, in quanto le linee che la costituiscono devono avere un verso coerente. Un altro tipo di linea che si può definire è una spline attraverso il comando *Spline*:

$$Spline(i) = \{n, m, p, q, \dots\}$$

Dove:

- i è il tag univoco associato alla linea.
- $n, m, p, q, \dots$ sono i tag univoci della serie di punti che definiscono la curva, che come per il comando *Line* definiscono un certo verso di percorrenza in base al primo e ultimo tag forniti.

Per la realizzazione di una linea chiusa si usa il comando *Line Loop*, il quale chiede di fornire i tag delle linee che dovrebbero comporre la linea chiusa. In questo tipo di comando è possibile combinare insieme elementi *Line* e *Spline* perché il linguaggio usato li considera elementi geometrici dello stesso tipo.

$$Line\ Loop(i) = \{n, m, p, q, \dots\}$$

Dopo aver definito una linea chiusa è possibile definire una superficie associata alla stessa attraverso il tag univoco n in ingresso.

$$Plane\ Surface(i) = \{n\}$$

A questo punto si sono fornite tutte le informazioni necessarie per la geometria e ci si interessa alla definizione della mesh associata.

1.2.2 Definizione caratteristiche della mesh

Per la definizione della mesh si possono utilizzare due approcci:

- Il primo approccio definisce il numero di punti associati ad una linea e una legge di progressione della distanza tra questi punti a partire da quello iniziale. Se si è interessati a far partire la legge di progressione dal punto finale della linea è sufficiente inserire un "-" quando si richiama il tag della linea. La sintassi per quanto descritto è:

$$Transfinite\ Line\{i\} = n\ Using\ Progression\ m$$

Dove:

- i è il tag univoco della linea sulla quale si posizionano i punti.
- n è il numero di punti che si vogliono inserire.
- m è un numero che determina la legge di progressione. A titolo di esempio se $m = 1.1$ significa che sulla linea i ci sono n nodi di calcolo con *Progression 1.1*, ovvero, che la distanza tra elementi successivi aumenta del 10% rispetto alla precedente, a partire dal nodo iniziale.

In alternativa alla progressione definita precedentemente, si può raffinare la griglia al centro di una certa linea e aumentare la distanza tra gli elementi allontanandosi da esso. Per realizzare questa progressione la sintassi rimane invariata ma anziché *Progression* si utilizza *Bump*:

$$Transfinite\ Line\{i\} = n\ Using\ Bump\ m$$

Se si è interessati ad una mesh strutturata, dopo aver dato questo comando si usa il comando *Transfinite Surface* che chiede in ingresso i tag delle *Transfinite Line* definite precedentemente.

$$Transfinite\ Surface\{i\} = \{n, m, p, q, \dots\}$$

- Il secondo approccio invece prevede di definire delle proprietà dei campi, in particolare un attrattore e una soglia. L'idea è quella di riferirsi ad una o più linee già definite e renderle attrattori. Per definire invece la soglia a partire dall'attrattore si definiscono 3 zone:
 - Una zona compresa tra la linea attrattore e una certa distanza definita come *DistMin*, nella quale la mesh sarà caratterizzata da una lunghezza caratteristica molto piccola che si manterrà costante all'interno di questa zona.
 - Una zona nella quale ci si aspetta che vi siano dei gradienti bassi. Questa zona parte da una certa distanza *DistMax* dalla parete ed è caratterizzata da una lunghezza caratteristica maggiore che si mantiene costante.
 - Una zona di buffer che si trova tra le due precedentemente definite. Questa zona è caratterizzata da una griglia avente una lunghezza caratteristica crescente in modo proporzionale alla distanza. Naturalmente l'andamento della lunghezza caratteristica è tale da raccordarsi con i valori che si trovano negli estremi (fig. 1.3).

Per definire quindi l'attrattore e le tre zone si usa la seguente sintassi:

$$Field[i] = Attractor$$

$$Field[i].EdgesList = \{n, m, p, q\}$$

In questo modo si è dato il tag univoco i al campo definito come attrattore e si sono assegnate ad esso, attraverso n, m, p, q , i tag delle linee che svolgono la funzione di campo.

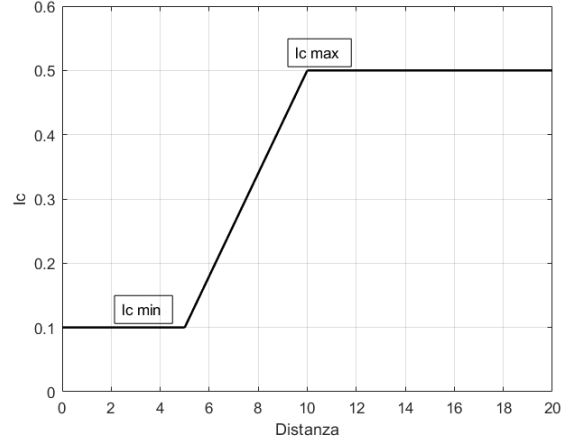


Figura 1.3: Andamento lc in funzione della distanza da attrattore

In seguito si definiscono la soglia e le proprietà delle zone che la compongono:

$$Field[i] = Threshold$$

$$Field[i].IField = n$$

Si definisce il campo i come $Threshold$ e lo si associa al campo n (attrattore). Sempre per il campo soglia i è necessario definire le lunghezze caratteristiche e le distanze che definiscono le tre zone:

$$Field[i].LcMin = \dots$$

$$Field[i].LcMax = \dots$$

$$Field[i].DistMin = \dots$$

$$Field[i].DistMax = \dots$$

Infine è necessario definire quali regole segue la mesh di background e quindi attraverso il seguente comando:

$$Background\ Field = i$$

Si fa in modo che la mesh di background segua le regole del campo i , che in questo caso è l'indice della soglia.

Quando si è interessati a studiare lo strato limite attorno ad una superficie, è possibile definire un campo in modo analogo a quanto fatto per gli attrattori, ma definendo le grandezze caratteristiche dello strato limite:

$$Field[i] = BoundaryLayer$$

$$Field[i].EdgesList = n$$

$$Field[i].hwall_n = \dots$$

$$Field[i].thickness = \dots$$

$$Field[i].ratio = \dots$$

$$BoundaryLayerField = i$$

L'ultima condizione di interesse che possiamo implementare è la condizione di superfici periodiche, infatti, quando siamo interessati ad esempio allo studio di una schiera palettata, anziché studiare l'intero campo possiamo studiare quello intorno ad un profilo e imporre che la superficie inferiore e quella superiore siano periodiche. Dal punto di vista della mesh questo comporta che le celle sul bordo superiore e inferiore del dominio devono essere caratterizzate da dei bordi esterni aventi la stessa lunghezza. In questo modo se si ripete il dominio uguale a sé stesso si avrà che la periodicità viene rispettata. Per farlo all'interno di gmsh è possibile sfruttare il comando:

$$\textit{Periodic Line}\{n\} = \{m\}$$

Il quale impone la condizione di periodicità tra le linee m ed n . Bisogna solo fare attenzione a come sono state definite queste due linee ed eventualmente porre un "-" per far coincidere gli indici e garantire che esse siano definite nello stesso verso.

Sapendo che il file di output di questo software sarà poi il file di input di un codice CFD si assegna ad ogni superficie e linea un tag che contrassegna cosa rappresenta ciascuna entità. In questo modo quando il codice leggerà il file e troverà i tag assegnati, saprà immediatamente quali condizioni applicare. Per farlo si usano due comandi:

$$\textit{Physical Surface}(i) = \textit{tag}$$

$$\textit{Physical Line}(i) = \{\textit{tag}\}$$

Codice CFD

Una volta realizzata la mesh in base alle caratteristiche del problema in esame, come descritto nel capitolo precedente, si sfrutta il risultato del software e lo si immette all'interno del codice CFD. Il codice che si intende sviluppare risolve le equazioni di **Eulero 2D** e il linguaggio di programmazione utilizzato è **Fortran 90** date le sue alte prestazioni.

2.1 Strutture

Il codice, per funzionare, ha bisogno di salvare i dati che arrivano dal file di gmsh in alcune variabili, che andranno opportunamente definite all'interno del codice. Per la loro definizione si è optato per utilizzare delle strutture, le quali permetteranno di sfruttare una notazione compatta per le operazioni.

La prima struttura che si definisce è *tipo_elemento*:

```
module struttura
implicit none

type tipo_elemento
5   integer :: nnodi, tipo_ele           ! tipo_ele da gmsh
      integer, dimension(:), pointer :: nodi ! array di nodi associato all'elemento
      integer :: entity
end type tipo_elemento
```

Al suo interno si trovano il numero di nodi che costituiscono l'elemento e il tipo di elemento che si ottiene dai tag di gmsh, che può essere ad esempio quadrilatero o triangolare. Si ha inoltre un array contenente i tag univoci dei nodi associati all'elemento stesso.

Si definisce poi la struttura *tipo_nodo*:

```
type tipo_nodo
  real(4), dimension(2) :: x
  integer :: nngbhrs, neles
5   integer, dimension(:), allocatable :: nghbr
  integer, dimension(:), allocatable :: ele
end type tipo_nodo
```


Al suo interno si trovano le coordinate x e y come elementi del vettore reale x . $nghbr$ rappresenta una lista contenente i tag dei nodi vicini e ele è vettore che contiene tag elementi che si affacciano su quel nodo.

La terza struttura è *tipo_interfaccia*:

```

type tipo_interfaccia
  integer :: e1, e2
  integer :: edge_e1, edge_e2
  integer :: nodo1, nodo2
  real(4), dimension(2) :: normal, x0
  real(4) :: length

  integer :: entity          !0 all ''interno, >0 sui bordi
  real(4), dimension(4) :: f
  integer :: int_perio       !condizioni di tipo periodico
end type tipo_interfaccia

```

Al suo interno troviamo i tag elementi associati a quella interfaccia con la convenzione che e_1 è quello di riferimento per la normale uscente. In questa struttura si ha anche l'informazione sui lati corrispondenti agli elementi separati all'interfaccia, nonché i tag nei nodi agli estremi. Si hanno anche delle quantità geometriche: il vettore contenente le componenti n_x e n_y della normale, le coordinate del baricentro x_0 e la lunghezza dell'interfaccia. Dentro questa struttura vengono anche definiti il vettore dei flussi che attraversano l'interfaccia, che ha 4 elementi perché in Eulero 2D si hanno 4 variabili conservative.

L'ultima struttura è *tipo_elemento_solido*:

```

type tipo_elemento_solido
  integer :: nnodi, tipo_ele, nlati          !numero nodi, tipo elemento e numero lati
  integer, dimension(:), allocatable :: nodi !lista dei nodi dell'elemento
  integer, dimension(:,:), allocatable :: edge !lista dei lati dell'elemento
  integer :: nghbrs                         !numero dei vicini dell'elemento
  integer, dimension(:), allocatable :: nghbr !lista dei vicini dell'elemento
  integer, dimension(:), allocatable :: interfaces !lista delle interfacce dell'elemento

  real(4) :: area                          !area dell'elemento
  real(4), dimension(2) :: x0              !coordinate baricentro dell'elemento
  real(4), dimension(4) :: ucons
  real(4) :: u, v, a, P, T, S              !grandezze primitive
  real(4), dimension(4) :: d_dt

end type tipo_elemento_solido

```

Oltre alle grandezze per le quali è presente un commento una grandezza di interesse introdotta è *ucons* che contiene al suo interno le 4 grandezze conservative:

$$ucons = \begin{Bmatrix} \rho \cdot E \\ \rho \\ \rho \cdot u \\ \rho \cdot v \end{Bmatrix}$$

Un'altra grandezza di interesse è d_dt la quale contiene le derivate temporali delle grandezze conservative:

$$d_dt = \begin{pmatrix} \frac{\partial(\rho \cdot E)}{\partial t} \\ \frac{\partial \rho}{\partial t} \\ \frac{\partial(\rho \cdot u)}{\partial t} \\ \frac{\partial(\rho \cdot v)}{\partial t} \end{pmatrix}$$

Sono presenti nel codice ancora due tipologie di strutture, le quali però hanno utilità per il pre-processing e per la generazione della matrice di connectivity a partire dal file di input, che sono aspetti che non verranno discussi nell'ambito di questo documento.

2.2 Variabili

Una volta definite le strutture, si passa alla definizione delle variabili che verranno impiegate in questo codice. Si definisce un nuovo modulo che richiama al suo interno il modulo strutture e presenta le seguenti variabili:

```

Module Variabili
use strutture
Implicit none
save
5
Integer :: k, kinf, kout, kfinal
real(4) :: gam, ga, gb, gc, gd, ge, gf, gg, gh, gi, gj
real(4) :: time, dt
real(4) :: CFL
10 real(4) :: periodo
character(100) :: mesh_file
real(4) :: ttotin, ptotin, alpha, pexit, machin
integer :: nnodi, ninterf, nele, nele_interni, nele_bordi, nentity, ndim, neqs
real(4), dimension(4) :: norm2_residuals, norminf_residuals
15
type(tipo_nodo), dimension(:), allocatable :: nodo
type(tipo_interfaccia), dimension(:), allocatable :: interf
type(tipo_elemento), dimension(:), allocatable :: ele_all      !1D e 2D
20 type(tipo_elemento_solido), dimension(:), allocatable :: ele      !2D
type(tipo_bordo), dimension(:), allocatable :: ele_bordi      !1D di bordo

type(tipo_entity), dimension(6) :: entita

25 integer, dimension(:), allocatable :: nodi_vis, nodi_agg_vis
integer :: nnodi_vis, index_periol

End module

```

Al suo interno si trovano una serie di variabili che non fanno uso delle strutture precedentemente definite e sono:

- k , $kinf$, $kout$, $kfinal$ gestiscono i passi temporali e le frequenze di output: k è il passo temporale di integrazione, $kinf$ la frequenza di output della soluzione a schermo, $kout$ ogni quanti passi salvare la soluzione e $kfinal$ rappresenta l'ultimo passo temporale.

- I diversi g riportati non sono altro che rapporti che coinvolgono γ e che servono per scrivere in modo compatto le varie equazioni che lo coinvolgono.
- *periodo* rappresenta il passo lungo y ed è attivo solo quando si ha un problema di tipo periodico.
- *mesh_file* è una stringa che conterrà il nome del file che viene passato in input.
- Le condizioni al contorno per il problema in esame.
- Il numero delle varie entità geometriche, dimensioni ed equazioni.
- I vettori contenenti la norma due e infinito dei residui.

Vi sono poi definite una serie di variabili che fanno uso delle strutture in cui verranno allocate tutte le informazioni sulla mesh che verranno poi sfruttate nelle subroutine successivamente definite.

2.3 Main

Una volta descritti i moduli del codice, si passa a quello che è il cuore vero e proprio del codice: il main nel quale vengono richiamate le diverse subroutine, che hanno lo scopo di valutare le grandezze di interesse e risolvere il campo. Dato che il main al suo interno richiama più funzioni, si è deciso di riportarlo nella sua interezza e poi di soffermarsi su ogni aspetto descrivendo dettagliatamente le diverse subroutine.

```
Program test2d
  Use Variabili
  Implicit none

5      !call system("rm *.plt") !ATTENZIONE: cancella tutti i file plt presenti nella
      !cartella! (su linux)
  call system("del *.plt") !ATTENZIONE: cancella tutti i file plt presenti nella
      !cartella! (su windows)

10     call input

  call leggi_gmsh

  call processa_elementi

15     !Verifica delle caratteristiche geometria per elemento 10 e interfaccia 10
  write(*,*) 'ele(10)%x0 = ', ele(10)%x0
  write(*,*) 'ele(10)%area = ', ele(10)%area
  write(*,*) 'interf(100)%length = ', interf(100)%length
  write(*,*) 'interf(100)%normal = ', interf(100)%normal

20     call init

  call write_tecplot_file(0,0.) !primo parametro determina il nome del file , secondo
      !parametro determina la traslazione lungo y

25     time=0.

  do k=1,kfinal
```

```

30      call compute_dt
      call compute_fluxes
      call integ_time

      time=time+dt

35      if(mod(k, kinf).eq.0) then
          write(*,*) ', '
          write(*,*) '*****'
          write(*,*) 'k,time,dt = ',k,time,dt
          call compute_norm_residuals

40      end if

      if(mod(k, kout).eq.0) then
          call write_tecplot_file(k,0.)
45      end if

50  end do

End Program

```

Il primo elemento che si incontra è *call system("del *.plt")*, questo comando si occupa di eliminare tutti i file nella cartella con estensione *.plt*, che è l'estensione in cui viene salvata la soluzione. Questo comando è molto utile perché quando si fanno più simulazioni, cambiando alcuni dettagli come le condizioni o mesh, non viene a crearsi confusione con i file contenenti le soluzioni precedenti.

2.3.1 Subroutine input

La prima subroutine che richiama attraverso *call* è *input* che è dedicata alla lettura del file di input e di due file contenenti le condizioni al contorno di ingresso e uscita.

```

subroutine input !subroutine per lettura degli input
use Variabili

implicit none

5      ndim=2 !numero dimensioni problema
      neqs=4 !numero equazioni

      open(unit=1,file='input.txt')
10     read(1,*)
      read(1,*)
      read(1,*) mesh_file
      read(1,*)
      read(1,*)
15     read(1,*) kfinal
      read(1,*)
      read(1,*)
      read(1,*) kinf
      read(1,*)
20     read(1,*)
      read(1,*) kout
      read(1,*)
      read(1,*)
      read(1,*) CFL
25     close(1)

      open(unit=1,file='inlet.txt')
      read(1,*)

```

```

30  read(1,*)
    read(1,*) ttotin
    read(1,*)
    read(1,*)
    read(1,*) ptotin
35  read(1,*)
    read(1,*)
    read(1,*) alpha
    read(1,*)
    read(1,*)
40  read(1,*) machin
    close(1)

    alpha=alpha*(4.*atan(1.0))/(180.)

45  open(unit=1,file='outlet.txt')
    read(1,*)
    read(1,*)
    read(1,*) pexit
    close(1)

50  end subroutine

```

Viene subito definito il numero di dimensioni e di equazioni del problema, che nell'ambito di questo codice è definito ed è fisso. Si ha poi la lettura di un file di testo dal quale si leggono le informazioni e si allocano le variabili corrispettive. Il file *input.txt* contiene al suo interno:

- *MESH FILE* il nome del file contenente la mesh.
- *KFINAL* il numero massimo di passi temporali da fare.
- *KINF* la frequenza di print della soluzione a schermo.
- *KOUT* la frequenza salvataggio in memoria della soluzione.
- *CFL* la condizione CFL.

Il file *inlet.txt* contiene al suo interno le condizioni al contorno da imporre all'inlet. Queste condizioni sono diverse a seconda che il flusso sia subsonico o supersonico, infatti a seconda di ciò il numero di linee caratteristiche entranti nel dominio cambiano e quindi anche il numero di condizioni. Nel caso di Eulero 2D si ha una condizione in più rispetto al caso unidimensionale, infatti è necessario fornire l'angolo associato al flusso in ingresso. Nel file saranno presenti le seguenti condizioni al contorno:

- T_{IN}° : temperatura totale del flusso in ingresso.
- P_{IN}° : pressione totale del flusso in ingresso.
- α : angolo flusso in ingresso espresso in gradi.
- M_{IN} : Mach flusso in ingresso, se l'ingresso è subsonico questo valore viene ignorato nel calcolo, viene usato solo per inizializzazione.

Per quanto riguarda il file *outlet.txt* esso contiene la condizione di uscita in termini di pressione statica che viene ignorata nel caso di condizione supersonica per le stesse considerazioni dell'inlet. Questa subroutine fornisce l'informazione sulla mesh da utilizzare, le informazioni per la gestione di output e memoria in termini di passi temporali e le condizioni al contorno.

2.3.2 Subroutine leggi_gmsh

Questa subroutine ha lo scopo di aprire il file *.msh* e riempire i vettori precedentemente definiti, definendo la connectivity nonché tutti gli elementi, i nodi associati e i rispettivi vicini.

```

subroutine leggi_gmsh  ! apre il file e costruisce matrice di connectivity
use strutture
use variabili
implicit none

5
integer :: i,j,ntags,jj, tipo_ele ,temp
integer ,dimension(20):: tempvet
integer ,dimension(11):: leggiint
character :: car
10 character(300):: stringa
logical :: trovato
real(4):: machinf2 ,temp_real

100 format(A)

15

write(*,*) 'Opening mesh file: ', mesh_file

20 open( unit=1, file=mesh_file )

read(1,*) car
read(1,*) car
read(1,*) car
25 read(1,*) car

read(1,*) nnodi

allocate( nodo( nnodi ) )

30
do i=1,nnodi
    read(1,*) j, nodo(i)%x(1), nodo(i)%x(2), temp_real
end do

35 read(1,*) car
write(*,*) 'Reading nodes ...          OK'

! *****
! Lettura elementi
40 read(1,*) car
read(1,*) nele

nentity=0

45 nele_bordi=0
nele_interni=0
allocate( ele_all( nele ) )

do i=1,nele
50
    read(1,100) stringa

    read(stringa,*) j, tipo_ele , ntags

55
    if( tipo_ele .eq. 1 ) then          !SEGMENTO
        nele_bordi=nele_bordi+1
        ele_all(i)%nnodi=2
        allocate( ele_all(i)%nodi( ele_all(i)%nnodi ) )
    elseif( tipo_ele .eq. 2 ) then      !TRIANGOLO LINEARE
60        nele_interni=nele_interni+1
        ele_all(i)%nnodi=3
        allocate( ele_all(i)%nodi( ele_all(i)%nnodi ) )
    elseif( tipo_ele .eq. 3 ) then      !QUADRILATERO LINEARE

```

```

        nele_interni=nele_interni+1
65     ele_all(i)%nnodi=4
        allocate (ele_all(i)%nodi(ele_all(i)%nnodi))
    else
        write(*,*) 'Element ', i, ' UNKNOWN!!! '
        stop
70     end if

    read(stringa,*)j, ele_all(i)%tipo_ele, ntags, ele_all(i)%entity, leggiint(2:ntags), ele_all(i)%
        nodi(1:ele_all(i)%nnodi)

75
end do

! Finora ho messo tutti gli elementi in ele_al: ora divido gli interni dai bordi

80
allocate (ele(nele_interni))
allocate (ele_bordi(nele_bordi))

85
nele_interni=0
nele_bordi=0

do i=1, nele
90
    if (ele_all(i)%tipo_ele.eq.1) then
        nele_bordi=nele_bordi+1
        ele_bordi(nele_bordi)%nnodi=ele_all(i)%nnodi
        ele_bordi(nele_bordi)%entity=ele_all(i)%entity
95        trovato=.false.
        do j=1, nentity
            if (entita(j)%indx.eq. ele_all(i)%entity) then
                trovato=.true.
                exit
100            end if
        end do

        if (trovato.eqv.. false.) then
            nentity=nentity+1
            entita(nentity)%indx=ele_all(i)%entity
105        end if

        allocate (ele_bordi(nele_bordi)%nodi(ele_bordi(nele_bordi)%nnodi))
        ele_bordi(nele_bordi)%nodi=ele_all(i)%nodi
        ele_bordi(nele_bordi)%tipo_ele=ele_all(i)%tipo_ele
    elseif (ele_all(i)%tipo_ele.eq.2.or. ele_all(i)%tipo_ele.eq.3) then
        nele_interni=nele_interni+1
        ele(nele_interni)%nnodi=ele_all(i)%nnodi
115        allocate (ele(nele_interni)%nodi(ele(nele_interni)%nnodi))
        ele(nele_interni)%nodi=ele_all(i)%nodi
        ele(nele_interni)%tipo_ele=ele_all(i)%tipo_ele
        if (ele_all(i)%tipo_ele.eq.2) then
            ele(nele_interni)%nnghbrs=3
120        elseif (ele_all(i)%tipo_ele.eq.3) then
            ele(nele_interni)%nnghbrs=4
        end if
    end if
end do
125
read(1,*) car
write(*,*) 'Reading elements ... OK'
end subroutine

```

Tramite questa subroutine è quindi possibile accedere alle informazioni di interesse riguardante la mesh e operare quindi delle valutazioni.

2.3.3 Subroutine processa_elementi

Questa subroutine ha al suo interno tutte le procedure per allocare tutte le variabili di interesse a partire dalle informazioni note dalla subroutine precedente. La subroutine è molto complessa nonché molto lunga da riportare, pertanto si è scelto di porre l'accento solo su alcune subroutine che vengono chiamate all'interno della stessa. In particolare si è interessati alle subroutine che si occupano della valutazione delle proprietà geometriche di ogni elemento e interfaccia presente nella mesh.

```

! Calcola baricentro elemento i-esimo
do i=1,nele_interni
5      call compute_center_ele(i)
end do

10 ! Calcola area elemento i-esimo
do i=1,nele_interni
      call compute_area_ele(i)
15 end do

! Calcola lunghezza e normale interfaccia i-esima
20 do i=1,ninterf
      call compute_length_inte(i)
      call compute_normal_inte(i)
25 end do

```

compute_center_ele

Questa subroutine si occupa di valutare le coordinate del baricentro per ogni elemento facendo la media aritmetica delle coordinate dei nodi appartenenti allo stesso.

```

subroutine compute_center_ele(i)
use variabili
implicit none
integer :: i, j
5
ele(i)%x0=0.
do j=1,ele(i)%nnodi
10  ele(i)%x0=nodo(ele(i)%nodi(j))%x+ele(i)%x0
end do
15 ele(i)%x0= ele(i)%x0/ele(i)%nnodi
end subroutine

```


compute_area_ele

Questa subroutine valuta l'area di ogni elemento attraverso la regola di Erone (figure 2.1, 2.2). Gli elementi utilizzati infatti, possono essere sia dei triangoli che dei quadrilateri e di conseguenza è bene distinguere i casi attraverso delle condizioni **if**.

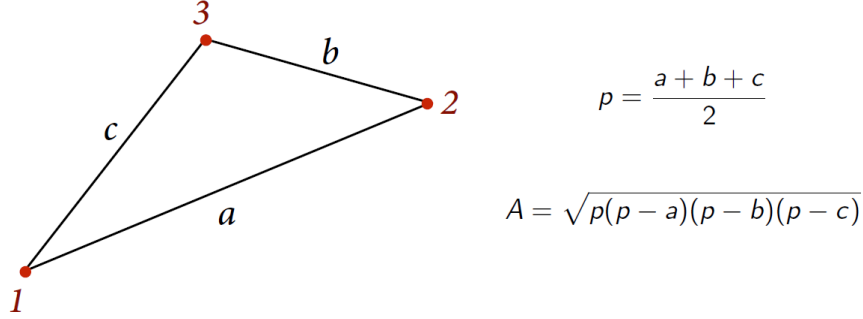


Figura 2.1: Area triangolo - Erone

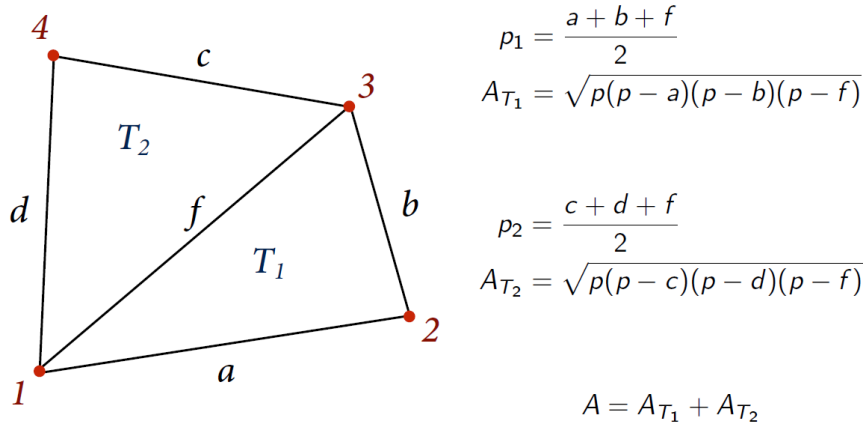


Figura 2.2: Area quadrilatero - Erone

```

subroutine compute_area_ele(i)
use variabili
implicit none
integer :: i
5 real :: a,b,c,d,f,At1,At2,p1,p2

if (ele(i)%nnodi == 3) then
10 a = sqrt((nodo(ele(i)%nnodi(1))%x(1)-nodo(ele(i)%nnodi(2))%x(1))**2 + (nodo(ele(i)%nnodi(1))%x(2)-nodo(ele(i)%nnodi(2))%x(2))**2)
b = sqrt((nodo(ele(i)%nnodi(2))%x(1)-nodo(ele(i)%nnodi(3))%x(1))**2 + (nodo(ele(i)%nnodi(2))%x(2)-nodo(ele(i)%nnodi(3))%x(2))**2)
c = sqrt((nodo(ele(i)%nnodi(3))%x(1)-nodo(ele(i)%nnodi(1))%x(1))**2 + (nodo(ele(i)%nnodi(3))%x(2)-nodo(ele(i)%nnodi(1))%x(2))**2)
15 p1 = (a+b+c)*0.5

```

```

ele(i)%area=sqrt(p1*(p1-a)*(p1-b)*(p1-c))

else
20  a = sqrt((nodo(ele(i)%nodi(1))%x(1)-nodo(ele(i)%nodi(2))%x(1))**2 + (nodo(ele(i)%nodi(1))%x
    (2)-nodo(ele(i)%nodi(2))%x(2))**2)
    b = sqrt((nodo(ele(i)%nodi(2))%x(1)-nodo(ele(i)%nodi(3))%x(1))**2 + (nodo(ele(i)%nodi(2))%x
    (2)-nodo(ele(i)%nodi(3))%x(2))**2)
    c = sqrt((nodo(ele(i)%nodi(4))%x(1)-nodo(ele(i)%nodi(3))%x(1))**2 + (nodo(ele(i)%nodi(4))%x
    (2)-nodo(ele(i)%nodi(3))%x(2))**2)
    d = sqrt((nodo(ele(i)%nodi(4))%x(1)-nodo(ele(i)%nodi(1))%x(1))**2 + (nodo(ele(i)%nodi(4))%x
    (2)-nodo(ele(i)%nodi(1))%x(2))**2)
25  f = sqrt((nodo(ele(i)%nodi(3))%x(1)-nodo(ele(i)%nodi(1))%x(1))**2 + (nodo(ele(i)%nodi(3))%x
    (2)-nodo(ele(i)%nodi(1))%x(2))**2)

    p1 = (a+b+f)*0.5
    At1 = sqrt(p1*(p1-a)*(p1-b)*(p1-f))

30  p2 = (c+d+f)*0.5
    At2 = sqrt(p2*(p2-c)*(p2-d)*(p2-f))

    ele(i)%area=At1+At2

35  end if

end subroutine

```

compute_length_inte

Questa subroutine si occupa di calcolare la lunghezza dell'interfaccia tra due elementi, per farlo si utilizza il teorema di Pitagora note le coordinate dei nodi di estremo che definiscono l'interfaccia stessa.

```

subroutine compute_length_inte(i)
use variabili
implicit none
5 integer :: i

interf(i)%length = sqrt((nodo(interf(i)%nodo1)%x(1)-nodo(interf(i)%nodo2)%x(1))**2 +
10 (nodo(interf(i)%nodo1)%x(2)-nodo(interf(i)%nodo2)%x(2))**2 )

end subroutine

```

compute_normal_inte

In questa subroutine si calcolano le componenti del versore normale all'interfaccia. Per farlo però si parte dalla valutazione del versore tangente in quanto è già nota, dalla routine precedente, la lunghezza dell'interfaccia. Il versore tangente $\hat{\tau}$ è definito infatti come:

$$\hat{\tau} = \frac{\vec{t}}{|\vec{t}|} \quad \text{con} \quad \vec{t} = \begin{pmatrix} x_2 - x_1 \\ y_2 - y_1 \end{pmatrix}$$

Essendo che il versore tangente e quello normale devono essere perpendicolari tra loro, è possibile definire una relazione tra le componenti di $\hat{t}au$ e $\hat{n}$:

$$\hat{n} \cdot \hat{t} = 0 \implies \tau_x n_x + \tau_y n_y = 0 \implies \hat{n} = \begin{Bmatrix} -\tau_y \\ +\tau_x \end{Bmatrix}$$

```

subroutine compute_normal_inte(i)
use variabili
implicit none
integer :: i
5 real :: tx, ty

call compute_length_inte(i)

10 tx = (nodo(interf(i)%nodo2)%x(1)-nodo(interf(i)%nodo1)%x(1))/interf(i)%length
ty = (nodo(interf(i)%nodo2)%x(2)-nodo(interf(i)%nodo1)%x(2))/interf(i)%length

interf(i)%normal(1) = -ty
interf(i)%normal(2) = tx
15

end subroutine

```

2.3.4 Subroutine init

Attraverso questa subroutine si fa l'inizializzazione del campo di moto, questa operazione è necessaria perchè si fornisce al solutore una distribuzione di tentativo e questa andrà poi integrata nel tempo fino a convergenza. Prima di riportare l'estratto di codice che si occupa dell'inizializzazione delle variabili, è bene fare un focus sulle equazioni in gioco e le variabili presenti.

Le equazioni di Eulero 2D possono essere scritte in diverse forme, quella conservativa differenziale può essere scritta come:

$$\frac{\partial}{\partial t}\{u\} + \frac{\partial}{\partial x}\{f\} + \frac{\partial}{\partial y}\{g\} = 0$$

Dove:

$$u = \begin{Bmatrix} \rho E \\ \rho \\ \rho u \\ \rho v \end{Bmatrix} \quad f = \begin{Bmatrix} u(P + \rho E) \\ \rho u \\ P + \rho u^2 \\ \rho uv \end{Bmatrix} \quad g = \begin{Bmatrix} v(P + \rho E) \\ \rho v \\ \rho uv \\ P + \rho v^2 \end{Bmatrix}$$

La forma di queste equazioni è dimensionale, mentre nei diversi codici CFD queste vengono adimensionalizzate. La ragione per cui si preferisce la forma adimensionale risiede nel fatto che, quando gli elementi della matrice del sistema da risolvere ha gli elementi degli stessi ordini di grandezza, il numero di condizionamento associato a quella matrice si riduce. Avere un numero di condizionamento basso è fondamentale essendo esso un fattore di amplificazione dell'errore, inoltre avere le grandezze tutte dello stesso ordine permette in fase di debugging di individuare facilmente quali variabili possono essere la causa di problemi.

Per adimensionalizzare le equazioni si definiscono delle grandezze di riferimento e si applica il rapporto:

$$\bar{x} = \frac{x}{L_{ref}} \quad \bar{y} = \frac{y}{L_{ref}} \quad \bar{P} = \frac{P}{P_{ref}} \quad \bar{T} = \frac{T}{T_{ref}}$$

Definite queste grandezze di base è possibile derivare le mancanti attraverso queste:

- Equazione dei gas perfetti:

$$P_{ref} = \rho_{ref} R T_{ref} \implies \rho_{ref} = \frac{P_{ref}}{R T_{ref}} \implies \bar{\rho} = \frac{\rho}{\rho_{ref}} = \frac{\bar{P}}{\bar{T}}$$

- Velocità di riferimento:

$$u_{ref} = \sqrt{\frac{P_{ref}}{\rho_{ref}}} = \sqrt{R T_{ref}} \implies \bar{u} = \frac{u}{u_{ref}}$$

- Velocità del suono:

$$\bar{a} = \frac{a}{u_{ref}} = \frac{\sqrt{\gamma R T}}{\sqrt{R T_{ref}}} = \sqrt{\gamma \bar{T}}$$

- Tempo:

$$t_{ref} = \frac{L_{ref}}{u_{ref}} \implies \bar{T} = \frac{t}{t_{ref}}$$

- Energia:

$$E_{ref} = u_{ref}^2 \quad \text{e} \quad E = c_v T + \frac{1}{2}(u^2 + v^2)$$

$$\Downarrow$$

$$\bar{E} = \frac{E}{E_{ref}} = \frac{c_v T}{R T_{ref}} + \frac{1}{2}(\bar{u}^2 + \bar{v}^2) = \frac{\bar{T}}{\gamma - 1} + \frac{1}{2}(\bar{u}^2 + \bar{v}^2)$$

$$\Downarrow$$

$$\bar{\rho} \bar{E} = \frac{\bar{\rho} \cdot \bar{T}}{\gamma - 1} + \frac{1}{2} \bar{\rho}(\bar{u}^2 + \bar{v}^2) = \frac{\bar{P}}{\gamma - 1} + \frac{1}{2} \bar{\rho}(\bar{u}^2 + \bar{v}^2)$$

L'energia può anche essere espressa in termini di entropia e quindi:

$$T dS = c_p dT - \frac{dP}{\rho} \implies S = c_p \log T - R \log P + S_0$$

$$\Downarrow$$

$$\bar{S} = \frac{S}{c_v} = \gamma \log \bar{T} - (\gamma - 1) \log P + S_0 \quad \text{Si impone} \quad S_0 = 0$$

Adimensionalizzate le variabili è possibile adimensionale le equazioni:

$$\frac{\partial \rho}{\partial t} + \frac{\partial \rho u}{\partial x} + \frac{\partial \rho v}{\partial y} = 0$$

A questo punto si riscrivono le variabili dimensionali in funzione di quelle adimensionali:

$$\frac{\rho_{ref}}{t_{ref}} \cdot \frac{\partial \bar{\rho}}{\partial \bar{t}} + \frac{\rho_{ref} u_{ref}}{L_{ref}} \cdot \frac{\partial \bar{\rho} \bar{u}}{\partial \bar{x}} + \frac{\rho_{ref} u_{ref}}{L_{ref}} \cdot \frac{\partial \bar{\rho} \bar{v}}{\partial \bar{y}} = 0$$

Se si ricorda la definizione di t_{ref} ci si rende conto che tutti i termini che moltiplicano le derivate sono uguali e quindi si elidono ottenendo:

$$\frac{\partial \bar{\rho}}{\partial \bar{t}} + \frac{\partial \bar{\rho} \bar{u}}{\partial \bar{x}} + \frac{\partial \bar{\rho} \bar{v}}{\partial \bar{y}} = 0$$

La conclusione a cui si arriva è che scrivere le equazione in termini dimensionali o adimensionali è indifferente in quanto le equazioni hanno la stessa forma. Se infatti si ripetesse la stessa sostituzione per le altre due equazioni si otterrebbe lo stesso esito.

Sapendo quindi come adimensionalizzare le variabili, si procede a definire un campo di moto uniforme che farà riferimento alle condizioni di inlet e outlet definite nella sezione 2.3.1. Nei casi in esame si considereranno come temperatura e pressione di riferimento $P_{IN}^o = 1$ e $T_{IN}^o = 1$, questo renderà possibile scrivere ad esempio la pressione adimensionale come:

$$\bar{P} = \frac{P}{P_{ref}} = \frac{P}{P^o} = \frac{P}{P \left(1 + \frac{\gamma-1}{2} M^2\right)^{\gamma/(\gamma-1)}} = \frac{1}{\left(1 + \frac{\gamma-1}{2} M^2\right)^{\gamma/(\gamma-1)}} = \frac{P^o}{\left(1 + \frac{\gamma-1}{2} M^2\right)^{\gamma/(\gamma-1)}}$$

Le stesse considerazioni possono essere ripetute per la temperatura totale.

```

subroutine init
use variabili
implicit none
integer :: i
5 real(4) :: rho,u,v,p,t

    gam=1.4 !rapporto dei calori specifici

10    ! variabili derivate dal rapporto dei calori specifici
    GA=gam/(gam-1.)
    GB=1./(gam-1.)
    GC=(gam+1.)/(gam-1.)
    GD=(gam-1.)/2.
15    GE=(gam+1.)/2.
    GF=sqrt(gam)
    GG=2./(gam-1.)
    GH=(gam+1.)/(2.*gam)
    GI=(gam-1.)/(2.*gam)
20    GJ=(gam-1.)/gam

    do i =1, nele_interni
25        ele(i)%P = ptotin/((1.+GD*machin**2)**GA) !valida per ptotIN = 1
        ele(i)%T = ttotin / (1. + GD * machin**2)
        ele(i)%a = sqrt(gam*ele(i)%T)
        ele(i)%u = machin*ele(i)%a*cos(alpha)
    enddo

```

```

30      ele(i)%v = machin*ele(i)%a*sin(alpha)
      ele(i)%S = gam * log(ele(i)%T) - (gam-1.)* log(ele(i)%P)

      ele(i)%ucons(2) = ele(i)%P/ele(i)%T
      ele(i)%ucons(1) = GB*ele(i)%P +0.5 *ele(i)%ucons(2)*(ele(i)%u**2 + ele(i)%v**2)
35      ele(i)%ucons(3) = ele(i)%P/ele(i)%T * ele(i)%u
      ele(i)%ucons(4) = ele(i)%P/ele(i)%T * ele(i)%v

      end do
end subroutine

```

2.3.5 Subroutine compute_dt

Questa subroutine calcola il passo di integrazione dt . Inizialmente viene assunto un valore molto grande e poi si fa un ciclo su tutti gli elementi calcolando il valore ammissibile di dt_{loc} nell'elemento. Il valore ammissibile locale viene confrontato con il valore di dt e se si ha che quello locale è minore di quello in memoria si impone:

$$dt = dt_{loc}$$

Così per tutti gli elementi fino a quando non si otterrà il valore minimo tra tutti gli elementi, infatti la condizione più restrittiva sarà quella che verrà imposta all'intero dominio di calcolo. Una volta individuato questo valore lo si riduce imponendo la condizione CFL, la quale in 2D non può essere espressa in termini di Δx ma piuttosto si approssima una lunghezza di riferimento h . Se si avesse una cella isotropa $h \propto \sqrt{Area}$, in 3D sarebbe $h \propto \sqrt[3]{Volume}$. Per quanto riguarda invece la velocità di propagazione massima in 1D sarebbe:

$$\lambda_{max} = |u| + a$$

Mentre in 2D

$$\lambda_{max} = |q| + a$$

λ_{max} rappresenta la velocità di propagazione delle onde acustiche, nel caso di onde d'urto si sta sottostimando la velocità in quanto si ha una propagazione più veloce, infatti l'urto viaggia ad una velocità superiore a quella del suono. Nella valutazione del Δt si stanno introducendo molte approssimazioni ma non è un problema in quando il valore di CFL è arbitrario e l'incertezza tiene conto delle approssimazioni introdotte, infatti maggiori le approssimazioni introdotte più restrittiva sarà la condizione. Il passo temporale è quindi:

$$\Delta t = CFL \frac{h}{|\lambda_{max}|}$$

Questo passo di integrazione viene aggiornato ad ogni iterazione.

```

subroutine compute_dt
use variabili
implicit none
real::dtloc,q, lambdamax
5 integer::i

dt = huge(1.)
10 do i = 1, nele_interni

```

```

    q = sqrt((ele(i)%u**2 + ele(i)%v**2))
    lambdamax = q + ele(i)%a
    dtloc = sqrt(ele(i)%area) / lambdamax
    if (dtloc.lt.dt) dt = dtloc
15
end do

dt = CFL * dt
20
end subroutine

```

2.3.6 Subroutine compute_fluxes

Questa subroutine si occupa della valutazione delle derivate temporali delle grandezze conservative, che andranno successivamente integrate per mezzo della prossima subroutine introdotta. In questa subroutine ne vengono chiamate al suo interno altre due che hanno lo scopo di valutare i flussi all'interfaccia tra elementi attraverso i solutori di Lax-Friedrichs locale o di Roe.

```

subroutine compute_fluxes
use variabili
implicit none
integer::i,e1,e2,j
5
do i=1,nele_interni
    ele(i)%d_dt(:)=0.
end do
10
do i=1,ninterf
    e1=interf(i)%e1
    e2=abs(interf(i)%e2)
15
    if(e1*e2.ne.0) then
        call compute_internal_flux(i)
        ! call compute_internal_flux_Roe(i)
20
    else
        call compute_boundary_flux(i)
25
    end if
    do j=1,4
        ele(e1)%d_dt(j)=ele(e1)%d_dt(j)-interf(i)%F(j)*interf(i)%length/ele(e1)%area
30
    end do

    if(e2.gt.0) then
        do j=1,4
            ele(e2)%d_dt(j)=ele(e2)%d_dt(j)+interf(i)%F(j)*interf(i)%length/ele(e2)
35
            %area
        end do
    end if
end do
40
end subroutine

```

compute_internal_fluxes

Questa routine si occupa di calcolare i flussi all'interfaccia tra elementi attraverso il solutore di Lax-Friedrichs locale. È infatti possibile, in un problema a più dimensioni, valutare i flussi alle interfacce per mezzo di una proiezione portandosi in un sistema di riferimento locale, per poi tornare a quello globale per mezzo di una rotazione. Si parta dalle equazioni di Eulero 2D:

$$\frac{\partial}{\partial t}\{u\} + \frac{\partial}{\partial x}\{f\} + \frac{\partial}{\partial y}\{g\} = 0$$

Dove:

$$u = \begin{Bmatrix} \rho E \\ \rho \\ \rho u \\ \rho v \end{Bmatrix} \quad f = \begin{Bmatrix} u(P + \rho E) \\ \rho u \\ P + \rho u^2 \\ \rho uv \end{Bmatrix} \quad g = \begin{Bmatrix} v(P + \rho E) \\ \rho v \\ \rho uv \\ P + \rho v^2 \end{Bmatrix}$$

Se si passa alla formulazione integrale:

$$\frac{\partial}{\partial t} \int_A \{u\} dA = - \int_{\partial A} (\{f\}n_x + \{g\}n_y) dS$$

Che nel caso della conservazione della massa è:

$$\frac{\partial}{\partial t} \int_A \rho dA = - \int_{\partial A} (\rho u n_x + \rho v n_y) dS$$

Dove:

$$\rho u n_x + \rho v n_y = \rho (\vec{q} \cdot \hat{n}) = \rho \tilde{u}$$

$\tilde{u}$ rappresenta la proiezione della velocità $\vec{q}$ in direzione normale all'interfaccia. Se si analizza l'equazione dell'energia:

$$\begin{aligned} \frac{\partial}{\partial t} \int_A \rho E dA &= - \int_{\partial A} [u(P + \rho E)n_x + v(P + \rho E)n_y] dS \\ &= - \int_{\partial A} [(un_x + vn_y)(P + \rho E)] dS = - \int_{\partial A} \tilde{u}(P + \rho E) dS \end{aligned}$$

Questi risultati fanno comprendere come la componente della velocità che è effettivamente presente nelle equazioni sia quella in direzione normale all'interfaccia e quindi si applica un cambio di sistema di riferimento.

$$\begin{Bmatrix} \tilde{u} \\ \tilde{v} \end{Bmatrix} = \begin{bmatrix} n_x & n_y \\ -n_y & n_x \end{bmatrix} \begin{Bmatrix} u \\ v \end{Bmatrix}$$

La matrice di rotazione del sistema di riferimento, come tutte le matrici di questo tipo, possiede la proprietà di ortogonalità per cui vale che l'inversa è pari alla trasposta. Se si è quindi interessati a passare dal sistema di riferimento ruotato locale a quello globale è sufficiente applicare la trasposta:

$$\begin{Bmatrix} u \\ v \end{Bmatrix} = \begin{bmatrix} n_x & -n_y \\ n_y & n_x \end{bmatrix} \begin{Bmatrix} \tilde{u} \\ \tilde{v} \end{Bmatrix}$$

Il solutore di Lax-Friedrichs si scrive in forma generale come:

$$F_{j+1/2} = \frac{F_L + F_R}{2} - \frac{\lambda_{max}}{2} (U_R - U_L)$$

Se si specializza la scrittura per le equazioni di massa ed energia:

$$F_{int} = \frac{\rho_L \tilde{u}_L + \rho_R \tilde{u}_R}{2} - \frac{\lambda_{max}}{2} (\rho_R - \rho_L)$$

$$F_{int} = \frac{\tilde{u}_L (P_L + \rho_L E_L) + \tilde{u}_R (P_R + \rho_R E_R)}{2} - \frac{\lambda_{max}}{2} (\rho_R E_R - \rho_L E_L)$$

Dove:

$$\lambda_{max} = \max(|u_L| + a_L, |u_R| + a_R)$$

Le due equazioni analizzate sono delle equazioni scalari e quindi invarianti rispetto al sistema di riferimento, mentre per quanto riguarda la quantità di moto è necessario modificare le equazioni, valutando i flussi nel sistema proiettato e riportarli a quelli normali successivamente. Se si vogliono proiettare le equazioni scritte nel sistema (x, y) nel nuovo sistema $(\tilde{x}, \tilde{y})$ si moltiplicano scalarmente le equazioni per i versori $\hat{n}$ e $\hat{\tau}$. In generale si ha:

$$\int_{\partial A} \vec{F} \cdot \hat{n} dS = \int_{\partial A} [\rho q (q \cdot \hat{n}) + P \hat{n}] dS = \int_{\partial A} [\rho q \tilde{u} + P \hat{n}] dS$$

Specializzando per $\tilde{x}$ e $\tilde{y}$:

$$\tilde{x} : \int_{\partial \Omega} [\rho q \cdot \hat{n} \tilde{u} + P \hat{n} \cdot \hat{n}] dS = \int_{\partial \Omega} [\rho \tilde{u}^2 + P] dS$$

$$\tilde{y} : \int_{\partial \Omega} [\rho q \cdot \hat{\tau} \tilde{u} + P \hat{n} \cdot \hat{\tau}] dS = \int_{\partial \Omega} [\rho \tilde{v} \tilde{u}] dS$$

Quindi:

$$F_{L,\tilde{x}} = P_L + \rho_L \tilde{u}_L^2$$

$$F_{L,\tilde{y}} = \rho_L \tilde{v}_L \tilde{u}_L$$

E di conseguenza:

$$F_{int,\tilde{x}} = \frac{1}{2} [P_L + \rho_L \tilde{u}_L^2 + P_R + \rho_R \tilde{u}_R^2] - \frac{\lambda_{max}}{2} [\rho_R \tilde{u}_R - \rho_L \tilde{u}_L]$$

$$F_{int,\tilde{y}} = \frac{1}{2} [\rho_L \tilde{v}_L \tilde{u}_L + \rho_R \tilde{v}_R \tilde{u}_R] - \frac{\lambda_{max}}{2} [\rho_R \tilde{v}_R - \rho_L \tilde{v}_L]$$

A questo punto è sufficiente passare da un sistema all'altro per mezzo della matrice di rotazione:

$$\begin{Bmatrix} F_{int,x} \\ F_{int,y} \end{Bmatrix} = \begin{bmatrix} n_x & -n_y \\ n_y & n_x \end{bmatrix} \begin{Bmatrix} F_{int,\tilde{x}} \\ F_{int,\tilde{y}} \end{Bmatrix}$$

```

subroutine compute_internal_flux(i)
use variabili
implicit none
integer :: i
5 real(4), dimension(4) :: uconsl, uconsr, fluxes
real(4) :: utildeA, utildeB, vtildeA, vtildeB, pa, pb, Ta, Tb, ua, ub, va, vb, lambda_max

uconsl = ele(interf(i)%e1)%ucons(:)
10 uconsr = ele(abs(interf(i)%e2)%ucons(:)

! calcolo grandezze primitive

ua = uconsl(3)/uconsl(2)
15 ub = uconsr(3)/uconsr(2)

va = uconsl(4)/uconsl(2)
vb = uconsr(4)/uconsr(2)

20 pa = (gam-1)*(uconsl(1)-0.5*uconsl(2)*(ua**2+va**2))
pb = (gam-1)*(uconsr(1)-0.5*uconsr(2)*(ub**2+vb**2))

Ta = pa/uconsl(2)
Tb = pb/uconsr(2)
25

utildea = interf(i)%normal(1)*ua + interf(i)%normal(2)*va
utildeb = interf(i)%normal(1)*ub + interf(i)%normal(2)*vb

30 vtildea = -interf(i)%normal(2)*ua + interf(i)%normal(1)*va
vtildeb = -interf(i)%normal(2)*ub + interf(i)%normal(1)*vb

! calcolo massima velocita di propagazione
35 lambda_max = max(abs(utildeA)+sqrt(gam*Ta), abs(utildeB)+sqrt(gam*Tb))

! Flussi nel sistema ruotato con metodo di Lax-Friedrichs locale
fluxes(1) = 0.5*(utildeA*(pa+uconsl(1))+utildeB*(pb+uconsr(1))) - lambda_max*0.5*(uconsr(1)-
uconsl(1))
fluxes(2) = 0.5*(utildeA*uconsl(2)+ utildeB*uconsr(2)) - lambda_max*0.5*(uconsr(2)-uconsl(2))
40 fluxes(3) = 0.5*(pa + uconsl(2)*utildeA**2 + pb + uconsr(2)*utildeB**2) - lambda_max*0.5*(
uconsr(2)*utildeB - uconsl(2)*utildeA)
fluxes(4) = 0.5*(uconsl(2)*utildeA*vtildeA+uconsr(2)*utildeB*vtildeB) - lambda_max*0.5*(uconsr
(2)*vtildeB - uconsl(2)*vtildeA)

45 !trasformazione dei flussi dal sistema locale a quello globale
interf(i)%F(1) = fluxes(1)
interf(i)%F(2) = fluxes(2)
interf(i)%F(3) = fluxes(3)*interf(i)%normal(1) - fluxes(4)*interf(i)%normal(2)
interf(i)%F(4) = fluxes(3)*interf(i)%normal(2) + fluxes(4)*interf(i)%normal(1)
50
end subroutine

```

A questo punto sono noti i flussi, si procede con l'integrazione temporale.

compute_internal_fluxes_Roe

Questa subroutine come la precedente si occupa del calcolo dei flussi all'interfaccia attraverso il metodo di Roe.

```

subroutine Roe(uL, uR, nx, ny, fluxes)
  real :: uL(4), uR(4) ! Input: conservative variables rho*[1, u, v, E]
  real :: nx, ny       ! Input: face normal vector, [nx, ny] (Left-to-Right)
  real :: fluxes(4)     ! Output: Roe flux function (upwind)
5 ! Local constants
  real :: gamma         ! Ratio of specific heat.
  real :: zero, fifth, half, one, two ! Numbers
! Local variables
  real :: tx, ty        ! Tangent vector (perpendicular to the face normal)
10 real :: vxL, vxR, vyL, vyR ! Velocity components.
  real :: rhoL, rhoR, pL, pR ! Primitive variables.
  real :: vnL, vnR, vtL, vtR ! Normal and tangent velocities
  real :: aL, aR, HL, HR    ! Speeds of sound.
  real :: RT, rho, vx, vy, H, a, vn, vt ! Roe-averages
15 real :: drho, dvx, dvy, dvn, dvt, dp, dV(4) ! Wave strengths
  real :: ws(4), dws(4), Rv(4,4) ! Wave speeds and right-eigenvectors
  real :: fL(4), fR(4), diss(4) ! Fluxes and dissipation term
  integer :: i, j

20 ! Constants.
  gamma = 1.4
  zero = 0.0
  fifth = 0.2
  half = 0.5
25  one = 1.0
  two = 2.0

! Tangent vector (Do you like it? Actually, Roe flux can be implemented
! without any tangent vector. See "I do like CFD, VOL.1" for details.)
30 tx = -ny
  ty = nx

! Primitive and other variables.
! Left state
35 rhoL = uL(1)
  vxL = uL(2)/uL(1)
  vyL = uL(3)/uL(1)
  vnL = vxL*nx+vyL*ny
  vtL = vxL*tx+vyL*ty
40 pL = (gamma-one)*( uL(4) - half*rhoL*(vxL*vxL+vyL*vyL) )
  aL = sqrt(gamma*pL/rhoL)
  HL = ( uL(4) + pL ) / rhoL
! Right state
45 rhoR = uR(1)
  vxR = uR(2)/uR(1)
  vyR = uR(3)/uR(1)
  vnR = vxR*nx+vyR*ny
  vtR = vxR*tx+vyR*ty
50 pR = (gamma-one)*( uR(4) - half*rhoR*(vxR*vxR+vyR*vyR) )
  aR = sqrt(gamma*pR/rhoR)
  HR = ( uR(4) + pR ) / rhoR

! First compute the Roe Averages
  RT = sqrt(rhoR/rhoL)
55 rho = RT*rhoL
  vx = (vxL+RT*vxR)/(one+RT)
  vy = (vyL+RT*vyR)/(one+RT)
  H = ( HL+RT* HR)/(one+RT)
  a = sqrt( (gamma-one)*(H-half*(vx*vx+vy*vy)) )
60 vn = vx*nx+vy*ny
  vt = vx*tx+vy*ty

! Wave Strengths

```

```

65      drho = rhoR - rhoL
        dp = pR - pL
        dvn = vnR - vnL
        dvt = vtR - vtL

70      dV(1) = (dp - rho*a*dvn)/(two*a*a)
        dV(2) = rho*dvt/a
        dV(3) = drho - dp/(a*a)
        dV(4) = (dp + rho*a*dvn)/(two*a*a)

!Wave Speed
75      ws(1) = abs(vn-a)
        ws(2) = abs(vn)
        ws(3) = abs(vn)
        ws(4) = abs(vn+a)

80 !Harten's Entropy Fix JCP(1983), 49, pp357-393:
! only for the nonlinear fields.
        dws(1) = fifth
        if ( ws(1) < dws(1) ) ws(1) = half * ( ws(1)*ws(1)/dws(1)+dws(1) )
        dws(4) = fifth
85      if ( ws(4) < dws(4) ) ws(4) = half * ( ws(4)*ws(4)/dws(4)+dws(4) )

!Right Eigenvectors
        Rv(1,1) = one
        Rv(2,1) = vx - a*nx
90      Rv(3,1) = vy - a*ny
        Rv(4,1) = H - vn*a

        Rv(1,2) = zero
        Rv(2,2) = a*tx
95      Rv(3,2) = a*ty
        Rv(4,2) = vt*a

        Rv(1,3) = one
        Rv(2,3) = vx
100     Rv(3,3) = vy
        Rv(4,3) = half*(vx*vx+vy*vy)

        Rv(1,4) = one
        Rv(2,4) = vx + a*nx
105     Rv(3,4) = vy + a*ny
        Rv(4,4) = H + vn*a

!Dissipation Term
        diss = zero
110     do i=1,4
        do j=1,4
            diss(i) = diss(i) + ws(j)*dV(j)*Rv(i,j)
        end do
    end do

115 !Compute the flux.
        fL(1) = rhoL*vnL
        fL(2) = rhoL*vnL * vxL + pL*nx
        fL(3) = rhoL*vnL * vyL + pL*ny
120     fL(4) = rhoL*vnL * HL

        fR(1) = rhoR*vnR
        fR(2) = rhoR*vnR * vxR + pR*nx
        fR(3) = rhoR*vnR * vyR + pR*ny
125     fR(4) = rhoR*vnR * HR

        fluxes = half * (fL + fR - diss)

end subroutine

```

2.3.7 Subroutine integ_time

Noti i flussi si può valutare la derivata delle grandezze conservative come:

$$\frac{\partial}{\partial t} \int_{\Omega_i} \vec{u}_i d\Omega_i + \int_{\partial\Omega_i} \vec{F} \cdot \hat{n} dS = 0$$

Se si approssima l'integrale con il valore medio e l'integrale di bordo come una sommatoria sul numero di lati, si può scrivere:

$$\frac{\partial}{\partial t} \vec{u}_i + \sum_{n_{lati}} \vec{F}_{ij} \cdot \hat{n}_{ij} \frac{l_{ij}}{\Omega_{ij}} = 0$$

Se si usa Eulero esplicito:

$$\frac{\partial}{\partial t} \vec{u}_i = \frac{\vec{u}_i^{k+1} - \vec{u}_i^k}{\Delta t}$$

Riarrangiando:

$$\vec{u}_i^{k+1} = \vec{u}_i^k + \Delta t \sum_{n_{lati}} \vec{F}_{ij} \cdot \hat{n}_{ij} \frac{l_{ij}}{\Omega_{ij}}$$

Tutti i termini sono noti e quindi è sufficiente aggiornare il valore delle grandezze conservative e ricalcolare le grandezze primitive attraverso quelle conservative appena aggiornate, in modo da modificare la condizione sul passo temporale per l'integrazione successiva.

```

subroutine integ_time
use variabili
implicit none
5 integer :: i, j

do i = 1, nele_interni
10 ele(i)%ucons(:) = ele(i)%ucons(:) + dt*ele(i)%d_dt(:)

ele(i)%u = ele(i)%ucons(3)/ele(i)%ucons(2)
ele(i)%v = ele(i)%ucons(4)/ele(i)%ucons(2)
15 ele(i)%p = (gam-1)*(ele(i)%ucons(1)-0.5*ele(i)%ucons(2)*(ele(i)%u**2+ele(i)%v**2))
ele(i)%T = ele(i)%P/ele(i)%ucons(2)
ele(i)%a = sqrt(ele(i)%T * gam)
ele(i)%S = gam * log(ele(i)%T) - (gam-1)* log(ele(i)%P)
20 end do
end subroutine

```

Bump - mesh strutturata

Avendo sviluppato il codice CFD si passa a discutere dei casi studio di interesse. Il primo che si analizzerà riguarda un condotto con un bump, il quale sostanzialmente rappresenta metà di un condotto convergente divergente; se ne studia solo metà a causa della simmetria. In questo studycase si inizierà il campo come uniforme e lo si farà evolvere fino a convergenza. Questo calcolo verrà ripetuto diverse volte raffinando sempre di più griglia al fine di svolgere un'analisi di convergenza della stessa.

3.1 Inizializzazione

Come discusso nell'analisi del codice CFD, è necessario fornire in ingresso al nostro codice alcuni parametri nonché il nome del file contenente la mesh. I parametri di ingresso sono:

- input.txt:
 - MESH FILE: *bump_structured_.msh*, dato che si è interessati a compiere un'analisi di convergenza questo file verrà richiamato più volte, ma ad ogni simulazione si avrà un numero crescente di nodi costituenti la griglia.
 - KFINAL: 100000
 - KINF: 1000
 - KOUT: 1000
 - CFL: 0.3
- inlet.txt:
 - $T_{IN}^{\circ} = 1$
 - $P_{IN}^{\circ} = 1$
 - $\alpha = 0^{\circ}$
 - $M_{IN} = 0.3$
- outlet.txt:

– $P_{exit} = 0.9395$, questo valore corrisponde alla pressione a $M_{isentropico} = 0.3$.

Per svolgere l'analisi di convergenza, che verrà discussa nel dettaglio più avanti, è sufficiente osservare che in questo caso non dovrebbe essere presente entropia nel campo trattandosi di Eulero e in assenza di onde d'urto, questa però invece risulterà presente e quindi potrà essere sfruttata per una valutazione quantitativa dell'errore.

3.1.1 Caso 750 nodi

Il primo caso che si analizza è caratterizzato da una griglia grezza costituita da soli 750 nodi come rappresentato in figura 3.1.

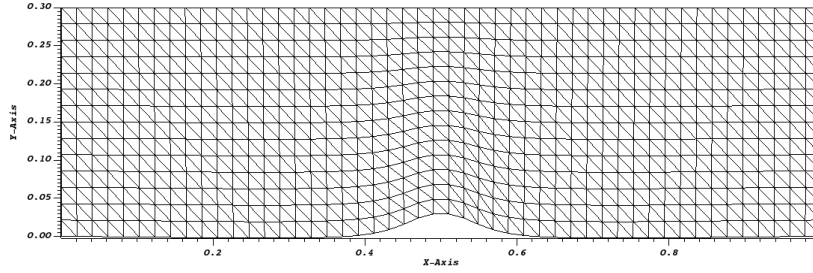


Figura 3.1: Mesh - 750 nodi

Attraverso il software Visit è possibile visualizzare l'evoluzione temporale delle grandezze di interesse, per tutti i casi descritti verranno riportate le visualizzazioni del campo di moto in termini di Mach e il campo di entropia una volta evoluti fino al raggiungimento di KFINAL o quando verrà raggiunta la convergenza delle grandezze caratteristiche.

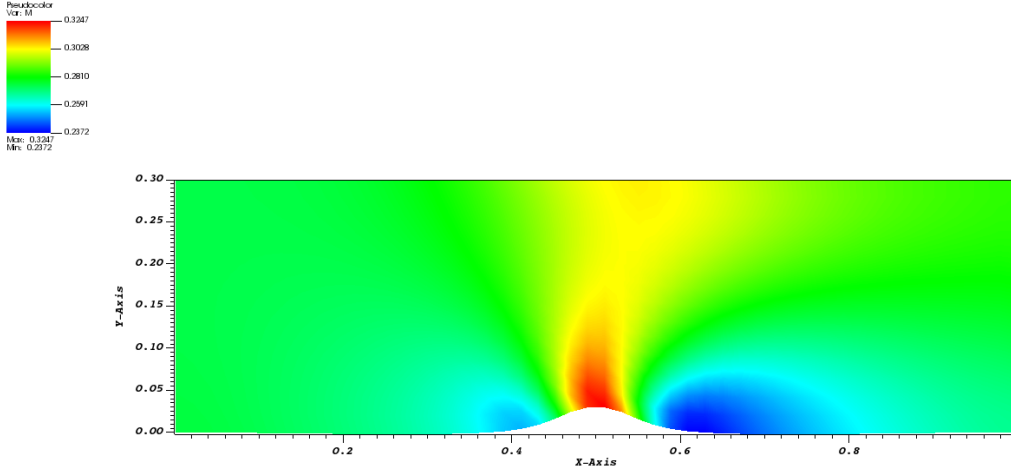


Figura 3.2: Campo di Mach - 750 nodi

Il campo di Mach ci si aspetterebbe che fosse simmetrico ma così non è dalla rappresentazione, la ricerca della simmetria del campo di moto infatti ci fornisce un modo qualitativo per comprendere che stiamo commettendo un errore, come anticipato però l'entropia ci dà la possibilità di compiere un'analisi quantitativa.

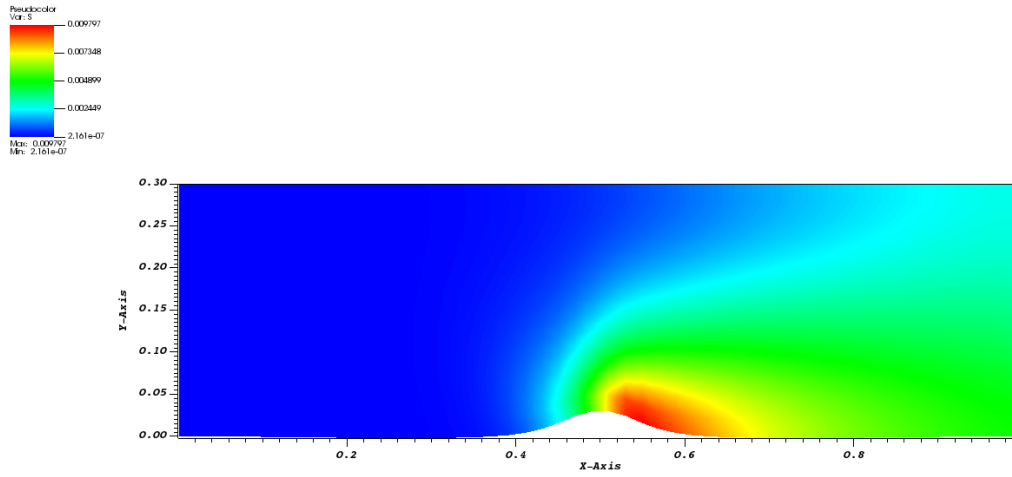


Figura 3.3: Campo di Entropia - 750 nodi

L'entropia che sarebbe dovuta essere nulla su tutto il dominio non lo è.

3.1.2 Caso 3000 nodi

Come per il caso precedente si riportano la mesh e i campi di Mach e entropia.

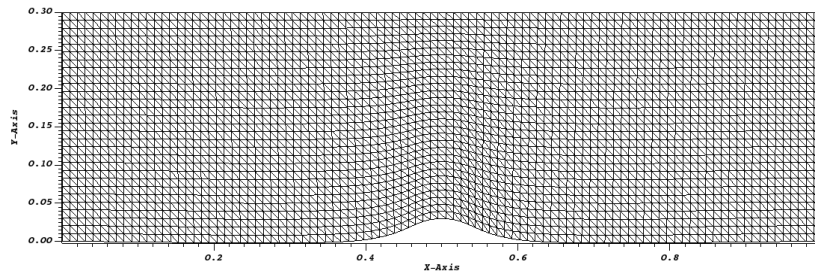


Figura 3.4: Mesh - 3000 nodi

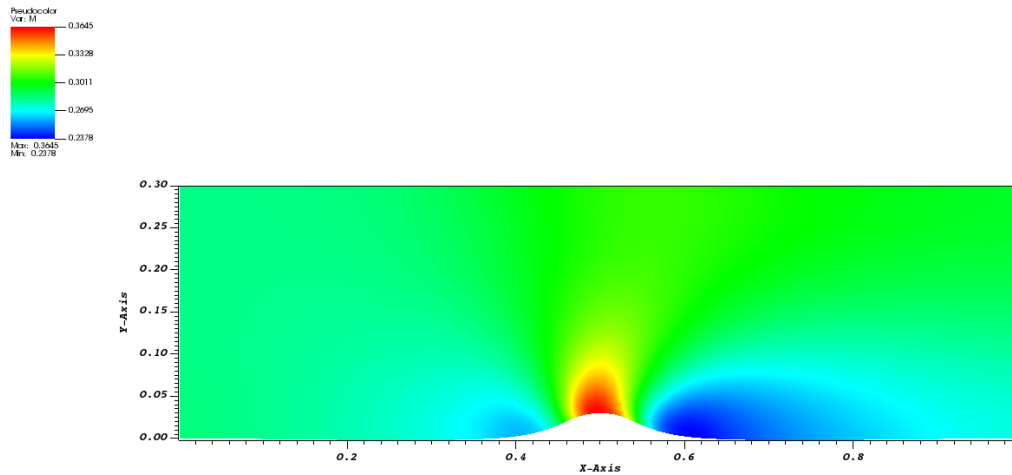


Figura 3.5: Campo di Mach - 3000 nodi

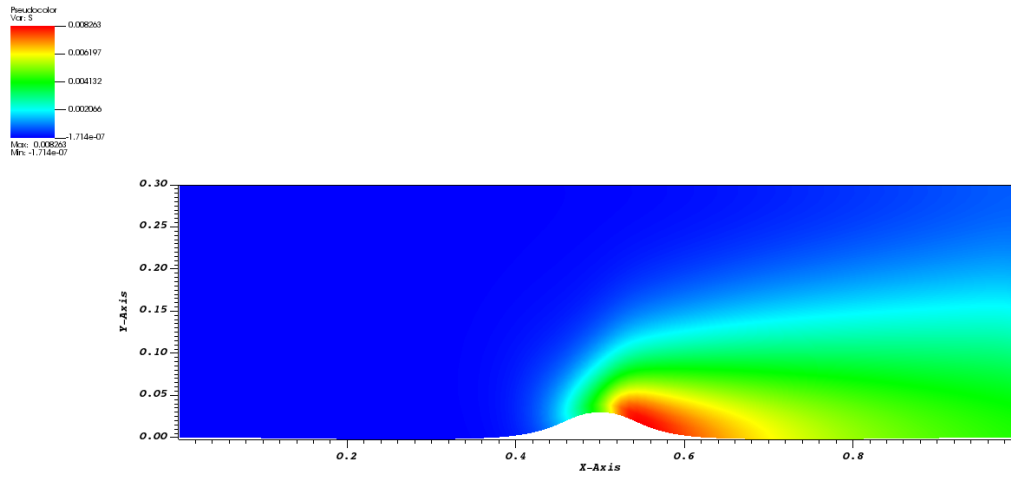


Figura 3.6: Campo di Entropia - 3000 nodi

3.1.3 Caso 6750 nodi

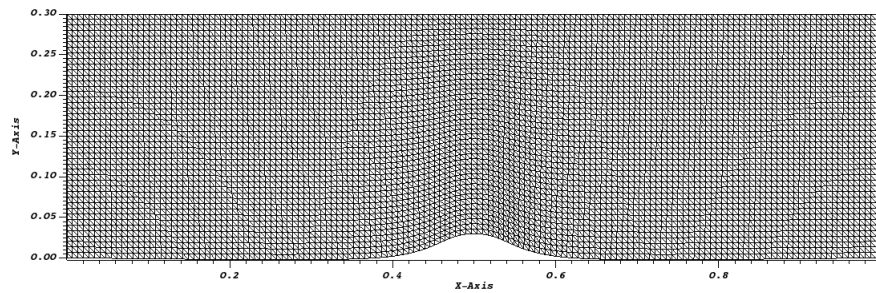


Figura 3.7: Mesh - 6750 nodi

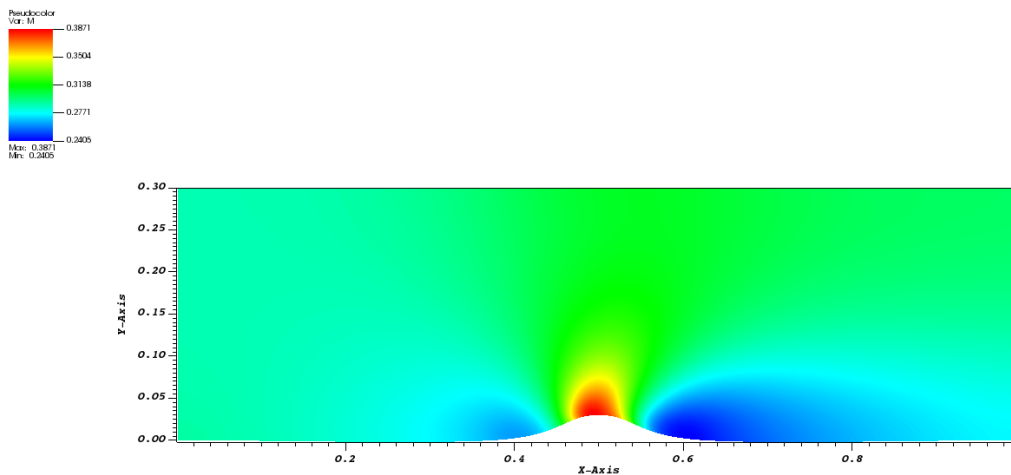


Figura 3.8: Campo di Mach - 6750 nodi

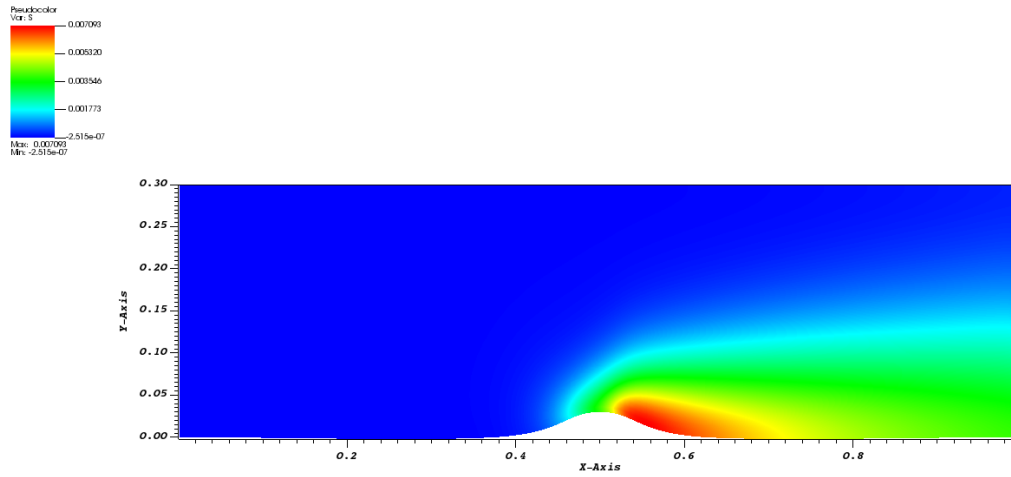


Figura 3.9: Campo di Entropia - 6750 nodi

3.1.4 Caso 12000 nodi

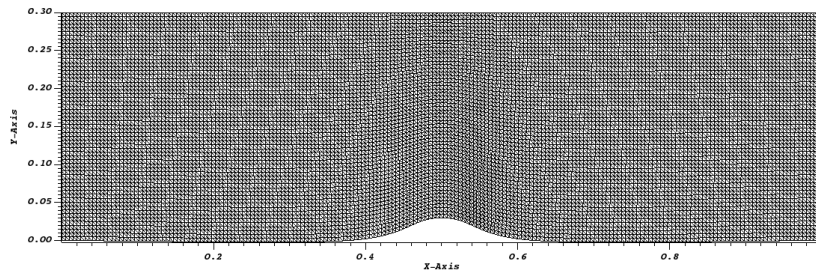


Figura 3.10: Mesh - 12000 nodi

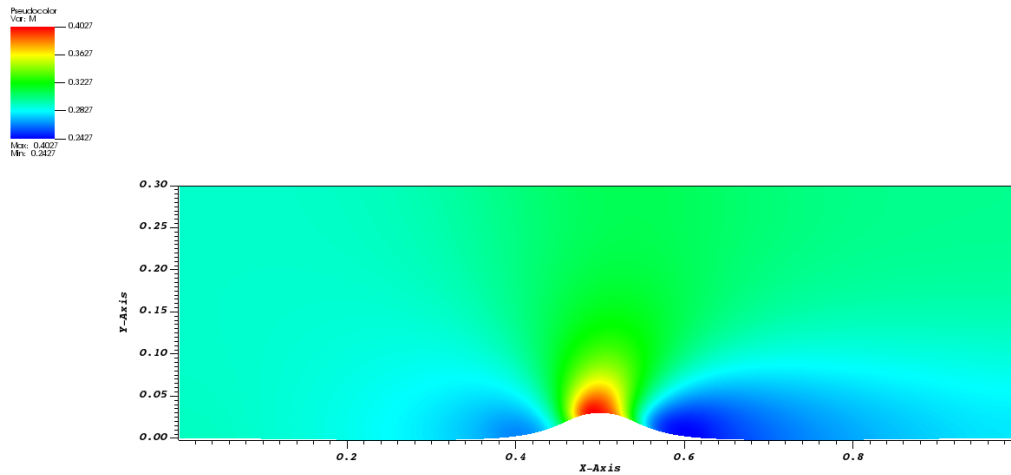


Figura 3.11: Campo di Mach - 12000 nodi

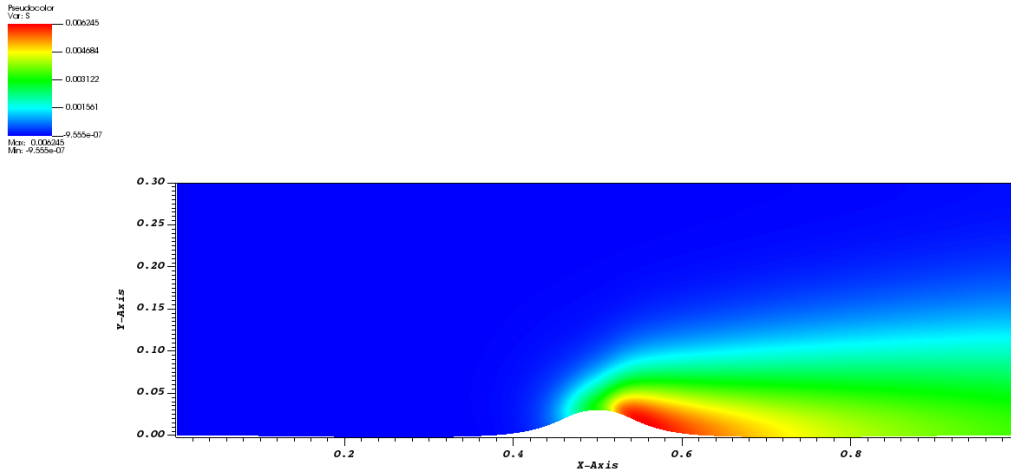


Figura 3.12: Campo di Entropia - 12000 nodi

3.2 Analisi di convergenza

Come si può osservare nelle figure rappresentanti il campo di Mach, questo diventa più simmetrico all'aumentare del numero di nodi e analogamente per il campo di entropia la zona in cui è presente un gradiente diminuisce al crescere di nodi. Per valutare quantitativamente questo risultato si valuta la norma 2 (valore quadratico medio), questa infatti ci fornisce un valore che ci consente di comprendere qual è l'errore numerico introdotto. La norma è così definita:

$$\|S\|_2 = \sqrt{\frac{\int_{\Omega} (S)^2 d\Omega}{\int_{\Omega} d\Omega}}$$

Si discretizza questa espressione sostituendo gli integrali con delle sommatorie sul numero di elementi interni:

$$\|S\|_2 = \sqrt{\frac{\sum (S)_i^2 \cdot Area_i}{\sum Area_i}}$$

Nel codice Fortran viene aggiunta quindi una funzione che ci consenta di valutare questa norma:

```

subroutine compute_norm_entropy
use variabili
implicit none
integer::i
5 real(4):: areatot, norm2_entropy

norm2_entropy = 0.
areatot = 0.

10 do i = 1, nele_interni

norm2_entropy = norm2_entropy + ele(i)%S**2 * ele(i)%area
areatot = areatot + ele(i)%area

15 end do

norm2_entropy = sqrt(norm2_entropy/areatot)
write(*,*) 'Norm2_entropy = ', norm2_entropy
end subroutine
    
```

A questo punto si riporta l'andamento della norma al variare dei numeri di nodi e, come era auspicabile e deducibile con le visualizzazioni precedentemente proposte, si ha che la norma, e quindi l'errore, decresce con il raffinarsi della griglia. Si riporta anche l'andamento della norma con la lunghezza caratteristica che segue un andamento concorde con quello in funzione del numero di nodi.

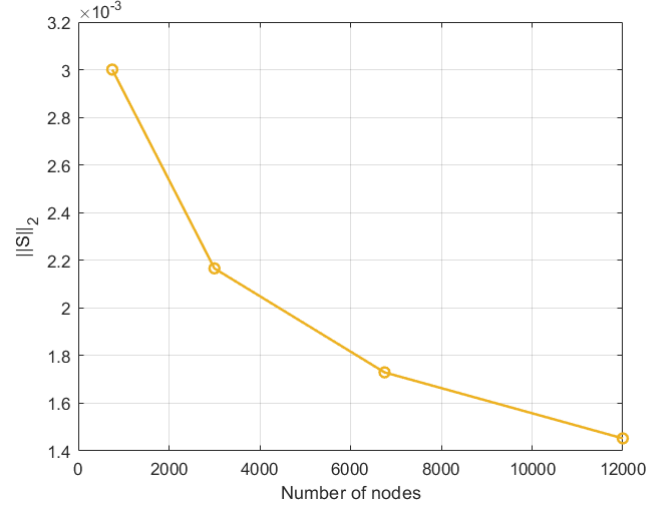


Figura 3.13: $\|S\|_2$ vs numero di nodi

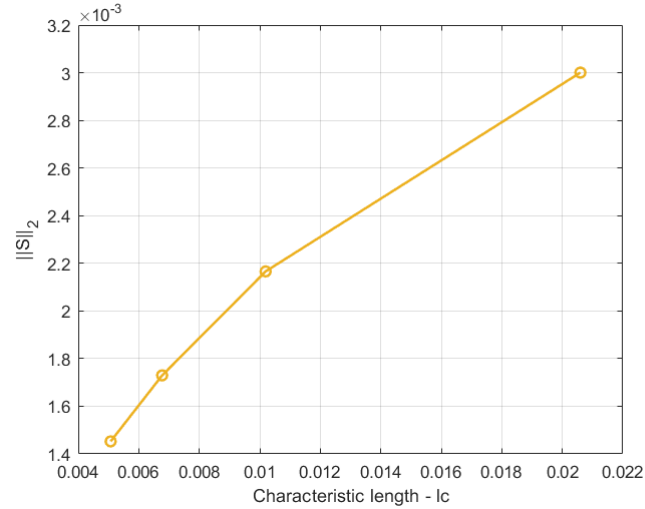


Figura 3.14: $\|S\|_2$ vs lc

In questo caso si è fatto evolvere il sistema fino al raggiungimento della convergenza, infatti per ogni griglia il numero di iterazioni è stato diverso. Se si riduce Δx , si riduce di conseguenza anche Δt e quindi con una griglia più raffinata si ha un'evoluzione temporale ridotta rispetto ad una meno fitta, inoltre si ha una diffusività minore e quindi è necessario aumentare il numero di iterazioni.

3.2.1 Lax–Friedrichs vs. Roe

Le analisi presentate nelle sezioni precedenti sono state tutte svolte valutando i flussi attraverso il metodo di Lax–Friedrichs, questo però non è l'unico applicabile, si vogliono infatti confrontare i risultati ottenuti con il metodo di Roe. Il metodo di Roe è un metodo upwind che tiene conto fortemente della fisica del problema, infatti tiene conto della propagazione ondosa delle equazioni a differenza di Lax–Friedrichs che è un metodo centrato, il metodo di Roe risulta inoltre meno dissipativo. Dalle simulazioni svolte, come in precedenza, si è valutata la norma due dell'entropia in modo da poter fare un confronto tra i due metodi:

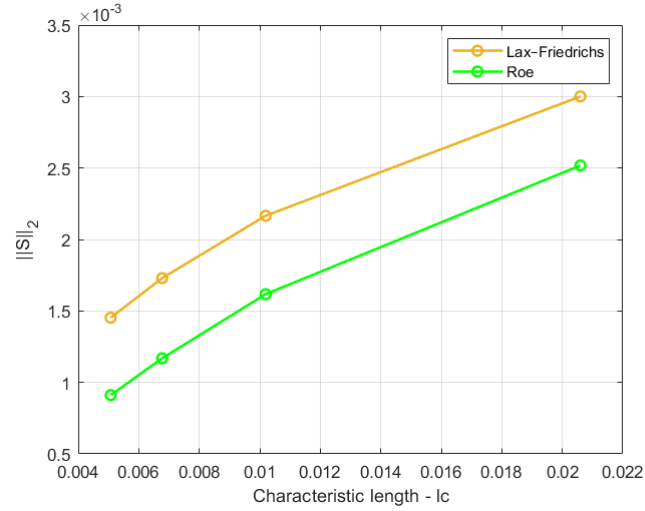


Figura 3.15: $\|S\|_2$ vs lc

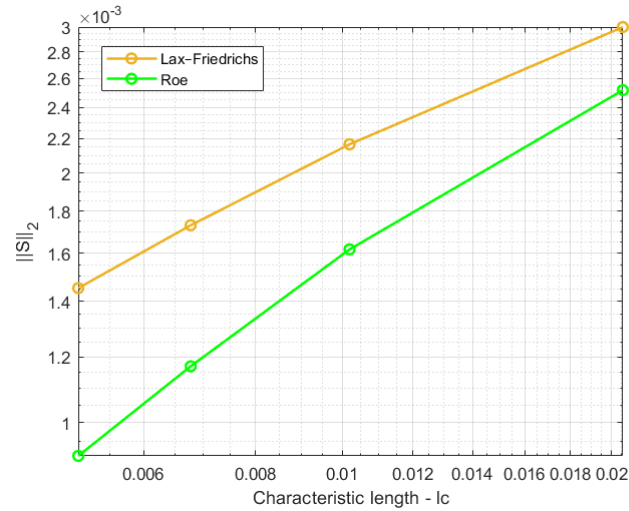


Figura 3.16: $\|S\|_2$ vs lc - log scale

Come si può osservare dalle figure 3.15 e 3.16 il metodo di Roe porta a dei risultati migliori a parità di mesh essendo che sfrutta la fisicità del problema.

3.2.2 Valutazione dell'ordine di convergenza

La valutazione della norma ci permette di comprendere quale sia l'errore di discretizzazione che si commette con la simulazione, un'altra quantità di cui si è interessati a conoscere è l'ordine di convergenza p . Infatti, vale che:

$$E = u_h - u_o = k \cdot h^p + \dots$$

Dove E è l'ordine di discretizzazione, u_h è la soluzione numerica di una certa grandezza e u_o è la soluzione esatta. Dall'altra parte dell'uguale si ha invece k una costante, h la dimensione caratteristica delle celle, p l'ordine di convergenza e $\dots$ sono termini di ordine superiore. Tramite questa espressione è possibile quindi valutare l'ordine di convergenza nonché l'errore di discretizzazione che si sta compiendo in una certa simulazione.

L'ordine di convergenza può essere al più pari a quello teorico del metodo, infatti bisogna valutare se ci si trova nelle condizioni per cui h è tale da garantire che p tenda a quello teorico. Se così non fosse si potrebbe avere un ordine di convergenza molto basso e la causa potrebbe risiedere nella griglia utilizzata. Vale infatti:

$$E = k \cdot h^p \quad \text{se } h \rightarrow 0 \implies p \rightarrow p_{teorico}$$

Se però h non tende a 0 non si possono escludere i termini di ordine superiore e quindi non si può raggiungere il valore teorico. Si analizzano in seguito tre casi rappresentativi per la valutazione dell'ordine di convergenza.

Soluzione esatta nota

Qualora la soluzione esatta fosse nota come nel caso del bump trattato precedentemente è possibile trovare p sfruttando le proprietà dei logaritmi. Immaginando di aver svolto due simulazioni caratterizzate da delle lunghezze caratteristiche diverse si può scrivere:

$$\begin{cases} E_1 &= k \cdot h_1^p \\ E_2 &= k \cdot h_2^p \end{cases} \implies \frac{E_1}{E_2} = \left(\frac{h_1}{h_2} \right)^p$$

Applicando le proprietà dei logaritmi e un opportuno cambio di base si ottiene:

$$p = \frac{\ln \left(\frac{E_1}{E_2} \right)}{\ln \left(\frac{h_1}{h_2} \right)}$$

Se è nota la soluzione esatta è quindi tutto noto e quindi è possibile stimare l'ordine di convergenza effettivo. Quando la soluzione esatta non è nota a priori, è necessario estrapolarla e per farlo si sfrutta l'estrapolazione di Richardson, la quale si divide in:

- Con ordine teorico.
- Con ordine effettivo.

Estrapolazione di Richardson con ordine teorico

Per prima cosa si ipotizza l'ordine effettivo sia pari a quello teorico e si immagina di svolgere due simulazioni aventi le seguenti lunghezze caratteristiche:

$$\begin{matrix} h_1 = h \\ h_2 = rh \end{matrix} \quad \text{con} \quad r > 1 \quad \Longrightarrow \quad \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{rh} - u_o = k \cdot r^p h^p \end{cases}$$

Si ha quindi un sistema 2x2 dove le incognite sono k e u_o . Manipolando le equazioni si ottiene:

$$u_o = \frac{r^p \cdot u_h - u_{rh}}{(r^p - 1)}$$

Da cui è possibile stimare l'errore di discretizzazione:

$$E_h = u_h - u_o$$

Si può anche valutare un indice che è bene associare alle proprie simulazioni che è il *GCI*, ovvero il grid convergence index così definito:

$$GCI = E_h \cdot F_s$$

Dove F_s è un fattore di sicurezza solitamente pari a 3.

Estrapolazione di Richardson con ordine effettivo

Se invece non si è confidenti che la propria griglia raggiunga il p teorico si può ripetere quanto fatto precedentemente aggiungendo una terza equazione in modo da introdurre p come incognita.

$$\begin{matrix} h_1 = h \\ h_2 = 2h \\ h_3 = 4h \end{matrix} \quad \Longrightarrow \quad \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{2h} - u_o = k \cdot 2^p h^p \\ E_3 &= u_{4h} - u_o = k \cdot 4^p h^p \end{cases}$$

Ottenendo:

$$p = \frac{\ln \left(\frac{u_{4h} - u_{2h}}{u_{2h} - u_h} \right)}{\ln 2}$$

$$u_o = u_h + \frac{u_{2h} - u_h}{2^p - 1}$$

$$E_h = u_h - u_o$$

$$GCI = E_h \cdot F_s$$

		Grandezze			
		p	u_o	E	CGI
Soluzione nota	Lax-Friedrichs	0.603	0.00E+00	1.45E-03	4.36E-03
	Roe	0.861	0.00E+00	9.11E-04	2.73E-03
Ordine teorico	Lax-Friedrichs	1.00	9.37E-04	5.16E-04	1.55E-03
	Roe	1.00	8.25E-04	8.59E-05	2.58E-04
Ordine effettivo	Lax-Friedrichs	0.226	5.66E-03	4.21E-03	1.26E-02
	Roe	0.350	3.49E-03	2.57E-03	7.72E-03

Tabella 3.1: Risultati Analisi di convergenza

Per questa analisi sono stati usati i seguenti valori di u_h :

$$u_h = \begin{cases} 1.45E - 03 & \text{Lax-Friedrichs} \\ 9.11E - 04 & \text{Roe} \end{cases}$$

$$u_{2h} = \begin{cases} 2.17E - 03 & \text{Lax-Friedrichs} \\ 1.62E - 03 & \text{Roe} \end{cases}$$

$$u_{4h} = \begin{cases} 3.00E - 03 & \text{Lax-Friedrichs} \\ 2.52E - 03 & \text{Roe} \end{cases}$$

Dalla tabella 3.1 è possibile osservare come Roe si comporti meglio in tutti i casi per quanto già discusso precedentemente, in particolare nel caso di soluzione esatta si può apprezzare che l'ordine di convergenza è prossimo a quello teorico.

Paletta di turbina LS59

In questo capitolo si analizza quello che è il campo di moto attorno al profilo di una paletta di turbina di alta pressione denominata LS59. Il campo di moto che si andrà a studiare, in verità, non è semplicemente quello di un profilo immerso in una corrente, ma quella dell'intera schiera di palette. Infatti, se si realizza un dominio attorno al profilo in modo da racchiuderlo e si impongono sulla parete inferiore e superiore delle condizioni di periodicità, si può ottenere con questa analisi il comportamento dell'intera schiera e quindi gli effetti di interazione tra profili. Dato che si tratta di una geometria complessa si utilizzerà in questo caso una mesh non strutturata e come per il caso del bumb verrà proposta un'analisi di convergenza della griglia ponendo a confronto il metodo di Lax–Friedrichs e quello di Roe.

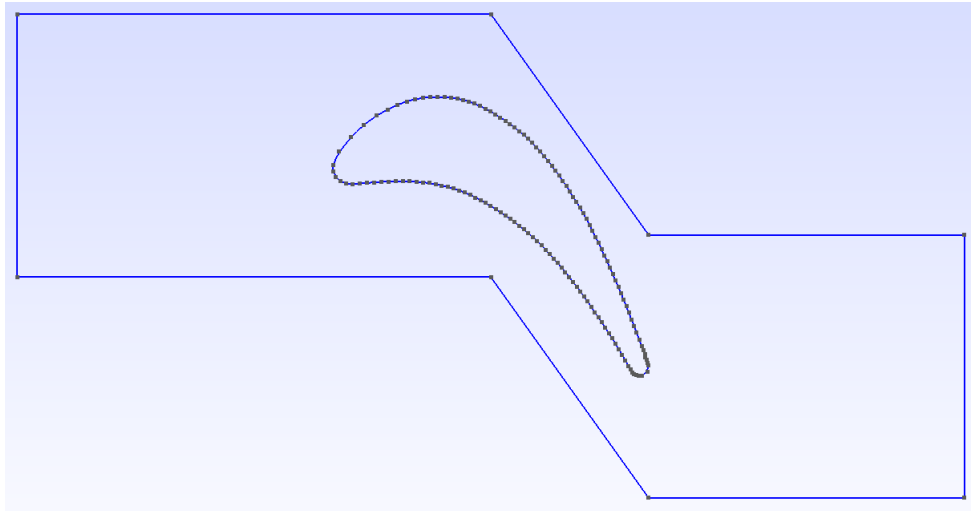


Figura 4.1: Dominio computazionale LS59

4.1 Inizializzazione

I parametri di ingresso sono:

- input.txt:
 - *LS59.msh*, dato che si è interessati a compiere un'analisi di convergenza questo file verrà richiamato più volte, ma ad ogni simulazione si avrà un numero crescente di nodi costituenti la griglia.
 - KFINAL: 300000
 - KINF: 1000
 - KOUT: 1000
 - CFL: 0.3
- inlet.txt:
 - $T_{IN}^\circ = 1$
 - $P_{IN}^\circ = 1$
 - $\alpha = 30^\circ$
 - $M_{IN} = 0.5$
- outlet.txt:
 - $P_{exit} = 0.4124$, questo valore corrisponde alla pressione a $M_{isentropico} = 1.2$, infatti è più semplice imporre una condizione di uscita in termini di Mach piuttosto che di pressione. In questo caso quindi la condizione è stata imposta ipotizzando che si conservi la pressione totale e che quindi noto il Mach di uscita sia possibile valutare la pressione come:

$$P_{exit} = \frac{P^\circ}{\left(1 + \frac{\gamma-1}{2} M_{is}^2\right)^{\frac{\gamma}{\gamma-1}}}$$

Dato che il flusso in uscita ci si aspetta che sia supersonico, la condizione di uscita di pressione non dovrebbe essere imposta, questo però non vale in questa situazione, in quanto se si scompone la velocità nelle sue due componenti, la componente perpendicolare al bordo di uscita del dominio risulta subsonica.

In questo caso particolare è noto che la corrente in ingresso abbia incidenza $\alpha = 30^\circ$, e in uscita ci si aspetta che la corrente venga deflessa verso il basso per azione della paletta ottenendo $\alpha = -60^\circ$. Alla luce di queste considerazioni, al fine di ridurre il transitorio molto severo che può presentarsi quando si svolge l'analisi, si decide di imporre una condizione iniziale differente a seconda della coordinata x del dominio. In particolare:

$$\begin{cases} \alpha = 30^\circ & \text{se } x < 0 \\ \alpha = -60^\circ & \text{se } x > 1 \\ \alpha = a + bx & \text{se } 0 \leq x \leq 1 \end{cases}$$

In questo caso dato che non è noto come varia α con la coordinata x si è scelto di ipotizzare un'approssimazione lineare dove le costanti sono:

$$a = 30^\circ \quad b = -90^\circ \implies \alpha = a + bx = 30 - 90x \quad \text{se } 0 \leq x \leq 1$$

In termini di codice si è quindi introdotta nella subroutine init.f90 la seguente condizione:

```

5  if (mesh_file == 'LS59.msh') then
    if (ele(i)%x0(1) <= 0) then
      alpha_init = 30*3.14/180
    elseif (ele(i)%x0(1) > 0 .and. ele(i)%x0(1) <= 1) then
      alpha_init = 30*3.14/180 - 90*3.14/180* ele(i)%x0(1)
    elseif (ele(i)%x0(1) > 1) then
      alpha_init = -60*3.14/180
    end if
    alpha = alpha_init
10 end if

```

Si riportano in seguito le visualizzazioni ottenute con il software *Visit*, tutte le analisi sono state realizzate fino a raggiungere la convergenza della griglia associata ad una certa lunghezza caratteristica.

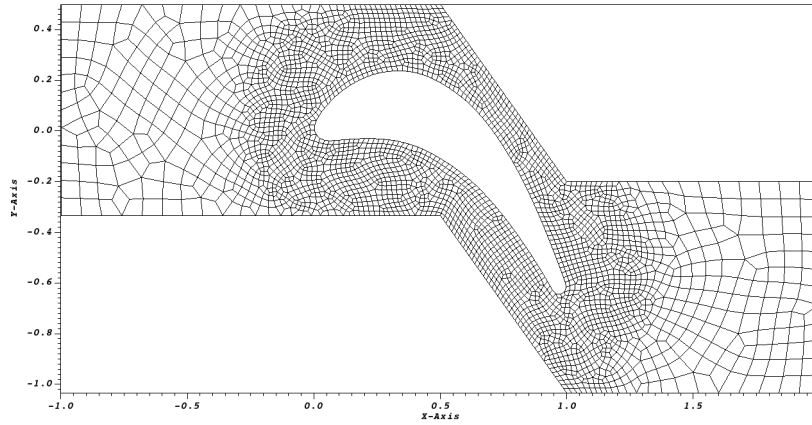


Figura 4.2: Mesh $l_c = 0.02$

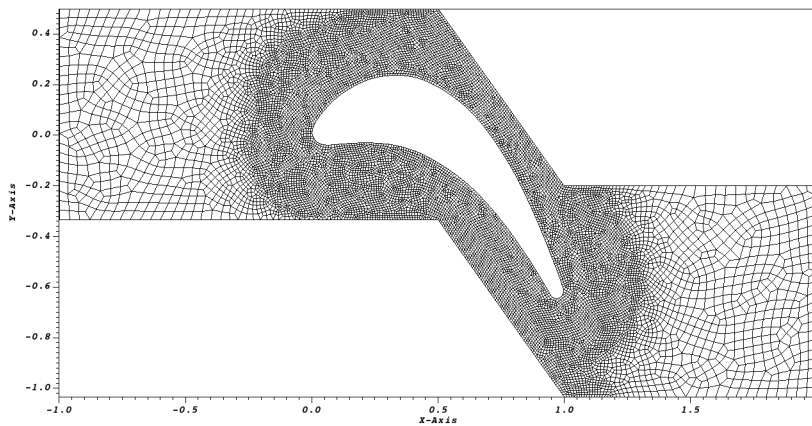


Figura 4.3: Mesh $l_c = 0.01$

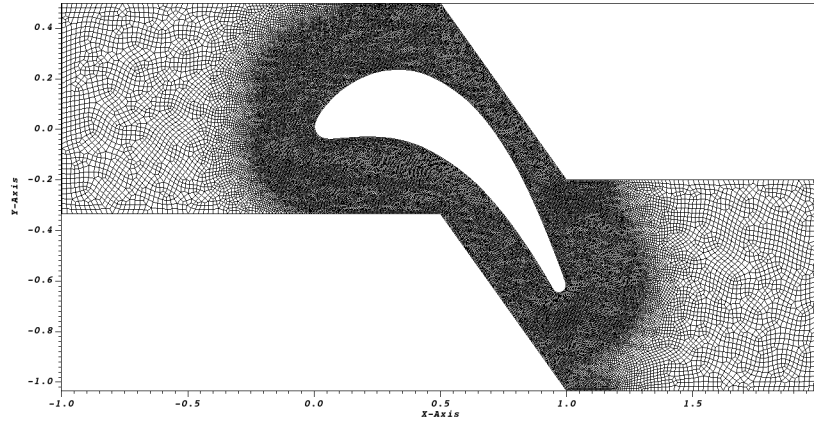


Figura 4.4: Mesh $l_c = 0.005$

4.1.1 Campo di Mach e Velocità

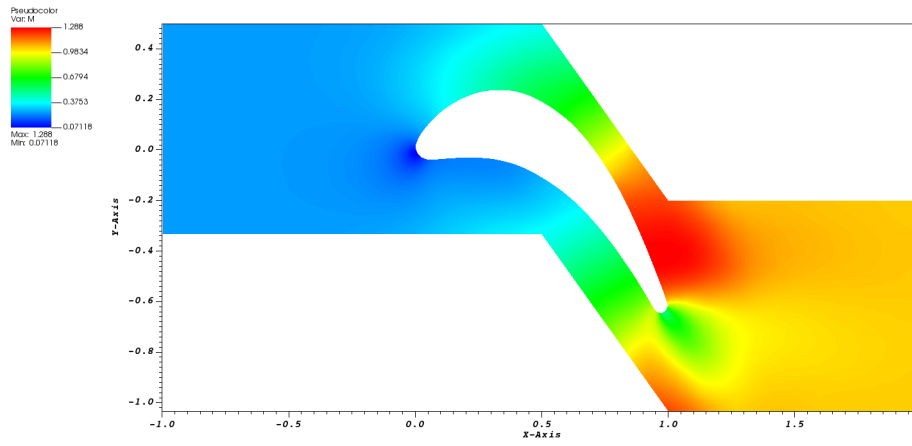


Figura 4.5: Campo di Mach $l_c = 0.02$ -Lax Friedrichs

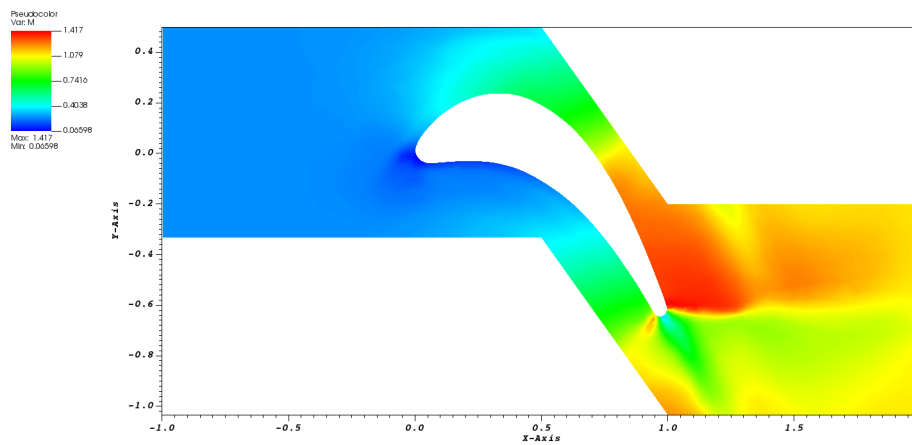


Figura 4.6: Campo di Mach $l_c = 0.02$ - Roe

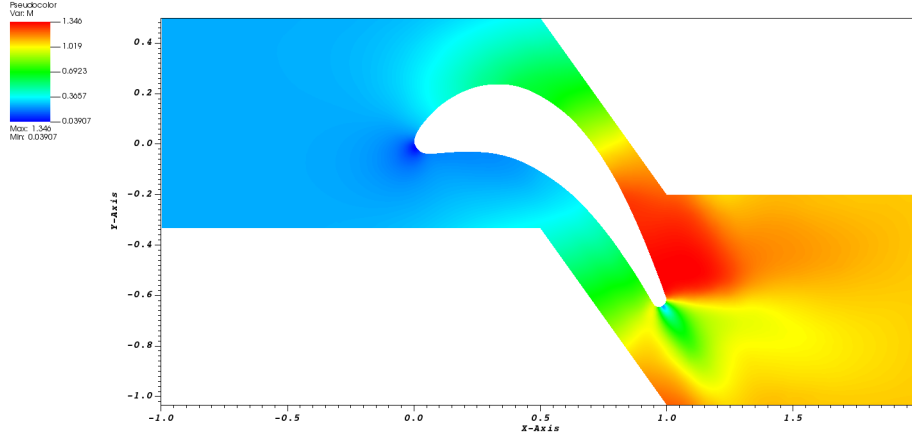


Figura 4.7: Campo di Mach $l_c = 0.01$ -Lax Friedrichs

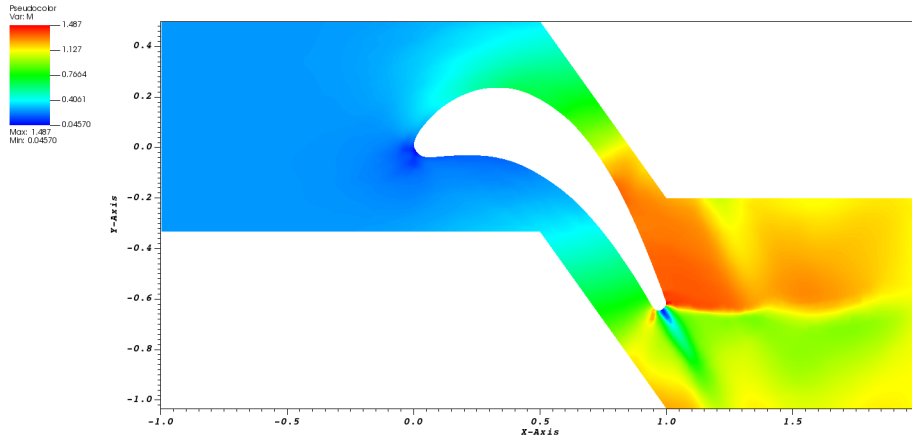


Figura 4.8: Campo di Mach $l_c = 0.01$ - Roe

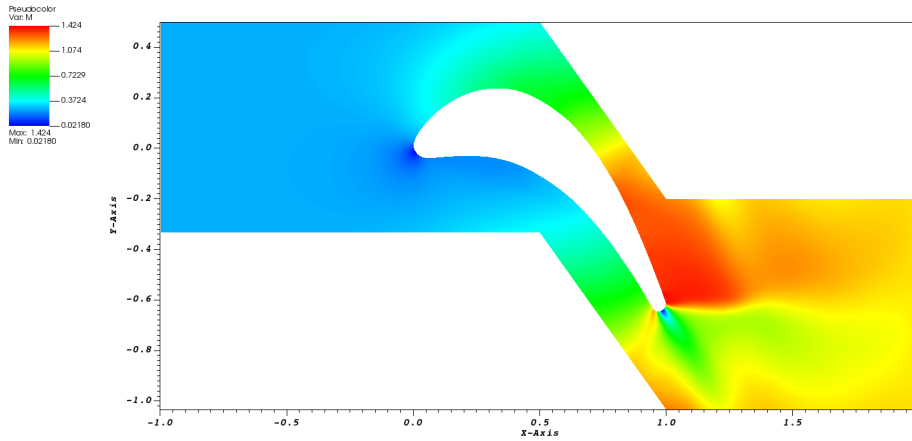


Figura 4.9: Campo di Mach $l_c = 0.005$ -Lax Friedrichs

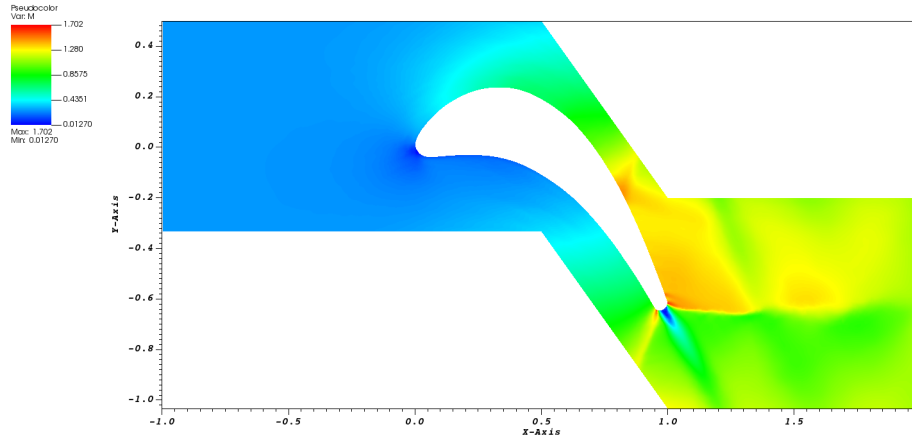


Figura 4.10: Campo di Mach $l_c = 0.005$ - Roe

Osservando le diverse rappresentazioni del campo di Mach si evince che le analisi svolte con il metodo di Roe permettono di catturare meglio i due urti presenti al bordo di fuga rispetto a quelle svolte con Lax - Friedrichs, l'ultima analisi in particolare mostra una notevole differenza tra i due campi di moto. Gli urti presenti al bordo di fuga si generano perché la corrente deve avere la stessa direzione di uscita al bordo di fuga come si può apprezzare dalle rappresentazioni del campo di velocità riportate di seguito.

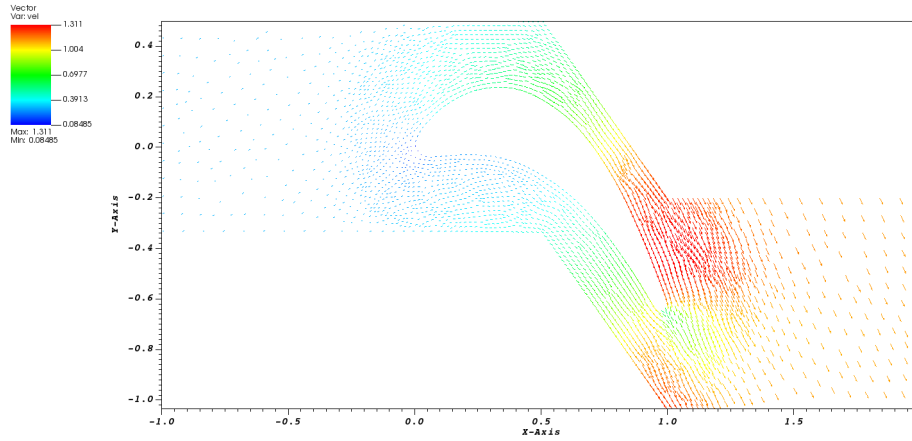


Figura 4.11: Campo di Velocità $l_c = 0.02$ -Lax Friedrichs

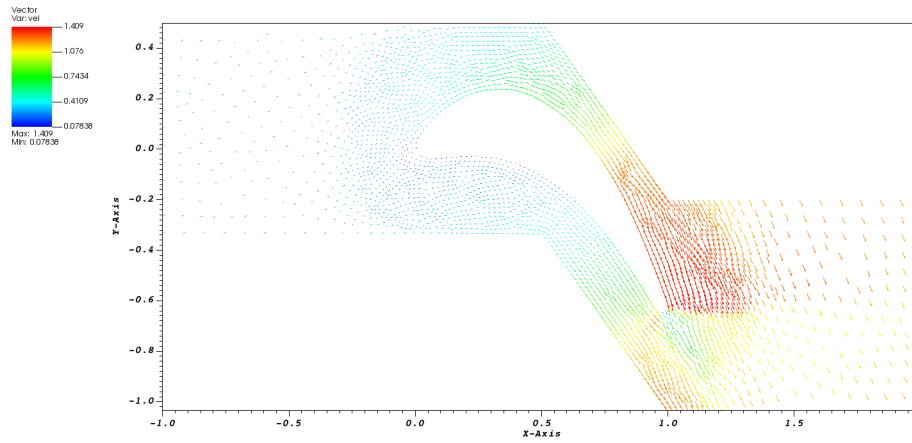


Figura 4.12: Campo di Velocità $l_c = 0.02$ - Roe

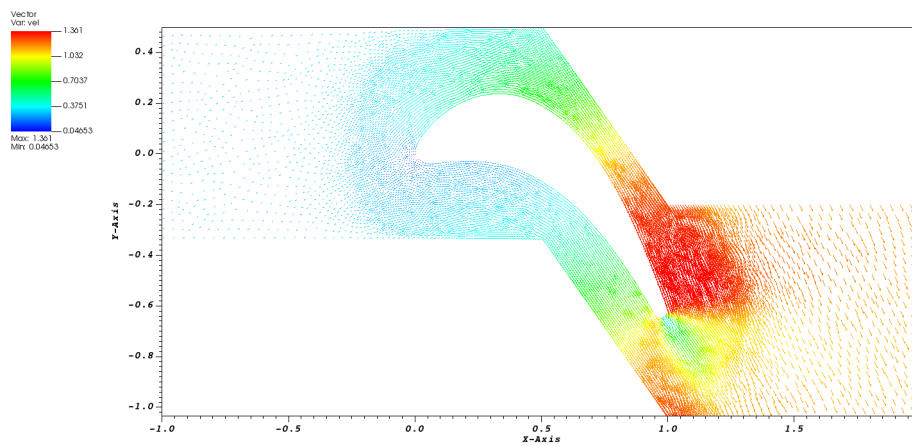


Figura 4.13: Campo di Velocità $l_c = 0.01$ -Lax Friedrichs

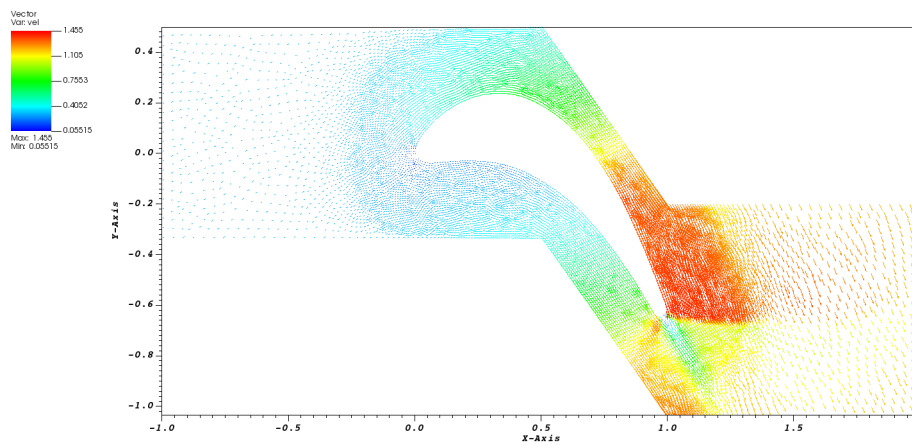


Figura 4.14: Campo di Velocità $l_c = 0.01$ - Roe

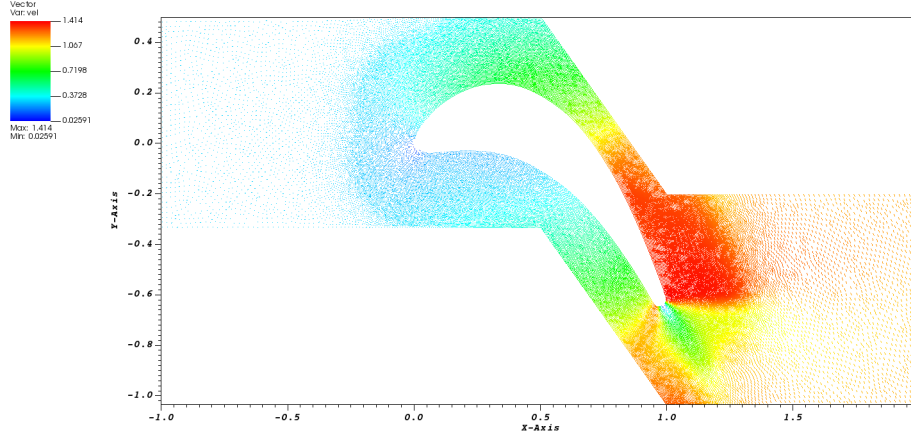


Figura 4.15: Campo di Velocità $l_c = 0.005$ -Lax Friedrichs

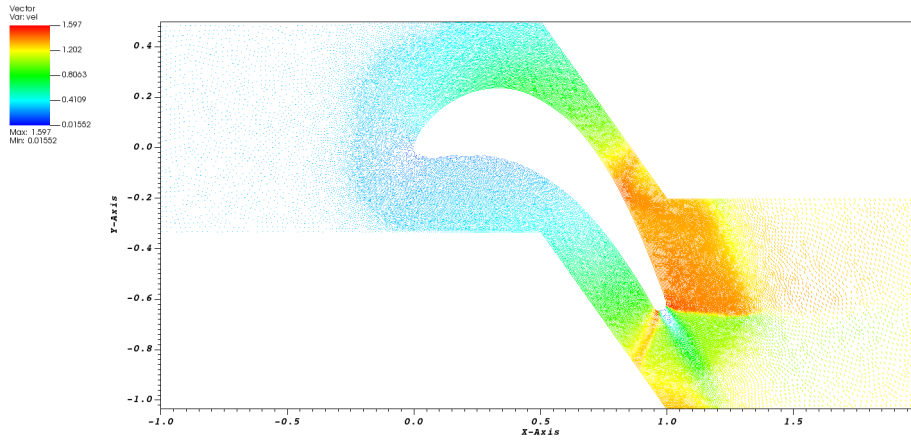


Figura 4.16: Campo di Velocità $l_c = 0.005$ - Roe

4.1.2 Campo di Entropia

Nel caso in analisi, come si è potuto osservare dal campo di moto, vi sono degli urti e quindi l'entropia non sarà nulla su tutto il dominio come avveniva invece per il caso del bump. Il fatto che l'entropia non sia più nulla non consente più di valutare l'ordine di convergenza tramite la conoscenza della soluzione esatta, in quanto questa non è più nota. Inoltre in questo studycase, dato che si sta usando un solutore che fa uso delle equazioni di Eulero, le scie e le variazioni di entropia dovute agli urti non possono essere catturati, dato che il campo di monte da cui pescano le linee di corrente non presenta sorgenti di entropia. Vale infatti che:

$$\frac{DS}{Dt} = 0$$

L'entropia che si vede nelle figure successive è solo di natura numerica, infatti aumentando la risoluzione della griglia si osserva che si riduce.

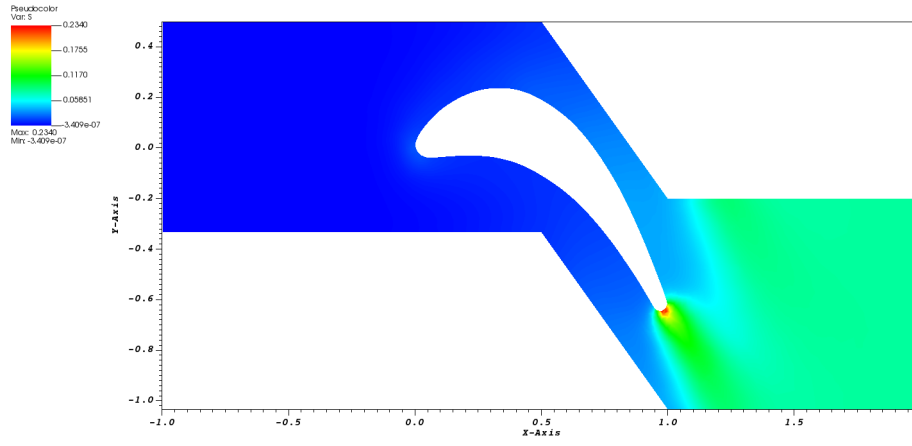


Figura 4.17: Campo di Entropia $l_c = 0.02$ -Lax Friedrichs

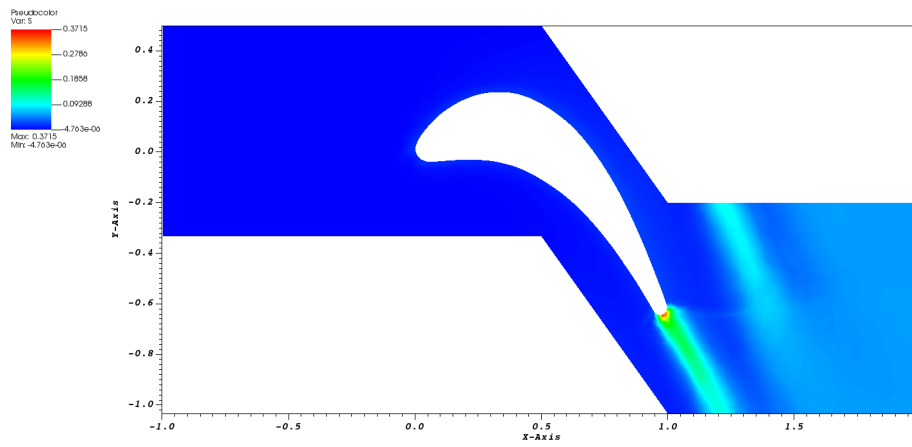


Figura 4.18: Campo di Entropia $l_c = 0.02$ - Roe

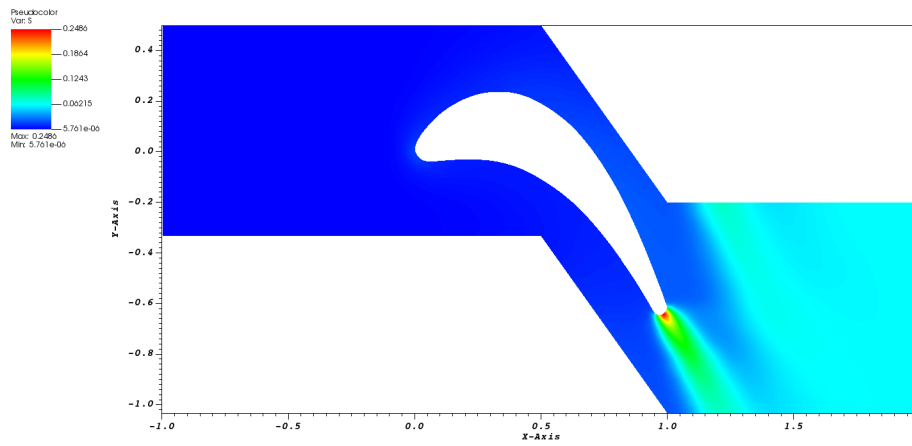


Figura 4.19: Campo di Entropia $l_c = 0.01$ -Lax Friedrichs

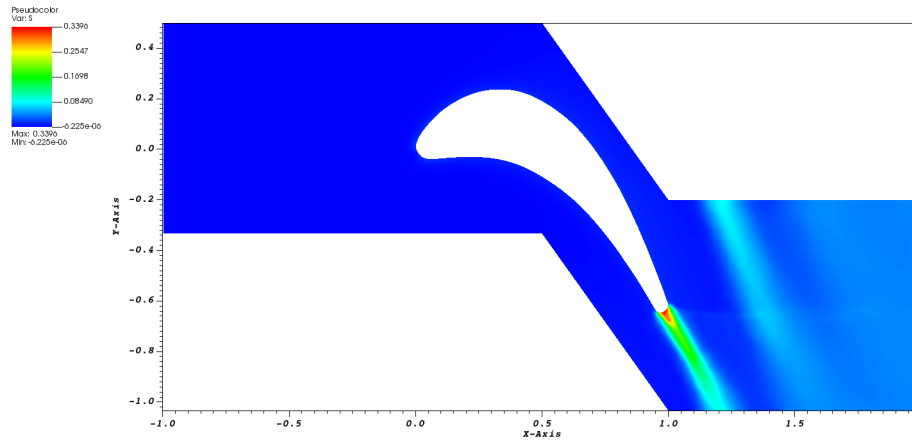


Figura 4.20: Campo di Entropia $l_c = 0.01$ - Roe

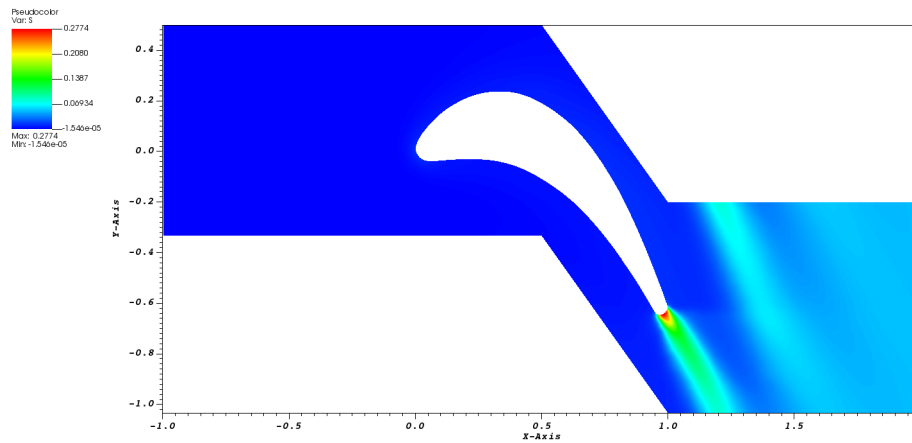


Figura 4.21: Campo di Entropia $l_c = 0.005$ -Lax Friedrichs

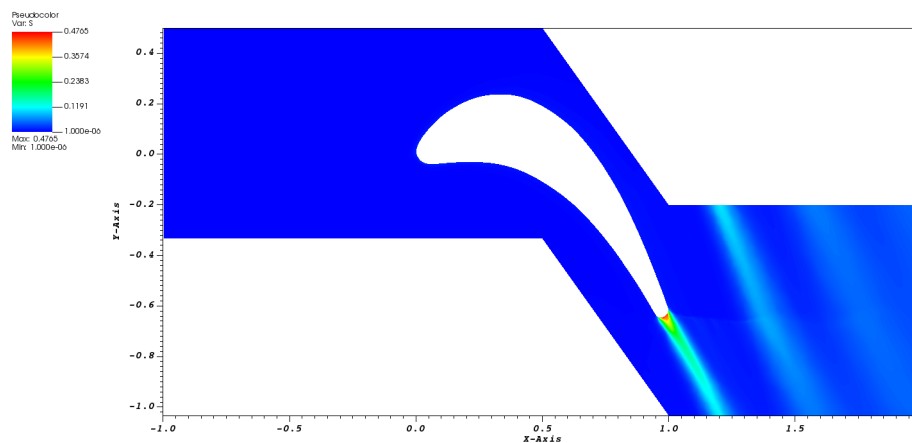


Figura 4.22: Campo di Entropia $l_c = 0.005$ - Roe

4.2 Analisi di convergenza

Per l'analisi di convergenza in questo studycase non si può più sfruttare l'informazione sull'entropia per una valutazione diretta dell'ordine di convergenza, per quello infatti attraverso l'estrapolazione di Richardson verranno valutati gli errori solo per ordine teorico ed effettivo. Comunque come evidente dalle visualizzazioni dell'entropia proposte precedentemente, è osservabile come al ridursi della lunghezza caratteristica la soluzione migliori. Si riportano quindi gli andamenti della norma 2 dell'entropia al variare della lunghezza caratteristica confrontando i due metodi. La norma è così definita:

$$\|S\|_2 = \sqrt{\frac{\int_{\Omega} (S)^2 d\Omega}{\int_{\Omega} d\Omega}}$$

Si discretizza questa espressione sostituendo gli integrali con delle sommatorie sul numero di elementi interni:

$$\|S\|_2 = \sqrt{\frac{\sum (S)_i^2 \cdot Area_i}{\sum Area_i}}$$

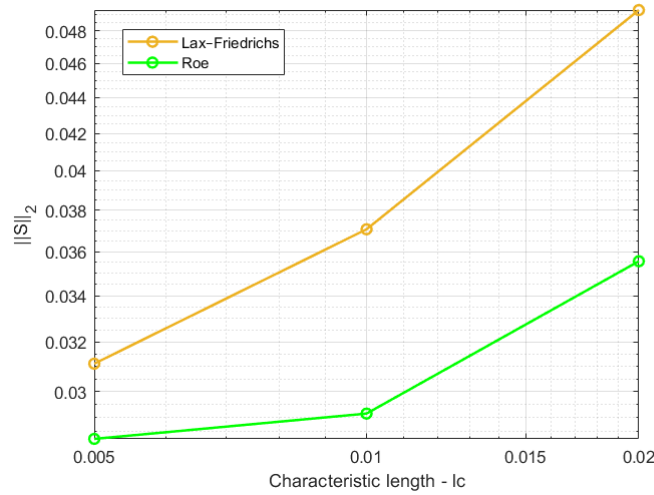


Figura 4.23: $\|S\|_2$ vs lc

Estrapolazione di Richardson con ordine teorico

Per prima cosa si ipotizza l'ordine effettivo sia pari a quello teorico e si immagina di svolgere due simulazioni aventi le seguenti lunghezze caratteristiche:

$$\begin{aligned} h_1 &= h \\ h_2 &= rh \quad \text{con } r > 1 \end{aligned} \implies \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{rh} - u_o = k \cdot r^p h^p \end{cases}$$

Si ha quindi un sistema 2x2 dove le incognite sono k e u_o . Manipolando le equazioni si ottiene:

$$u_o = \frac{r^p \cdot u_h - u_{rh}}{(r^p - 1)}$$

Da cui è possibile stimare l'errore di discretizzazione:

$$E_h = u_h - u_o$$

Si può anche valutare un indice che è bene associare alle proprie simulazioni che è il *GCI*, ovvero il grid convergence index così definito:

$$GCI = E_h \cdot F_s$$

Dove F_s è un fattore di sicurezza solitamente pari a 3.

Estrapolazione di Richardson con ordine effettivo

Se invece non si è confidenti che la propria griglia raggiunga il p teorico si può ripetere quanto fatto precedentemente aggiungendo una terza equazione in modo da introdurre p come incognita.

$$\begin{aligned} h_1 &= h \\ h_2 &= 2h \\ h_3 &= 4h \end{aligned} \implies \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{2h} - u_o = k \cdot 2^p h^p \\ E_3 &= u_{4h} - u_o = k \cdot 4^p h^p \end{cases}$$

Ottenendo:

$$\begin{aligned} p &= \frac{\ln \left(\frac{u_{4h} - u_{2h}}{u_{2h} - u_h} \right)}{\ln 2} \\ u_o &= u_h + \frac{u_{2h} - u_h}{2^p - 1} \\ E_h &= u_h - u_o \\ GCI &= E_h \cdot F_s \end{aligned}$$

In questo caso si è scelto di utilizzare la norma 2 dell'entropia come variabile u , ottenendo i risultati riportati in tabella 4.1.

		Grandezze			
		p	u_o	E	CGI
Ordine teorico	Lax-Friedrichs	1.00	2.51E-02	6.07E-03	1.82E-02
	Roe	1.00	2.58E-02	2.45E-03	7.34E-03
Ordine effettivo	Lax-Friedrichs	1.04	3.67E-02	5.63E-03	1.69E-02
	Roe	2.76	2.48E-02	1.64E-04	4.92E-04

Tabella 4.1: Risultati Analisi di convergenza

I valori p risultano superiori a quelli teorici a causa del fatto che non si è raggiunta la regione asintotica. Per valutare correttamente l'ordine di convergenza della griglia sarebbe necessario

includere i termini di ordine superiore, dei quali però non si ha nessuna informazione. Per questa analisi sono stati usati i seguenti valori di u_h :

$$u_h = \begin{cases} 3.11E-02 & \text{Lax-Friedrichs} \\ 2.82E-02 & \text{Roe} \end{cases}$$

$$u_{2h} = \begin{cases} 3.71E-02 & \text{Lax-Friedrichs} \\ 2.92E-02 & \text{Roe} \end{cases}$$

$$u_{4h} = \begin{cases} 4.93E-02 & \text{Lax-Friedrichs} \\ 3.56E-02 & \text{Roe} \end{cases}$$

4.3 Confronto con dati sperimentali

Per verificare la bontà della simulazione è possibile confrontare le distribuzioni di pressione a parete con quelle ottenute sperimentalmente. Nel caso particolare di questa paletta sono disponibili i valori del Mach isentropico a parete in funzione della coordinata x rapportata alla corda in un sistema di riferimento ruotato rispetto a quello definito nelle simulazioni, sarà quindi necessario applicare un cambio di sistema di riferimento.

Per poter confrontare i dati simulati con quelli sperimentali si procede realizzando una nuova subroutine, che salva all'interno di un file di testo i valori di pressione a parete nonché le coordinate dei punti associati.

```

Subroutine save_wall_data

  Use Variabili

5
  Implicit none
  integer :: i

10
  open (unit =1, file= 'wall_data.txt')

  do i =1, ninterf

15
    if (interf(i)%entity.eq.99) then
      write(1, *) interf(i)%x0, ele(interf(i)%e1)%P

    end if

20
  end do
  close(1)

end Subroutine

```

Dato che si sta usando una mesh non strutturata, i dati salvati non saranno ordinati secondo delle coordinate x crescenti, per ordinarli è possibile aggiungere un algoritmo di ordinamento

all'interno del codice stesso o implementare questa routine durante il post processing. Può essere interessante, inoltre, distinguere il dorso e il ventre del profilo e quindi filtrare i dati di pressione. Per realizzare questo filtro si definisce una soglia interpolando la geometria della linea media del profilo attraverso una parabola (fig. 4.24) e imponendo che le coordinate y al di sotto di essa facciano parte del ventre e viceversa quelle del dorso.

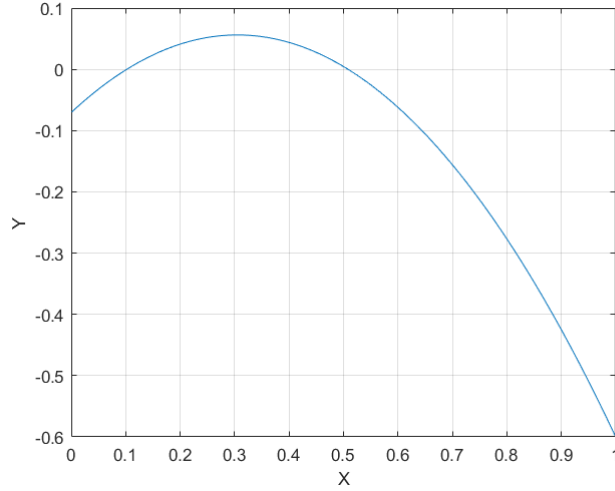


Figura 4.24: Linea media interpolata

A questo punto dato che i dati sperimentali sono forniti in termini di Mach isentropico si procede a valutarlo ricordando che $P^\circ = 1$ e avendo le informazioni su P_w dalle simulazioni.

$$M_{is} = \sqrt{\frac{2}{\gamma - 1} \left[\left(\frac{P^\circ}{P_w} \right)^{\frac{\gamma-1}{\gamma}} - 1 \right]}$$

A questo punto per completare l'analisi è necessario applicare una rotazione al sistema di riferimento utilizzato nelle simulazioni in modo da realizzare un confronto coerente tra i dati:

$$\beta = 33.3^\circ$$

$$x' = x \cdot \cos \beta - y \cdot \sin \beta$$

$$c = 1 / \cos \beta$$

In figura 4.25 si possono osservare i risultati ottenuti dal confronto dei dati ottenuti attraverso l'analisi con la griglia più raffinata e con il metodo di Roe che ha dimostrato essere quello più adatto. I risultati della simulazione si avvicinano molto a quelli sperimentali fatta eccezione per la parte dorsale verso il bordo di fuga, la causa di questa differenza può essere imputabile alla presenza di urti che potrebbero non essere catturati accuratamente dalla mesh impiegata. Per migliorare i risultati si potrebbe pensare di modificare la griglia applicando delle tecniche più sofisticate e catturando meglio il comportamento. In figura 4.26 si riporta il campo di pressione riferito al caso in questione.

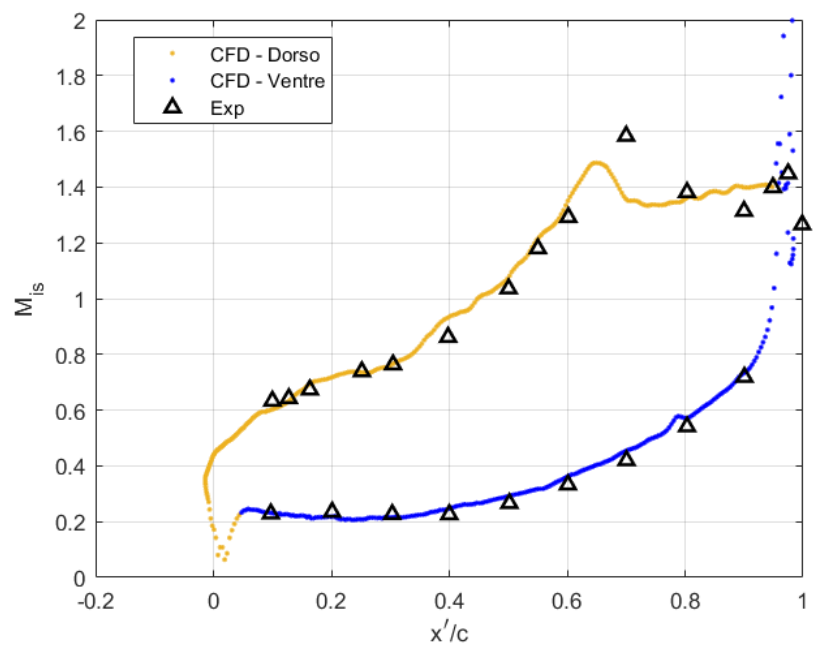


Figura 4.25: Confronto tra CFD e dati sperimentali LS59

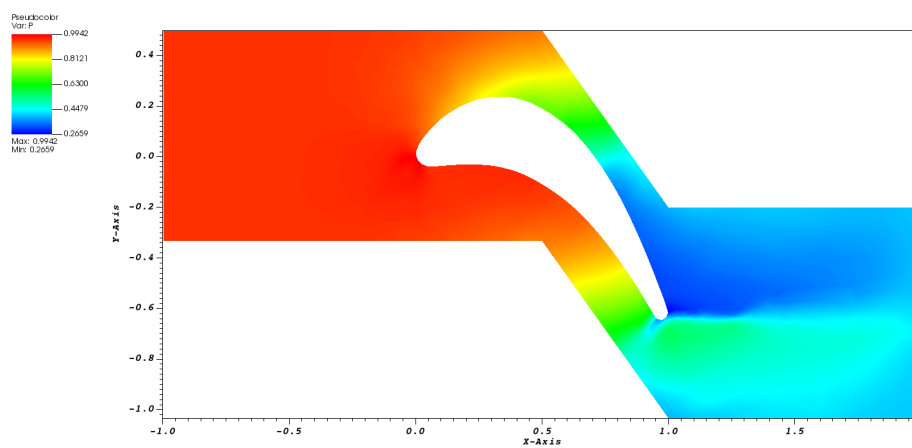


Figura 4.26: Campo di Pressione $lc = 0.005$

4.4 Analisi con ANSYS Fluent

Si propone adesso di ripetere le analisi svolte precedentemente attraverso il software commerciale ANSYS Fluent, si ripeteranno infatti gli stessi passaggi compiuti per lo studio con il codice CFD sviluppato nell'ambito di questo corso ma facendo uso dei software integrati in ANSYS.

4.4.1 Realizzazione geometria

Per la realizzazione della geometria si utilizza il programma CAD *DesignModeler*. Il software in questione consente di realizzare all'interno dello stesso le geometrie dei corpi che dovranno essere simulati oppure, come è più usuale nell'ambito industriale, permette di importare delle geometrie da file esterni. Nell'ambito industriale infatti, solitamente la geometria viene fornita a chi compie le simulazioni attraverso dei modelli 3D o dei file già predisposti. Nel caso in analisi è possibile ottenere un modello 3D a partire da *gmsk*, infatti aggiungendo al file *.geo* il comando:

$$\text{SetFactory}(\text{"OpenCASCADE"})$$

Il software permetterà di salvare la geometria in un file CAD, in particolare si salverà il file con estensione standard *.step*. Una volta importato sarà necessario scalare le dimensioni del corpo in quanto *gmsk* riduce la scala di un fattore 1000.

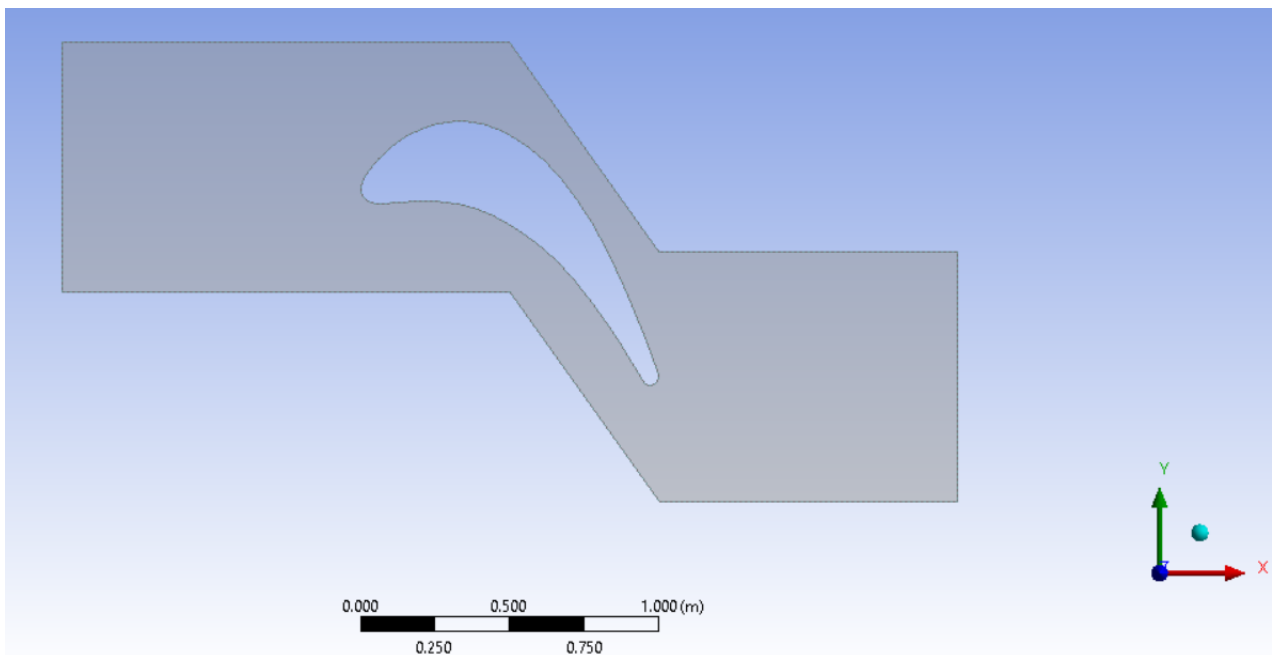


Figura 4.27: Geometria con DesignModeler

4.4.2 Realizzazione mesh

Il passo successivo è la realizzazione di una mesh sulla geometria del corpo, in questo caso si utilizza il software *Meshing* che permette a partire di realizzare una griglia a partire dal corpo realizzato nell'ambiente precedente. Per la generazione di questa mesh si è deciso di utilizzarne una non strutturata composta da elementi quadrangolari quando possibile, inoltre si è scelto di includere un raffinamento sulla parete della paletta per cogliere lo strato limite, il quale servirà per lo studio attraverso un modello RANS. La mesh realizzata ha le seguenti proprietà:

- Element size 0.01
- Inflation layers max 30
- Inflation Growth rate 1.2
- Transition ratio 0.8

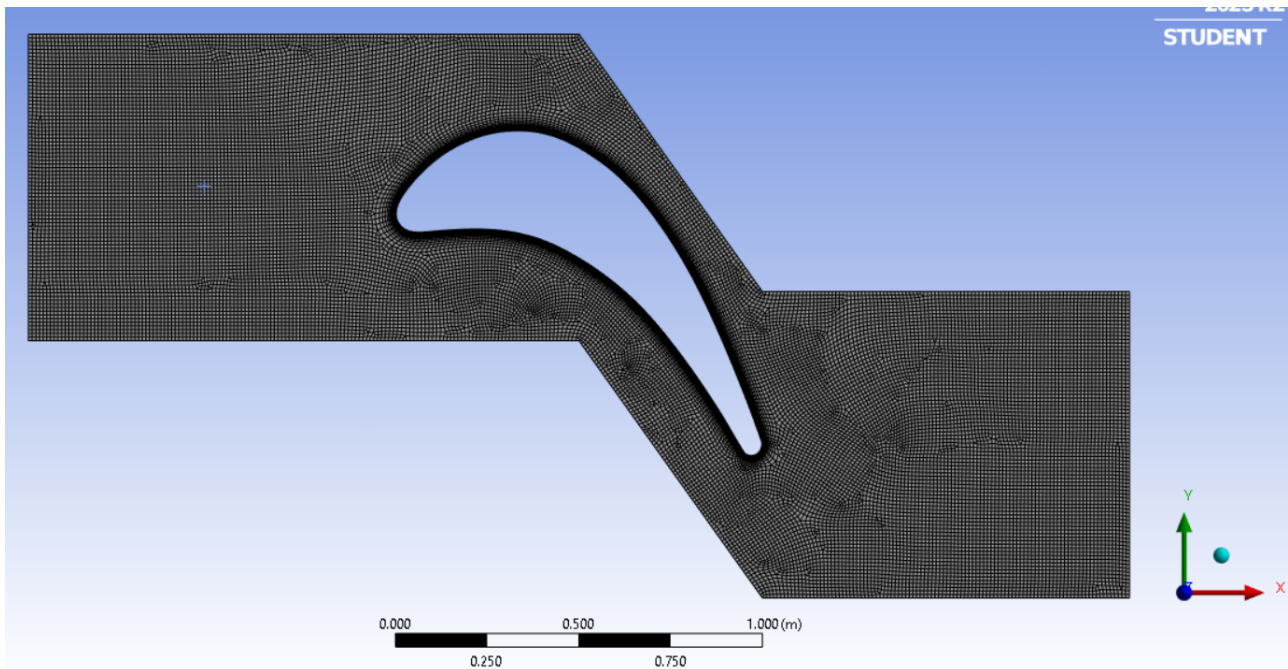


Figura 4.28: Mesh attraverso il software Meshing

Nell'ambiente *Meshing* è anche possibile associare alle varie superfici diversi label, i quali verranno riconosciuti dal software *Fluent* e nel momento in cui si definiranno le condizioni al contorno l'assegnazione sarà semplificata. In questo caso l'inlet è definito dal bordo verticale di sinistra, l'outlet dal bordo verticale di destra, la parete solida è la geometria del profilo e infine si definiscono come bordi periodici le pareti superiori e inferiori del dominio sulle quali verrà imposta successivamente una condizione di periodicità.

4.4.3 Analisi Fluent

A questo punto si hanno tutti gli strumenti per avviare la simulazione con *Fluent*, si apre quindi il software e si definiscono alcuni parametri per avviare la simulazione. Nel caso in analisi si svolgerà una simulazione inviscida e una facendo uso delle RANS.

Il primo passo una volta avviato il software è quello di scegliere che tipo di solutore si vuole utilizzare, in questo caso si usa un solutore *density based* il quale richiede la definizione di una pressione di riferimento. Questa pressione può essere scelta in diversi modi, in questo caso la si pone nulla al fine di ottenere dei valori di pressione assoluti e non relativi. Una volta definito il tipo di solutore si passa alla scelta del modello:

- Se si analizza un caso non viscoso si sceglierà la voce *inviscid*.
- Se si analizza un caso viscoso si hanno a disposizione diversi modelli, nell'ambito di questa analisi si sceglie *Spalart - Allmaras*.

Sempre nell'ambiente modello è bene verificare che l'equazione dell'energia venga risolta e quindi se non è on bisogna renderla tale.

Una volta definito il modello si passa all'assegnazione delle proprietà del materiale, in particolari quelle del fluido aria. Nel caso dell'analisi inviscida è sufficiente dire che la densità non è costante ma segue la legge dei gas perfetti, infatti in un caso inviscido imporre M equivale a imporre ρ e le proprietà dipendono solo da M . Quando si analizza invece un caso viscoso si ha che le proprietà dipendono non solo dal Mach ma anche da Re e di conseguenza bisogna agire su altre grandezze:

$$Re = \frac{\rho u c_x / \cos \beta}{\mu}$$

Se infatti si ha un numero di Re fissato si può agire o sulla lunghezza caratteristica oppure su μ dato che la densità e la velocità sono funzioni del Mach. Agire sulla lunghezza caratteristica una volta definita la geometria non è fattibile, di conseguenza non rimane che imporre il valore della viscosità.

$$Re = 554140 \implies \mu = 4.0819 \cdot 10^{-4} [Ns/m^2]$$

Si passa ora alla definizione delle condizioni al contorno, per farlo si distinguono le varie superfici facendo uso dei label assegnati nella fase di meshing e si impongono due condizioni all'inlet, essendo che il campo è subsonico, e una condizione all'outlet. Le condizioni all'inlet che si impongono sono:

$$P^\circ = 100000 [Pa] \quad T^\circ = 300 [K]$$

Per l'outlet invece si impone la pressione statica associata ad un Mach isentropico di 1.2:

$$P = 41237.7 [Pa]$$

Bisogna inoltre assegnare la direzione del flusso in ingresso che in questo caso risulta inclinato di $\alpha = 30^\circ$.

Bisogna scegliere il *solution methods*, in questo caso essendo già implementato si opta per un metodo implicito e si fa uso di Roe utilizzando una discretizzazione spaziale del secondo ordine upwind. Si sceglie inoltre un numero di Courant pari a 5, nonché un numero di iterazioni pari a 10000 al fine di raggiungere la convergenza.

A questo punto è possibile avviare la simulazione e ottenere i risultati per i due casi. Oltre ad una rappresentazione del campo di moto in termini di Mach è possibile ottenere l'andamento della pressione a parete e confrontarla con i dati delle simulazioni precedenti svolte con il codice CFD, l'unico accorgimento da fare è che i valori di pressione ottenuti devono essere rapportati alla pressione totale in ingresso per ottenere dei dati coerenti con quelli precedenti.

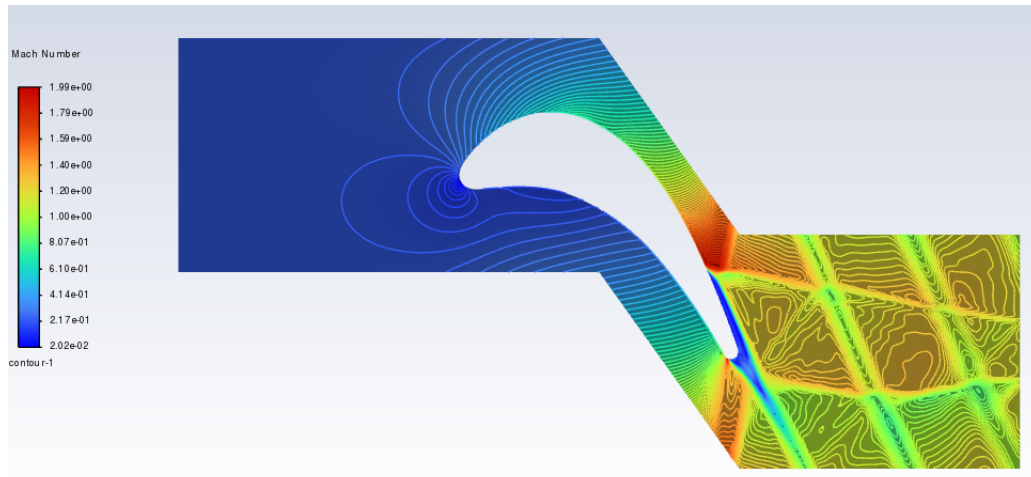


Figura 4.29: Isolivello Mach caso inviscido

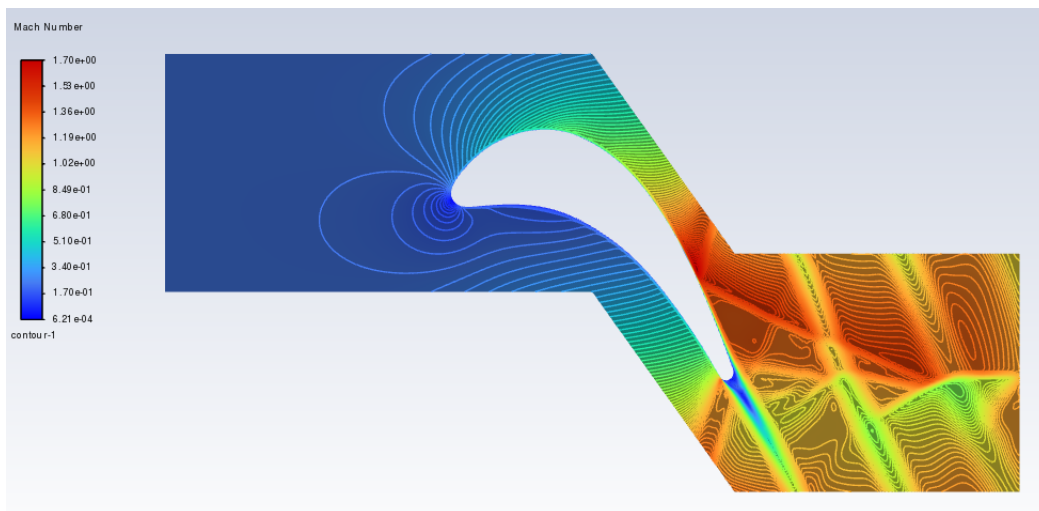


Figura 4.30: Isolivello Mach caso viscoso

Se si mettono a confronto i risultati delle due simulazioni si osserva che gli andamenti delle soluzioni sono pressoché gli stessi. Questo è dovuto al fatto che in assenza separazioni le

equazioni di Eulero colgono bene le distribuzioni di pressione a parete, che nel caso viscoso differiscono solo per la presenza dello strato limite, che per alti numeri di Reynolds risulta piccolo rispetto alle dimensioni del corpo. Se nel problema fossero state presenti delle separazioni allora Eulero non sarebbe stato in grado di catturarle. Nella figura 4.30 si può apprezzare la presenza di strato limite che porta il Mach ad essere nullo a parete, avendo una griglia tale da avere il baricentro di una cella di calcolo all'interno del sottostrato viscoso. Per risolvere accuratamente lo strato limite è necessario che la mesh sia tale da avere $y^+ \leq 5$, si riporta in figura 4.31 l'andamento lungo la parete della pala.

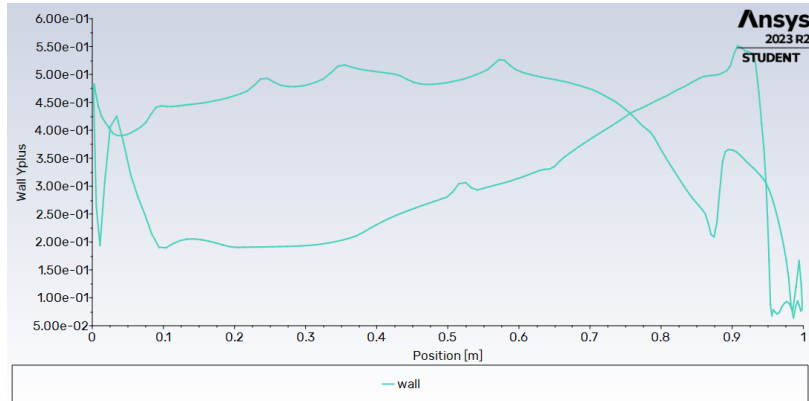


Figura 4.31: y^+ a parete

Dal grafico si osserva che in tutte le coordinate si è al di sotto della soglia e che quindi lo strato limite risulta accuratamente risolto.

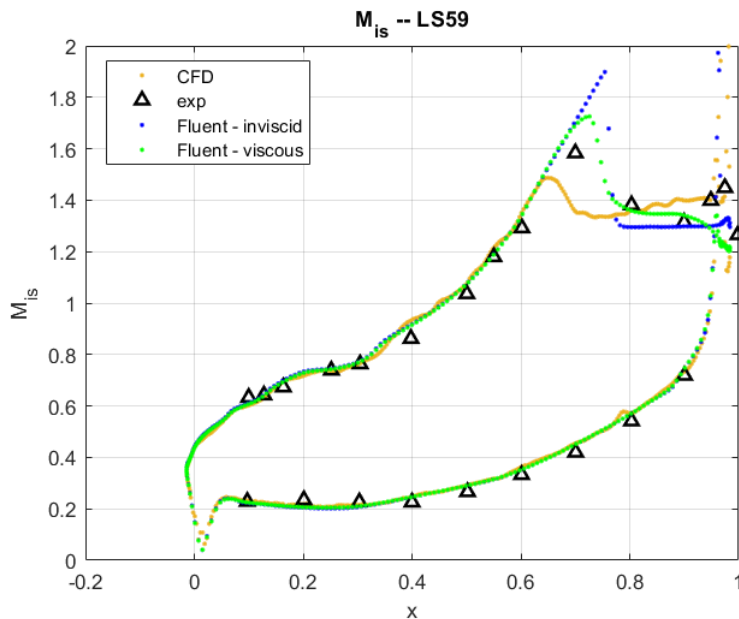


Figura 4.32: Confronto tra dati sperimentali CFD e Fluent

Come si osserva nella figura 4.32 tutti i casi si comportano all'incirca allo stesso modo nel cogliere i dati sperimentali, le uniche differenze tra tutti i modelli utilizzati le differenze si

localizzano nel tratto finale del dominio, infatti la presenza di bordi di fuga e spigoli porta ad una generazione di entropia numerica e si ha la presenza di urti. Nelle due analisi, inoltre, sono stati impiegati due solutori con ordine differente e due griglie differenti, è quindi normale che gli andamenti presentino delle differenze, quello più accurato coglie meglio gli urti. L'utilizzo di un modello RANS consente di ottenere dei risultati più accurati avendo rilassato alcune ipotesi e avendo introdotto la viscosità. Si ricorda, infatti, che i calcoli CFD svolti sono affetti da due tipologie di errori, uno associato alla discretizzazione che si può controllare attraverso una griglia più fitta e pagando in termini di potenza di calcolo, l'altro errore è quello di modello che non può essere controllato poiché ogni modello è intrinsecamente sbagliato e di conseguenza se si vuole agire su di esso si deve cambiare modello utilizzandone uno più raffinato.

Preso doppia rampa

Il successivo caso di interesse è quello di una presa a doppia rampa soggetta ad un flusso supersonico, questo caso è interessante a causa della presenza di urti dovuti alla corrente incidente su una geometria di questo tipo. Come per i casi precedenti si svolgeranno delle analisi di convergenza della griglia per Roe e Lax-Friedrichs e si presenterà un confronto tra i dati sperimentali e quelli ottenuti dalle simulazioni per verificare la bontà dei conti.

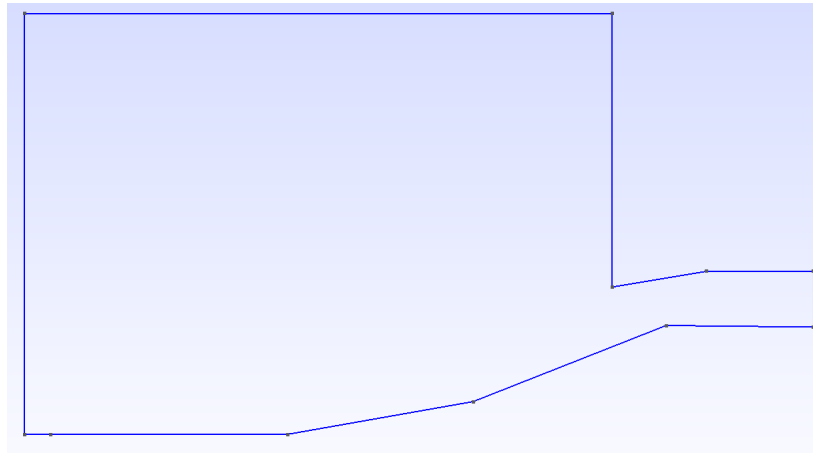


Figura 5.1: Dominio computazionale Doppia rampa

5.1 Inizializzazione

I parametri di ingresso sono:

- input.txt:
 - *presa_doppia_rampa.msh*, dato che si è interessati a compiere un'analisi di convergenza questo file verrà richiamato più volte, ma ad ogni simulazione si avrà un numero crescente di nodi costituenti la griglia.
 - KFINAL: 300000

- KINF: 100
- KOUT: 100
- CFL: 0.3
- inlet.txt:
 - $T_{IN}^{\circ} = 1$
 - $P_{IN}^{\circ} = 1$
 - $\alpha = 0^{\circ}$
 - $M_{IN} = 3$
- outlet.txt:
 - $P_{exit} = 0.001$, il valore non verrà considerato dato che si avrà un flusso supersonico al bordo di uscita del dominio. In questo caso è buona norma mettere un valore basso per l'inizializzazione.

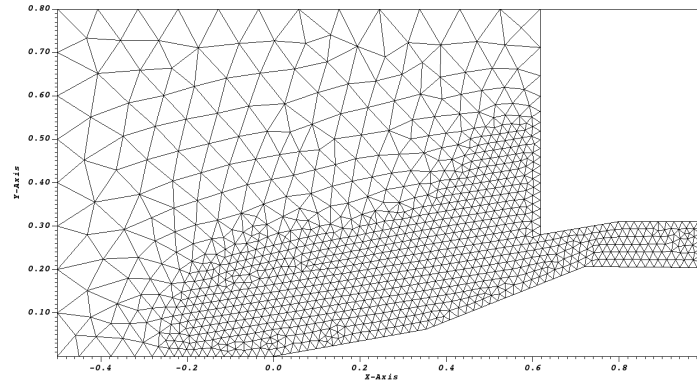


Figura 5.2: Mesh $l_c = 0.02$

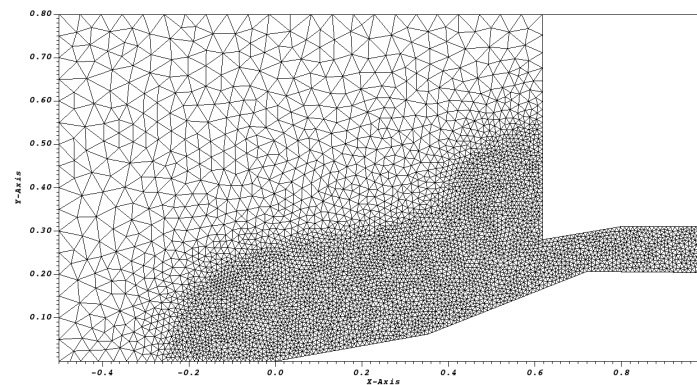


Figura 5.3: Mesh $l_c = 0.01$

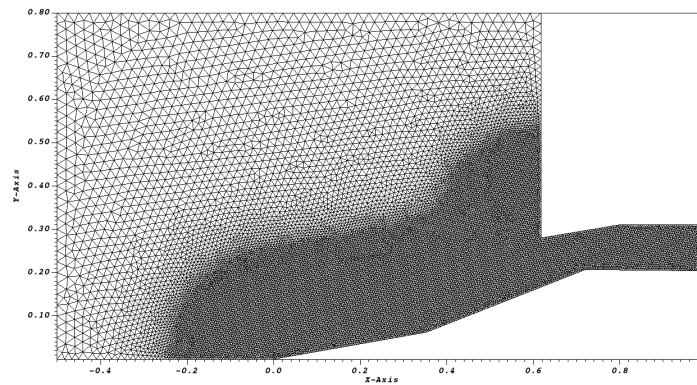


Figura 5.4: Mesh $l_c = 0.005$

5.1.1 Campo di Mach

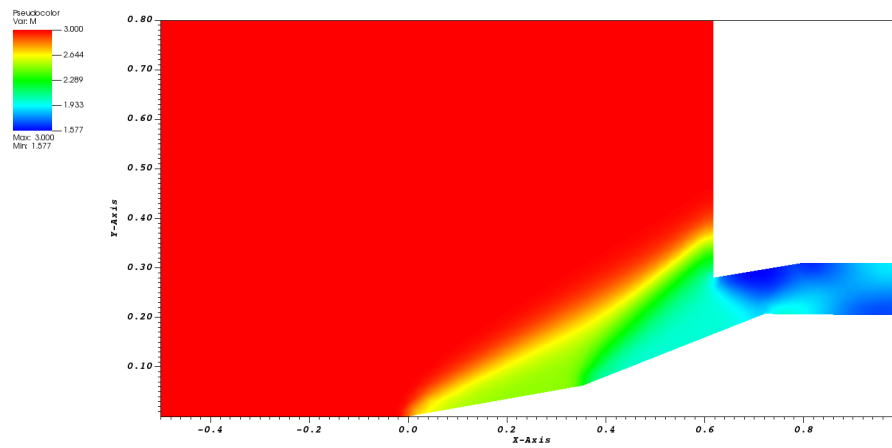


Figura 5.5: Campo di Mach $l_c = 0.02$ -Lax Friedrichs

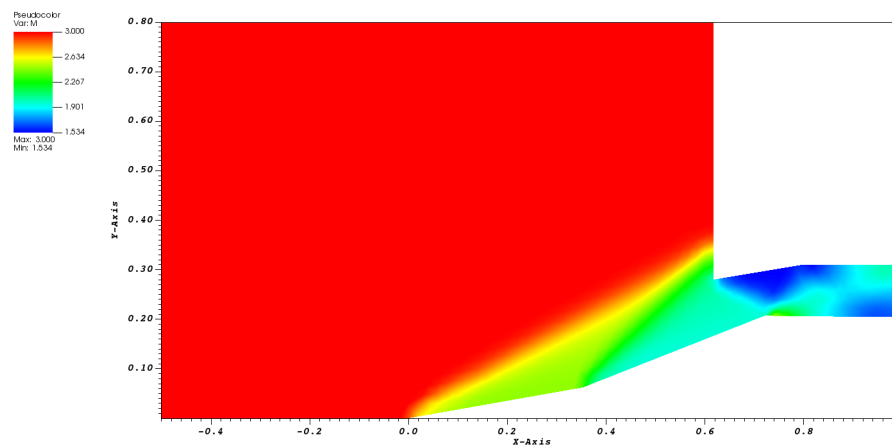


Figura 5.6: Campo di Mach $l_c = 0.02$ - Roe

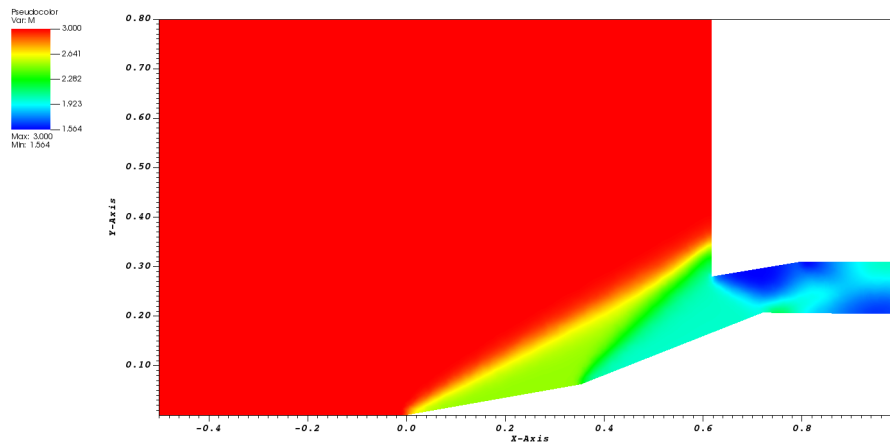


Figura 5.7: Campo di Mach $l_c = 0.01$ -Lax Friedrichs

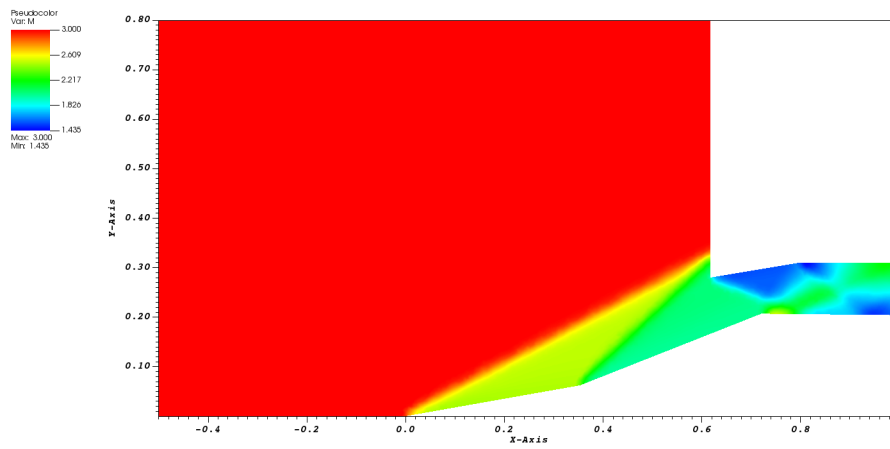


Figura 5.8: Campo di Mach $l_c = 0.01$ - Roe

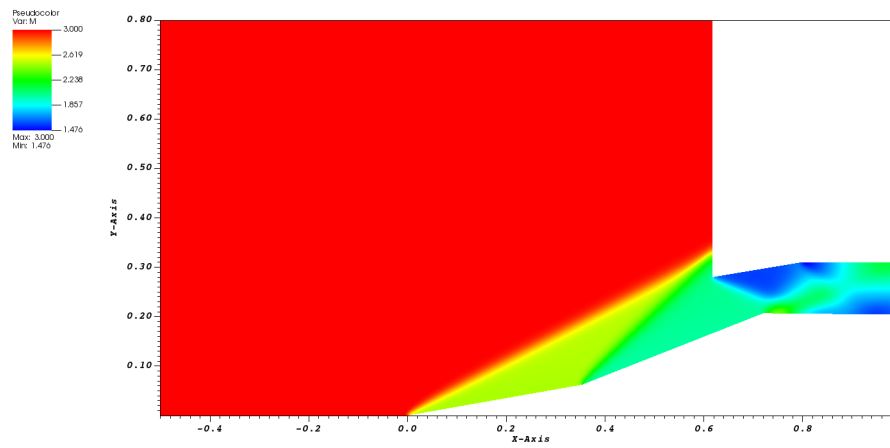


Figura 5.9: Campo di Mach $l_c = 0.005$ -Lax Friedrichs

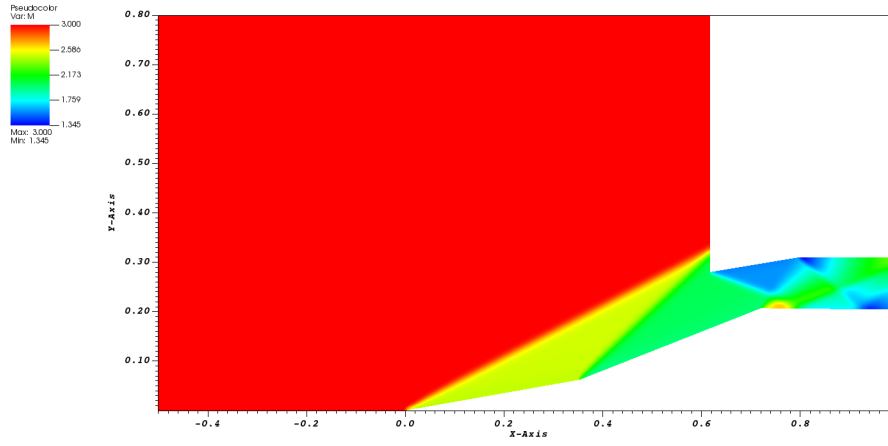


Figura 5.10: Campo di Mach $l_c = 0.005$ - Roe

Dalle diverse rappresentazioni è possibile apprezzare la formazione dei due urti sulle due rampe, nonché quello presente sull'estremità superiore sul bordo. Un particolare che si osserva realizzando le simulazioni, è che tutte le griglie raggiungono la convergenza molto velocemente grazie al fatto che la maggior parte del dominio è in regime supersonico e quindi le evoluzioni del transitorio riguardano unicamente una zona limitata del dominio. Si riportano in seguito i campi di velocità rappresentati per il caso di griglia più raffinata.

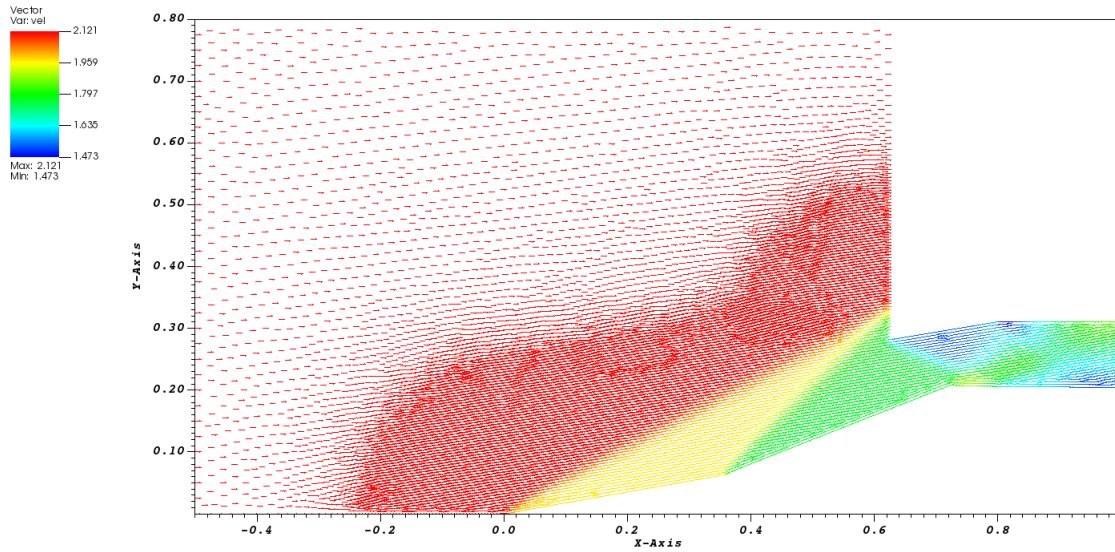


Figura 5.11: Campo di Velocità $l_c = 0.005$ -Lax Friedrichs

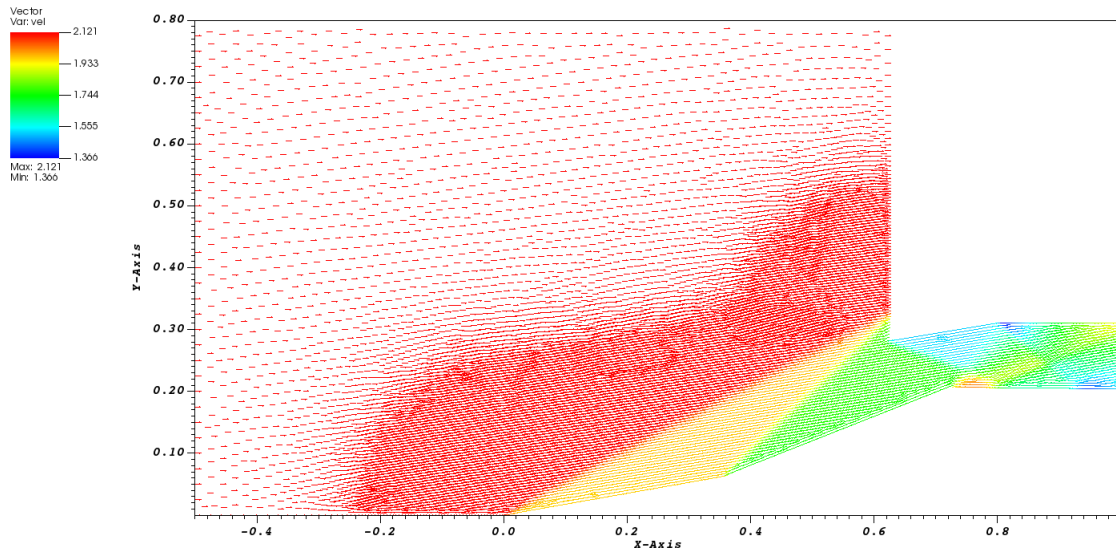


Figura 5.12: Campo di Velocità $l_c = 0.005$ - Roe

5.1.2 Campo di Entropia

Nel caso in analisi come si è potuto osservare dal campo di moto vi sono degli urti e quindi l'entropia non sarà nulla su tutto il dominio come avveniva invece per il caso del bump. Il fatto che l'entropia non sia più nulla non consente più di valutare l'ordine di convergenza tramite la conoscenza della soluzione esatta, in quanto questa non è più nota.

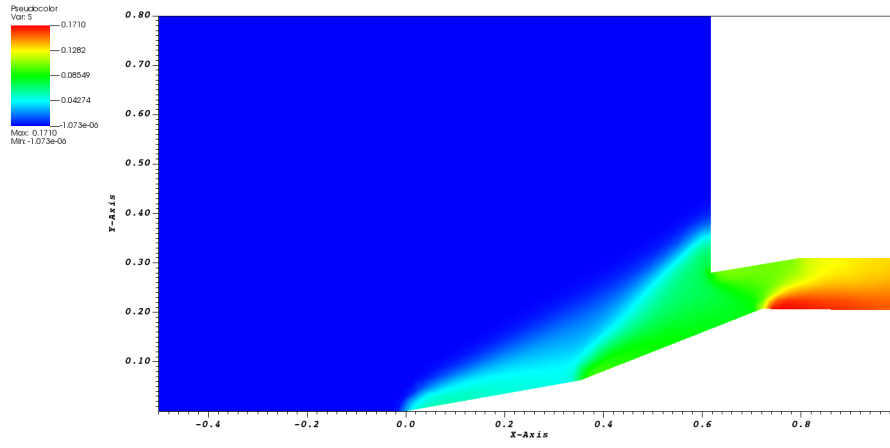


Figura 5.13: Campo di Entropia $l_c = 0.02$ -Lax Friedrichs

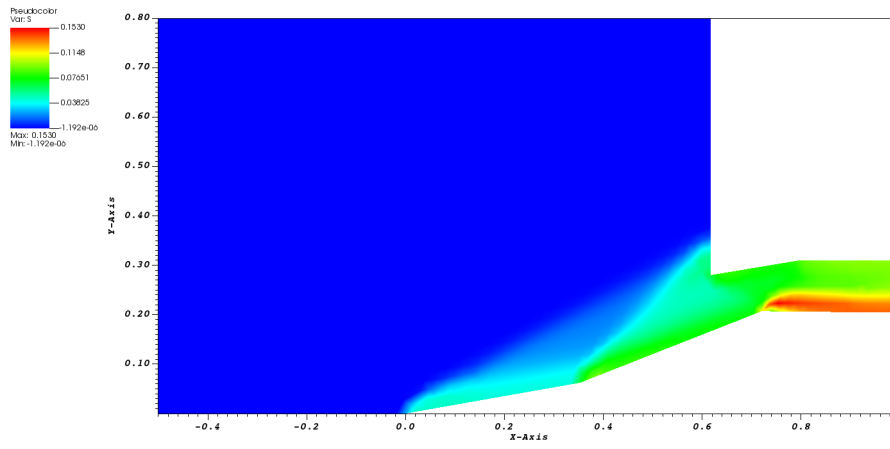


Figura 5.14: Campo di Entropia $l_c = 0.02$ - Roe

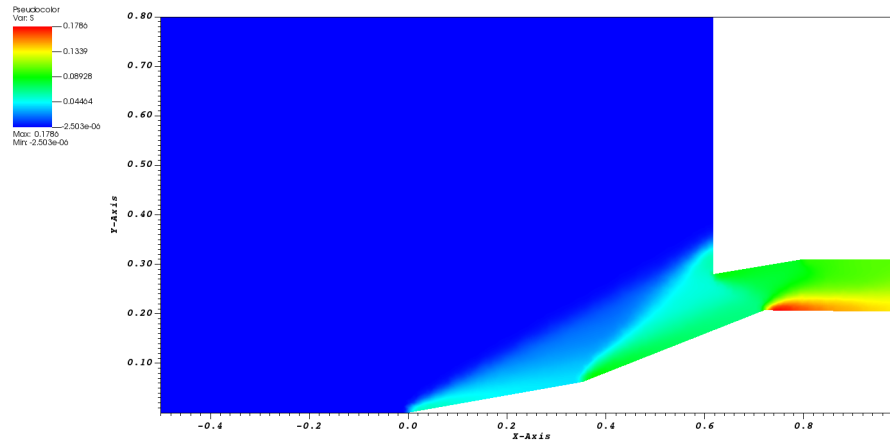


Figura 5.15: Campo di Entropia $l_c = 0.01$ -Lax Friedrichs

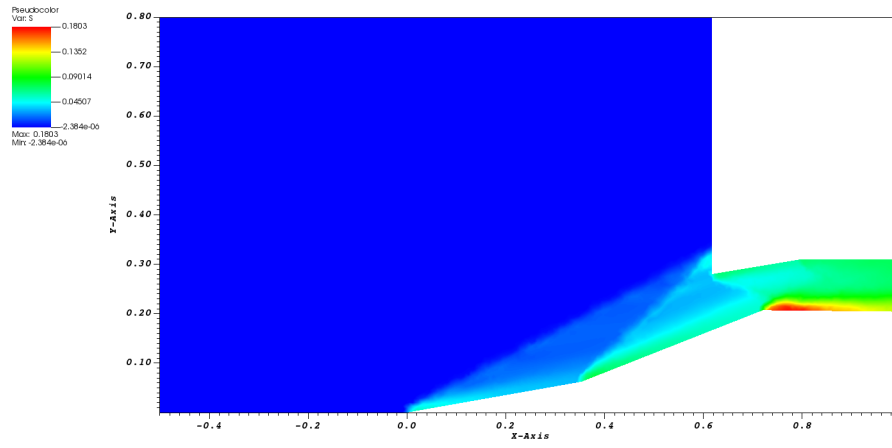


Figura 5.16: Campo di Entropia $l_c = 0.01$ - Roe

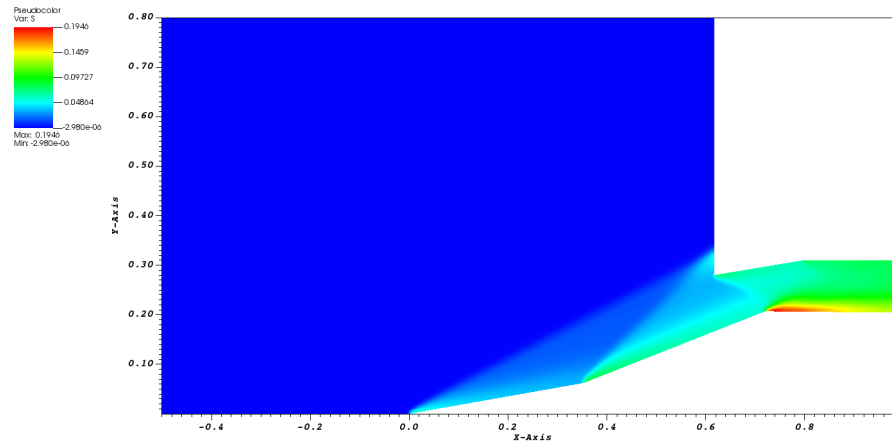


Figura 5.17: Campo di Entropia $l_c = 0.005$ -Lax Friedrichs

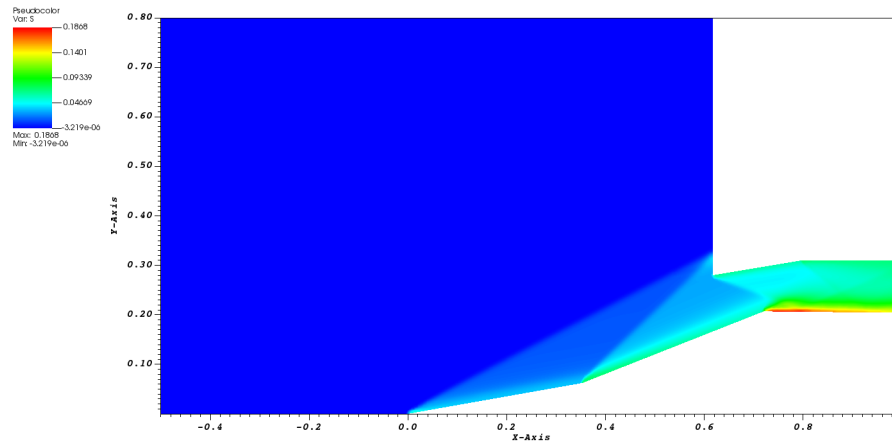


Figura 5.18: Campo di Entropia $l_c = 0.005$ - Roe

5.2 Analisi di convergenza

Per l'analisi di convergenza in questo studycase non si può più sfruttare l'informazione sull'entropia per una valutazione diretta dell'ordine di convergenza, per quello infatti attraverso l'estrapolazione di Richardson verranno valutati gli errori solo per ordine teorico ed effettivo. Comunque come evidente dalle visualizzazioni dell'entropia proposte precedentemente è osservabile come al ridursi della lunghezza caratteristica la soluzione migliori. Si riportano quindi gli andamenti della norma 2 dell'entropia al variare della lunghezza caratteristica confrontando i due metodi. La norma è così definita:

$$\|S\|_2 = \sqrt{\frac{\int_{\Omega} (S)^2 d\Omega}{\int_{\Omega} d\Omega}}$$

Si discretizza questa espressione sostituendo gli integrali con delle sommatorie sul numero di elementi interni:

$$\|S\|_2 = \sqrt{\frac{\sum (S)_i^2 \cdot Area_i}{\sum Area_i}}$$

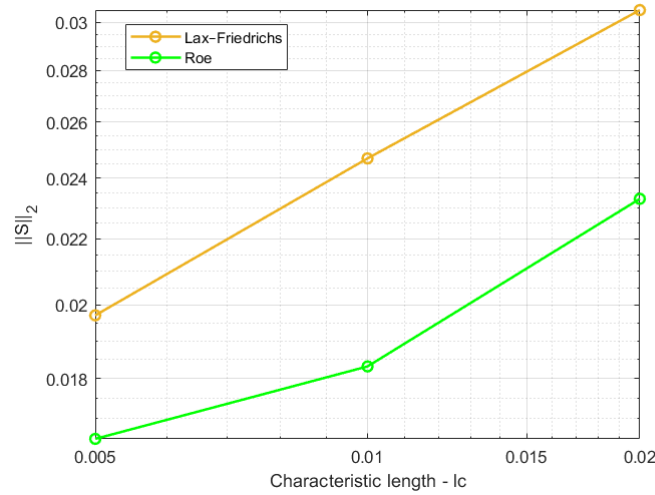


Figura 5.19: $\|S\|_2$ vs lc

Estrapolazione di Richardson con ordine teorico

Per prima cosa si ipotizza l'ordine effettivo sia pari a quello teorico e si immagina di svolgere due simulazioni aventi le seguenti lunghezze caratteristiche:

$$\begin{aligned} h_1 &= h \\ h_2 &= rh \quad \text{con } r > 1 \end{aligned} \implies \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{rh} - u_o = k \cdot r^p h^p \end{cases}$$

Si ha quindi un sistema 2x2 dove le incognite sono k e u_o . Manipolando le equazioni si ottiene:

$$u_o = \frac{r^p \cdot u_h - u_{rh}}{(r^p - 1)}$$

Da cui è possibile stimare l'errore di discretizzazione:

$$E_h = u_h - u_o$$

Si può anche valutare un indice che è bene associare alle proprie simulazioni che è il *GCI*, ovvero il grid convergence index così definito:

$$GCI = E_h \cdot F_s$$

Dove F_s è un fattore di sicurezza solitamente pari a 3.

Estrapolazione di Richardson con ordine effettivo

Se invece non si è confidenti che la propria griglia raggiunga il p teorico si può ripetere quanto fatto precedentemente aggiungendo una terza equazione in modo da introdurre p come incognita.

$$\begin{aligned} h_1 &= h \\ h_2 &= 2h \\ h_3 &= 4h \end{aligned} \implies \begin{cases} E_1 &= u_h - u_o = k \cdot h^p \\ E_2 &= u_{2h} - u_o = k \cdot 2^p h^p \\ E_3 &= u_{4h} - u_o = k \cdot 4^p h^p \end{cases}$$

Ottenendo:

$$\begin{aligned} p &= \frac{\ln \left(\frac{u_{4h} - u_{2h}}{u_{2h} - u_h} \right)}{\ln 2} \\ u_o &= u_h + \frac{u_{2h} - u_h}{2^p - 1} \\ E_h &= u_h - u_o \\ GCI &= E_h \cdot F_s \end{aligned}$$

In questo caso si è scelto di utilizzare la norma 2 dell'entropia come variabile u , ottenendo i risultati riportati in tabella 5.1.

		Grandezze			
		p	u_o	E	CGI
Ordine teorico	Lax-Friedrichs	1.00	1.61E-02	3.61E-03	1.08E-02
	Roe	1.00	1.42E-02	2.26E-03	6.79E-03
Ordine effettivo	Lax-Friedrichs	0.234	4.79E-02	2.82E-02	8.47E-02
	Roe	1.46	1.75E-02	1.03E-03	3.09E-03

Tabella 5.1: Risultati Analisi di convergenza

Il valore di p nel caso di Roe risulta superiore a quello teorico a causa del fatto che non si è raggiunta la regione asintotica. Per valutare correttamente l'ordine di convergenza della griglia

sarebbe necessario includere i termini di ordine superiore, dei quali però non si ha nessuna informazione. Per questa analisi sono stati usati i seguenti valori di u_h :

$$u_h = \begin{cases} 1.97E-02 & \text{Lax-Friedrichs} \\ 1.65E-02 & \text{Roe} \end{cases}$$

$$u_{2h} = \begin{cases} 2.47E-02 & \text{Lax-Friedrichs} \\ 1.83E-02 & \text{Roe} \end{cases}$$

$$u_{4h} = \begin{cases} 3.05E-02 & \text{Lax-Friedrichs} \\ 2.33E-02 & \text{Roe} \end{cases}$$

5.3 Confronto con dati sperimentali

Per verificare la bontà della simulazione è possibile confrontare le distribuzioni di pressione a parete con quelle ottenute sperimentalmente. Per poter confrontare i dati simulati con quelli sperimentali, si procede inserendo realizzando una nuova subroutine che salva all'interno di un file di testo i valori di pressione a parete nonché le coordinate dei punti associati.

```

Subroutine save_wall_data

  Use Variabili

5
  Implicit none
  integer :: i

10
  open (unit =1, file= 'wall_data.txt')

  do i =1, ninterf

15
    if (interf(i)%entity.eq.99) then
      write(1, *) interf(i)%x0, ele(interf(i)%e1)%P

    end if

20
  end do
  close(1)

end Subroutine

```

Dato che si sta usando una mesh non strutturata i dati salvati non saranno ordinati secondo delle coordinate x crescenti, per ordinarli è possibile aggiungere un algoritmo di ordinamento all'interno del codice stesso o implementare questa routine durante il post processing. Per valutare le pressioni a parete si devono filtrare i dati attraverso una soglia sulla coordinata y in modo da identificare solamente la parete fisica inferiore.

In figura 5.20 si possono osservare i risultati ottenuti dal confronto dei dati ottenuti attraverso l'analisi con la griglia più raffinata e con il metodo di Roe che ha dimostrato essere quello più adatto. I risultati colgono abbastanza bene il profilo sperimentale sebbene non si sovrappongano perfettamente con questi ultimi, le differenze però se si presta attenzione alla scala sono piuttosto piccole e quindi l'errore commesso è basso. Come per il caso della LS59 le discrepanze maggiori si manifestano alla coordinata x a valle di un urto. In figura 5.21 si riporta il campo di pressione riferito al caso in questione.

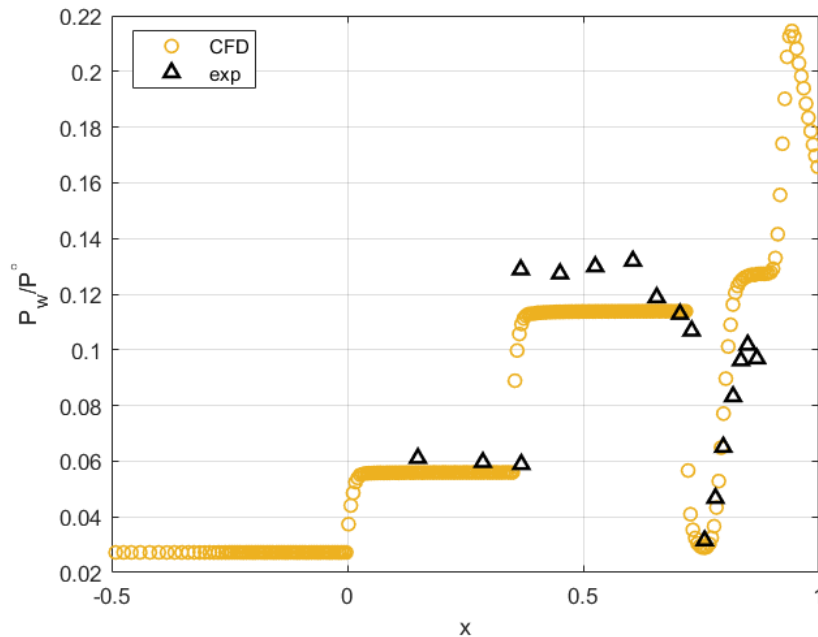


Figura 5.20: Confronto tra CFD e dati sperimentali presa doppia rampa

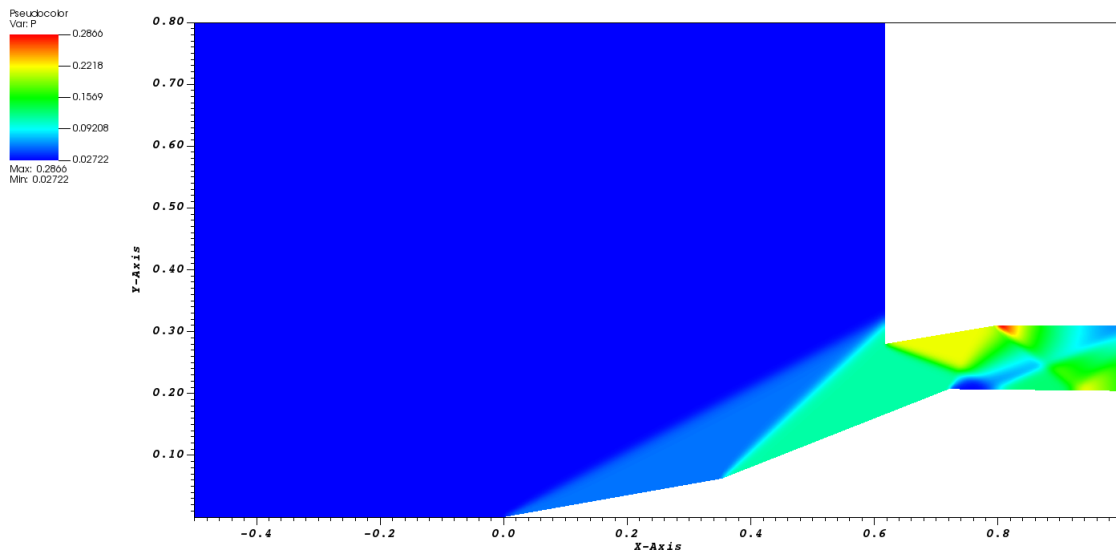


Figura 5.21: Campo di Pressione $lc = 0.005$

5.4 Analisi con ANSYS Fluent

Si propone adesso di ripetere le analisi svolte precedentemente attraverso il software commerciale ANSYS Fluent, si ripeteranno infatti gli stessi passaggi compiuti per lo studio con il codice CFD sviluppato nell'ambito di questo corso ma facendo uso dei software integrati in ANSYS.

5.4.1 Realizzazione geometria

Per la realizzazione della geometria si utilizza il programma CAD *DesignModeler*. Il software in questione consente di realizzare all'interno dello stesso le geometrie dei corpi che dovranno essere simulati oppure, come è più usuale nell'ambito industriale, permette di importare delle geometrie da file esterni. Nell'ambito industriale infatti, solitamente la geometria viene fornita a chi compie le simulazioni attraverso dei modelli 3D o dei file già predisposti. Nel caso in analisi è possibile ottenere un modello 3D a partire da *gmsh*, infatti aggiungendo al file *.geo* il comando:

$$\text{SetFactory}(\text{"OpenCASCADE"})$$

Il software permetterà di salvare la geometria in un file CAD, in particolare si salverà il file con estensione standard *.step*. Una volta importato sarà necessario scalare le dimensioni del corpo in quanto *gmsh* riduce la scala di un fattore 1000.

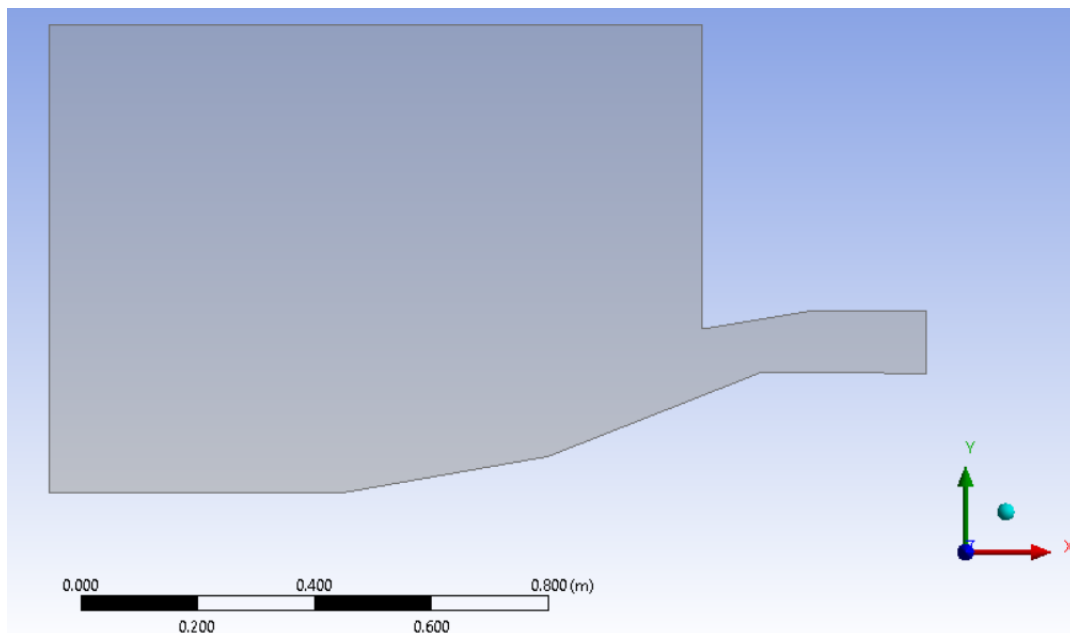


Figura 5.22: Geometria con DesignModeler

5.4.2 Realizzazione mesh

Il passo successivo è la realizzazione di una mesh sulla geometria del corpo, in questo caso si utilizza il software *Meshing* che permette a partire di realizzare una griglia a partire dal corpo realizzato nell'ambiente precedente. Per la generazione di questa mesh si è deciso di utilizzarne una non strutturata composta da elementi quadrangolari quando possibile, inoltre si è scelto di includere un raffinamento sullo strato inferiore per cogliere lo strato limite, il quale servirà per lo studio attraverso un modello RANS. Lo strato limite non si forma solo sulla parete inferiore ma anche in quella superiore all'interno del condotto della presa, tuttavia mettere un raffinamento in quella zona introdurrebbe delle complicazioni dato che il corpo presenta uno spigolo e raccordare la griglia in quel punto richiede delle accortezze che non sono oggetto del lavoro proposto. La mesh realizzata ha le seguenti proprietà:

- Element size 0.008
- Inflation layers max 15
- Inflation Growth rate 1.2

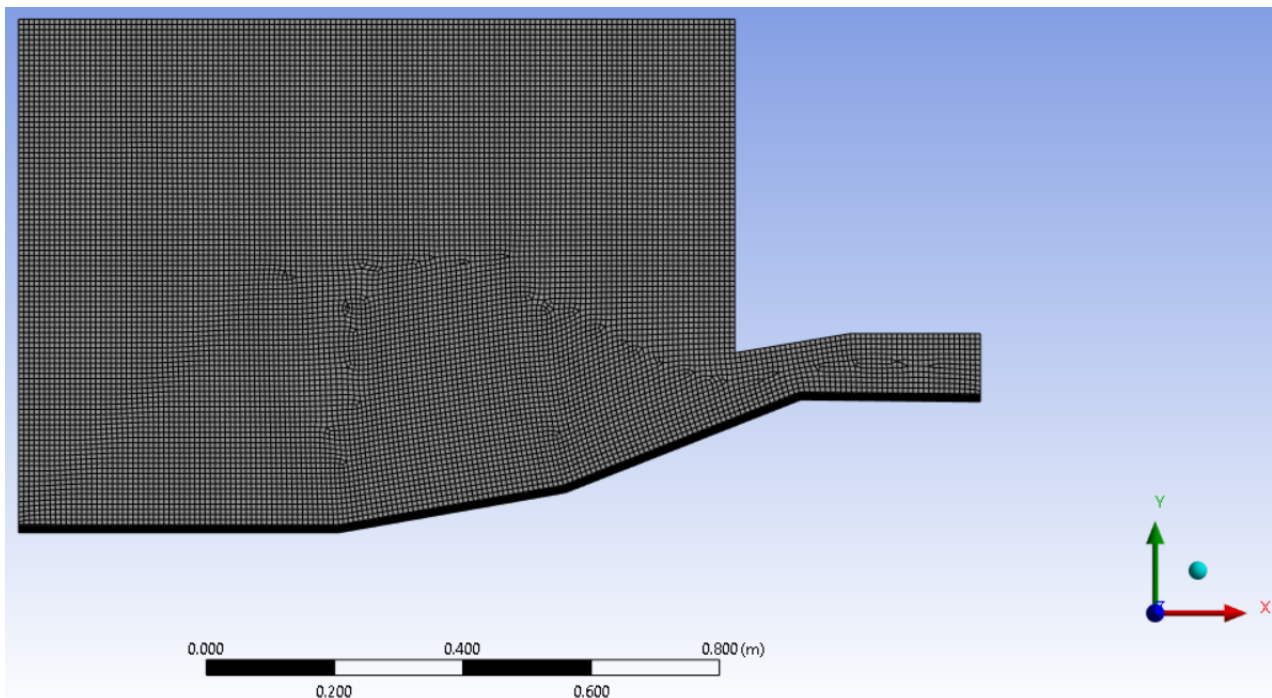


Figura 5.23: Mesh attraverso il software Meshing

Nell'ambiente *Meshing* è anche possibile associare alle varie superfici diversi label, i quali verranno riconosciuti dal software *Fluent* e nel momento in cui si definiranno le condizioni al contorno l'assegnazione sarà semplificata. In questo caso l'inlet è definito dal bordo verticale di sinistra, l'outlet dai bordi verticali di destra, le pareti solide sono le rampe e le pareti del condotto e infine alla linea superiore e la prima linea che si incontra inferiormente vengono trattate come linee di corrente.

5.4.3 Analisi Fluent

A questo punto si hanno tutti gli strumenti per avviare la simulazione con *Fluent*, si apre quindi il software e si definiscono alcuni parametri per avviare la simulazione. Nel caso in analisi si svolgerà una simulazione inviscida e una facendo uso delle RANS.

Il primo passo una volta avviato il software è quello di scegliere che tipo di solutore si vuole utilizzare, in questo caso si usa un solutore *density based* il quale richiede la definizione di una pressione di riferimento. Questa pressione può essere scelta in diversi modi, in questo caso la si pone nulla al fine di ottenere dei valori di pressione assoluti e non relativi. Una volta definito il tipo di solutore si passa alla scelta del modello:

- Se si analizza un caso non viscoso si sceglierà la voce *inviscid*.
- Se si analizza un caso viscoso si hanno a disposizione diversi modelli, nell'ambito di questa analisi si sceglie *Spalart - Allmaras*.

Sempre nell'ambiente modello è bene verificare che l'equazione dell'energia venga risolta e quindi se non è on bisogna renderla tale.

Una volta definito il modello si passa all'assegnazione delle proprietà del materiale, in particolari quelle del fluido aria. Nel caso dell'analisi inviscida è sufficiente dire che la densità non è costante ma segue la legge dei gas perfetti, infatti in un caso inviscido imporre M equivale a imporre ρ e le proprietà dipendono solo da M . Quando si analizza invece un caso viscoso si ha che le proprietà dipendono non solo dal Mach ma anche da Re e di conseguenza bisogna agire su altre grandezze:

$$Re = \frac{\rho u L}{\mu}$$

Se infatti si ha un numero di Re fissato si può agire o su L oppure su μ dato che la densità e la velocità sono funzioni del Mach. Agire sulla lunghezza caratteristica una volta definita la geometria non è fattibile, di conseguenza non rimane che imporre il valore della viscosità.

$$Re = 1.3 \cdot 10^6 \implies \mu = 1.4946 \cdot 10^{-5} [Ns/m^2]$$

Si passa ora alla definizione delle condizioni al contorno, per farlo si distinguono le varie superfici facendo uso dei label assegnati nella fase di meshing e si impongono unicamente le condizioni al contorno all'inlet essendo che il campo all'outlet è supersonico. La condizione all'inlet che si impone è di pressione e si definiscono la pressione totale e quella statica in ingresso:

$$P^\circ = 100000 [Pa] \quad P = 2722.37 [Pa] \quad T^\circ = 300 [K]$$

La pressione statica riportata è ottenuta dalla definizione di totale con $M = 3$. Bisogna inoltre assegnare la direzione del flusso in ingresso che in questo caso risulta normale al bordo di inlet.

Bisogna scegliere il *solution methods*, in questo caso essendo già implementato si opta per un metodo implicito e si fa uso di Roe, utilizzando una discretizzazione spaziale del secondo ordine upwind. Si sceglie inoltre un numero di Courant pari a 5, nonché un numero di iterazioni pari a 10000 al fine di raggiungere la convergenza.

A questo punto è possibile avviare la simulazione e ottenere i risultati per i due casi, oltre ad una rappresentazione del campo di moto in termini di Mach è possibile ottenere l'andamento della pressione a parete e confrontarla con i dati delle simulazioni precedenti svolte con il codice CFD, l'unico accorgimento da fare è che i valori di pressione ottenuti devono essere rapportati alla pressione totale in ingresso per ottenere dei dati coerenti con quelli precedenti.

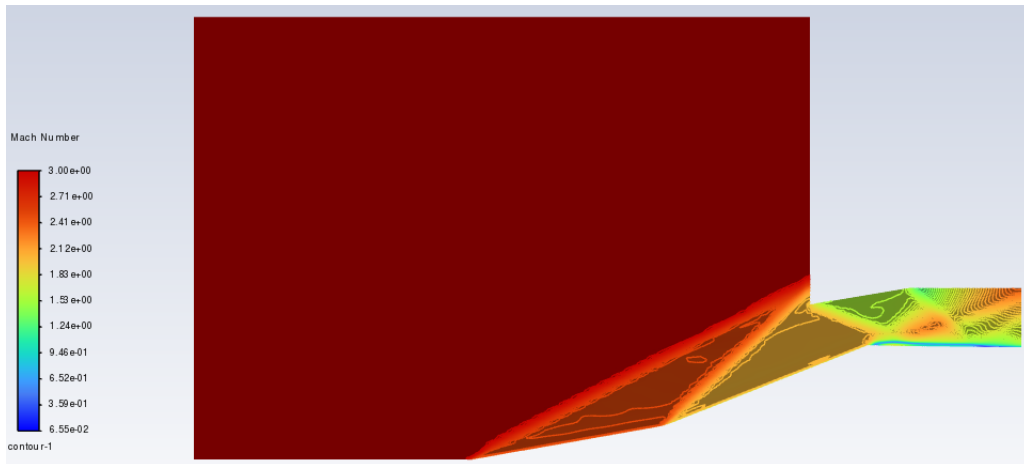


Figura 5.24: Isolivello Mach caso inviscido

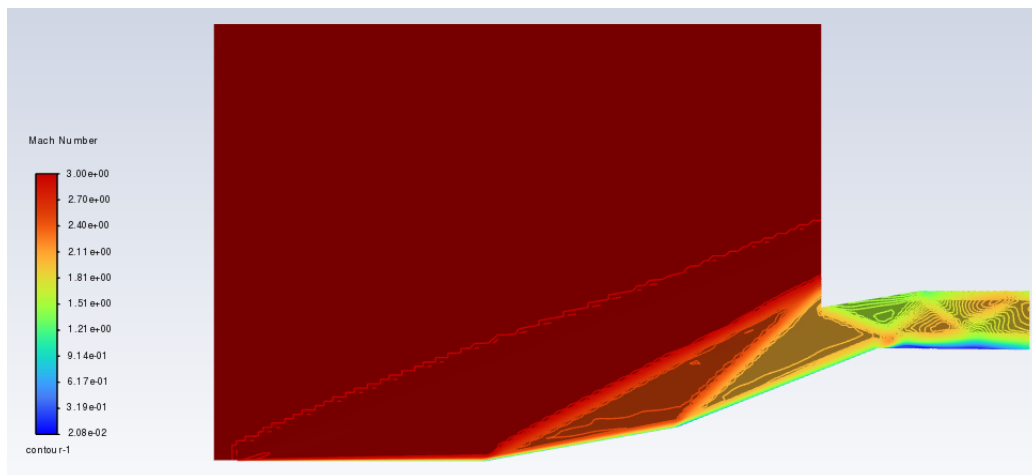


Figura 5.25: Isolivello Mach caso viscoso

Nella figura 5.25 si può apprezzare la presenza di strato limite che porta il Mach ad essere quasi nullo a parete, per averlo nullo del tutto è necessario che la griglia sia tale da avere il baricentro di una cella di calcolo all'interno del sottostrato viscoso. Per risolvere accuratamente

lo strato limite è necessario che la mesh sia tale da avere $y^+ \leq 5$, si riporta in figura 5.26 l'andamento lungo la parete inferiore.

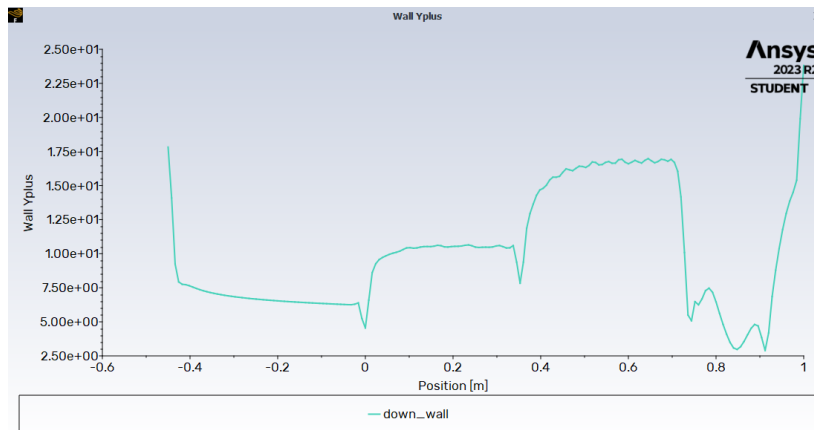


Figura 5.26: y^+ a parete

Dal grafico si osserva che nella maggior parte della parete si è al di sopra del valore di soglia tranne che per parte della porzione interna della rampa. Sarebbe stato necessario ripetere la simulazione aumentando i layers associati allo strato limite.

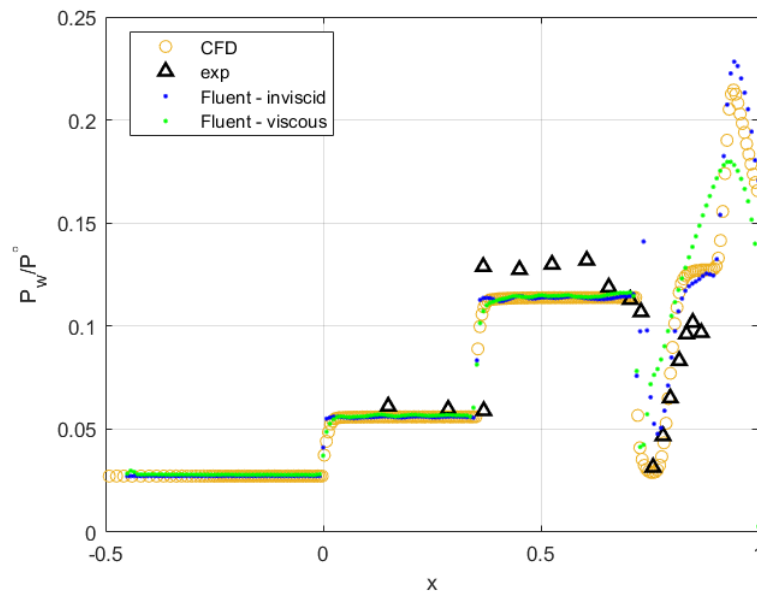


Figura 5.27: Confronto tra dati sperimentali CFD e Fluent

Come si osserva nella figura 5.27 tutti i casi si comportano all'incirca allo stesso modo nel cogliere i dati sperimentali, in particolare i casi inviscidi trovano riscontro diretto l'uno con l'altro, mentre quello viscoso si comporta diversamente nel tratto finale del dominio. Dato che però non sono presenti dei dati sperimentali nel tratto finale non è possibile comprendere chi si comporti più correttamente, tuttavia il metodo viscoso introduce delle equazioni in più e

rilassa alcune delle ipotesi e quindi ci si aspetta che sia più rappresentativo della realtà, infatti il modello viscoso è in grado di cogliere la separazione che si genera all'interno della presa. Nelle due analisi, inoltre, sono stati impiegati due solutori con ordine differente e due griglie differenti, è quindi normale che gli andamenti presentino delle differenze, lo scopo comunque utilizzando un modello RANS è quello di ottenere dei risultati più accurati. Si ricorda, infatti, che i calcoli CFD svolti sono affetti da due tipologie di errori, uno associato alla discretizzazione che si può controllare attraverso una griglia più fitta e pagando in termini di potenza di calcolo, l'altro errore è quello di modello che non può essere controllato poiché ogni modello è intrinsecamente sbagliato e di conseguenza se si vuole agire su di esso si deve cambiare modello utilizzandone uno più raffinato.

Ottimizzazione di Forma

In questa esercitazione si propone uno studio di ottimizzazione di forma multiobiettivo di un ugello convergente divergente attraverso il software *modeFRONTIER*. L'obiettivo di questo studio di ottimizzazione è quello di trovare la combinazione di parametri che massimizzi la spinta e al contempo renda minimo il peso.

6.1 Procedura

Il sistema che si analizzerà avrà come parametri liberi:

- La lunghezza del tratto divergente $10 \leq L \leq 17$.
- La pendenza della parabola nel punto iniziale B $20^\circ \leq \theta_1 \leq 40^\circ$.

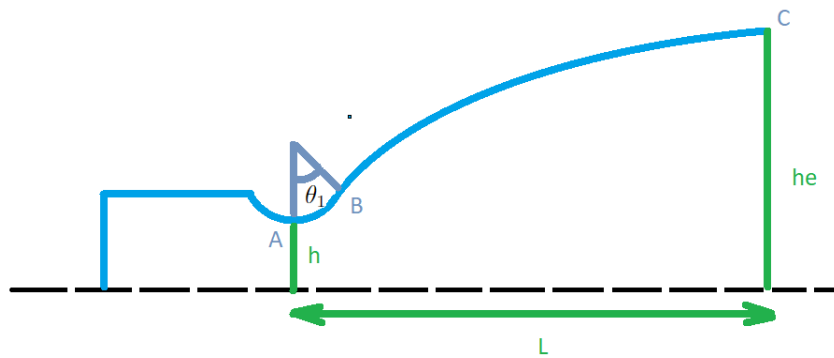


Figura 6.1: Ugello convergente-divergente

Di questo ugello viene presa un'altezza di gola h unitaria e si fissa l'altezza nel punto C, dato che si tratta di un problema 2D non assialsimmetrico parlare in termini di altezze equivale a parlare in termini di aree. Avere fissato i valori delle altezze implica aver definito il rapporto di aree:

$$\varepsilon = \frac{h_e}{h}$$

Aniché ragionare in termini di spinta si preferisce ragionare in termini di coefficiente di spinta nel vuoto definito come:

$$C_{F,vac} = \frac{F_{vac}}{P_c A_t} = \frac{F_{vac}/b}{P_c h} \quad \text{Dove} \quad F_{vac} = \int_e (\rho (\vec{u} \cdot \hat{n}) u_x + P n_x) dA$$

Della spinta si prende la proiezione lungo la direzione assiale perché è quella che effettivamente realizza la propulsione, essa rappresenta l'indice prestazionale che si vuole massimizzare. La valutazione del $C_{F,vac}$ e della superficie del tratto superiore della camera di spinta viene realizzata attraverso delle subroutine aggiunte nel codice CFD discusso nelle sezioni precedenti. La superficie del tratto superiore della camera di spinta può essere direttamente correlato alla massa dell'ugello stesso.

Gli obiettivi che si vogliono realizzare sono tra di loro in conflitto, dato che se si vuole rendere il peso minimo bisognerebbe andare verso una configurazione ad ugello conico, la quale però presenta delle perdite di divergenza notevoli rendendo $C_{F,vac}$ basso. Viceversa se si volesse un $C_{F,vac}$ molto alto bisognerebbe realizzare una configurazione avente un tratto di divergente più allungato che porterebbe il peso a salire, sarà quindi necessario trovare delle soluzioni di compromesso.

6.2 Software modeFRONTIER

Il software utilizzato in questa esercitazione automatizzerà il processo che si è già svolto nell'analisi dei casi studio precedenti seguendo i seguenti passi:

- Costruzione di una geometria.
- Generazione di una mesh.
- Simulazione CFD.

All'interno del software infatti attraverso uno schema a blocchi è possibile integrare tra loro diverse task e sfruttare i risultati prodotti ad un certo step del workflow come input per lo step successivo. Il workflow viene ripetuto per diverse geometrie facendo variare i parametri di ingresso attraverso una logica dettata dall'algoritmo di ottimizzazione, ottenendo infine nello spazio dei parametri una rappresentazione delle possibili soluzioni nel rispetto dei vincoli imposti. Quando si svolge un processo di ottimizzazione vi sono due principali strade:

- Design of Experiments (DOE): consiste nel raccogliere informazioni in tutto lo spazio delle soluzioni per realizzare un database ben strutturato per analisi statistica che è la base del design, comprendendo quali sono le variabili che hanno una maggiore influenza e soprattutto valutare se vi è una correlazione fisica tra le variabili, facilitando il design e orientandolo. In questo tipo di approccio non vi è una logica coinvolta, l'algoritmo genera una serie di design e attraverso le simulazioni si comprende quale sia il design di interesse, non si impone nessun obiettivo.
- Ottimizzazione: viene utilizzato un DOE iniziale che ha lo scopo di indirizzare la ricerca dei design di ottimo, ma la logica di definizione dei design successivi da sperimentare sono definiti attraverso una logica interna dell'algoritmo di ottimizzazione stesso.

Nel caso in esame l'algoritmo di risoluzione esplorerà lo spazio delle soluzioni a partire da un'analisi di un database prodotto da un DOE, da cui sarà possibile valutare le configurazioni di ottimo. In figura 6.2 viene riportato il workflow che il software segue, si procede a descriverne

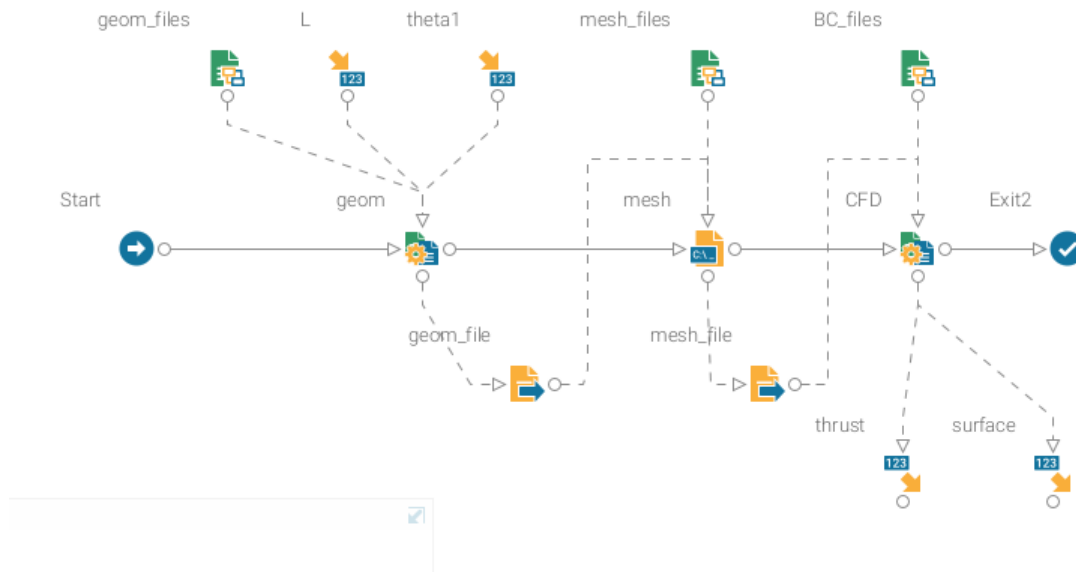


Figura 6.2: Workflow

tutti gli elementi:

- *geom* è un oggetto easydriver che consente di richiamare un eseguibile che utilizza come input e output dei file di testo. In questo caso l'eseguibile che viene richiamato è quello che si occupa di creare i punti della geometria dell'ugello. Questo oggetto è collegato ad altri tre in input e uno in output:
 - *geom_files* è un insieme di file di progetto che vengono richiamati dall'oggetto easy-driver *geom*, in questo caso l'unico file di supporto che viene fornito è l'eseguibile stesso.

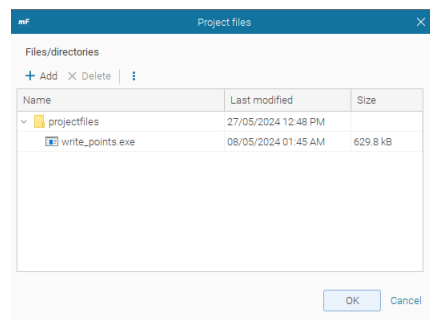
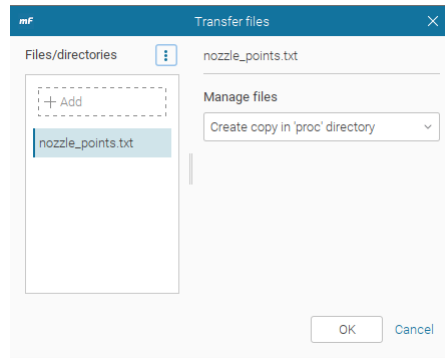


Figura 6.3: Project files *geom_files*

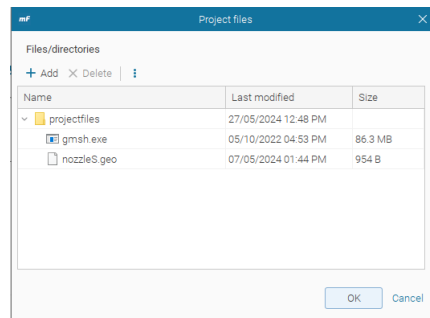
- *L* e *theta1* sono gli input parameter

Figura 6.4: Transferite files *geom_file*

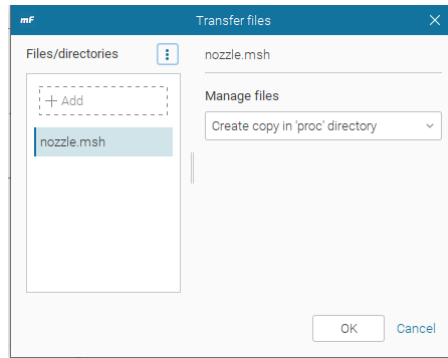
- *geom_file* è un blocco di tipo transferite file, ovvero esso trasporta il prodotto di un blocco ad un altro.

All'interno dell'ambiente easydriver sono presenti 3 ulteriori ambienti:

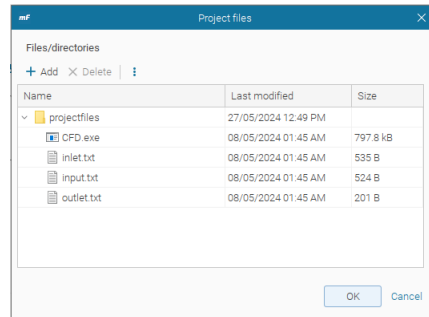
- input template: all'interno di questo ambiente vengono definite le regole di sostituzione degli input all'eseguibile.
 - Driver: contiene i comandi da eseguire, in questo caso vi è solamente "write_points.exe".
 - output: definisce le regole di mining.
- *mesh* è un elemento DOS batch che viene utilizzato per la generazione della mesh, per farlo si utilizza il software *gmsh*. Questo blocco è connesso al file proveniente dalla generazione della geometria e da un altro project file contenente l'eseguibile *gmsh* e il file *.geo*.

Figura 6.5: Project files *mesh_files*

Questo DOS batch una volta prodotta la mesh la trasferirà al blocco successivo per l'analisi CFD.

Figura 6.6: Transferite files *mesh_file*

- *CFD* questo blocco easydriver si occupa di realizzare la simulazione CFD attraverso il software sviluppato precedentemente e riceverà in ingresso la mesh in output al passo precedente nonché i project files e l'output prodotto saranno la spinta e la superficie.

Figura 6.7: Project files *BC_files*

6.3 Risultati

A seguito delle diverse simulazioni è possibile rappresentare una matrice di interazione tra i parametri del problema come risultato del DOE nonché i risultati ottenuti nello spazio delle soluzioni attraverso un bubble chart. Se si osserva la matrice prodotta come risultato si osserva che la dipendenza tra θ_1 e L non è esattamente zero e che i 30 design proposti forse non erano sufficienti, in ogni caso è possibile apprezzare le correlazioni fisiche che sussistono, come ci si aspetta, tra la superficie e la lunghezza del tratto divergente in quanto tra loro vi è una dipendenza indicata da un coefficiente prossimo a 1. Tra gli obiettivi invece si ha un coefficiente circa di 0.8, questo comporterà la presenza di un fronte di pareto; se le due funzioni obiettivo avessero avuto un coefficiente unitario l'analisi multiobiettivo non avrebbe avuto molto senso in quanto massimizzare o minimizzare uno dei due avrebbe portato lo stesso effetto sull'altro.



Figura 6.8: Scatter Matrix DOE

Focalizzandosi invece sulla figura 6.9 è possibile comprendere la strategia dell'algoritmo, infatti i punti blu che sono i primi ad essere analizzati sono molto esplorativi, mentre quelli rossi si posizionano sul fronte di pareto. Il fronte di Pareto si forma perché sono presenti due obiettivi tra loro contrastanti e questo fronte è definito come il luogo dei punti non dominati. Un punto si definisce non dominato se non esiste nessun altro punto che presenti valori migliori per tutte le funzioni obiettivo. Questo è l'insieme quindi delle soluzioni di compromesso lungo il quale bisogna indirizzare la soluzione.

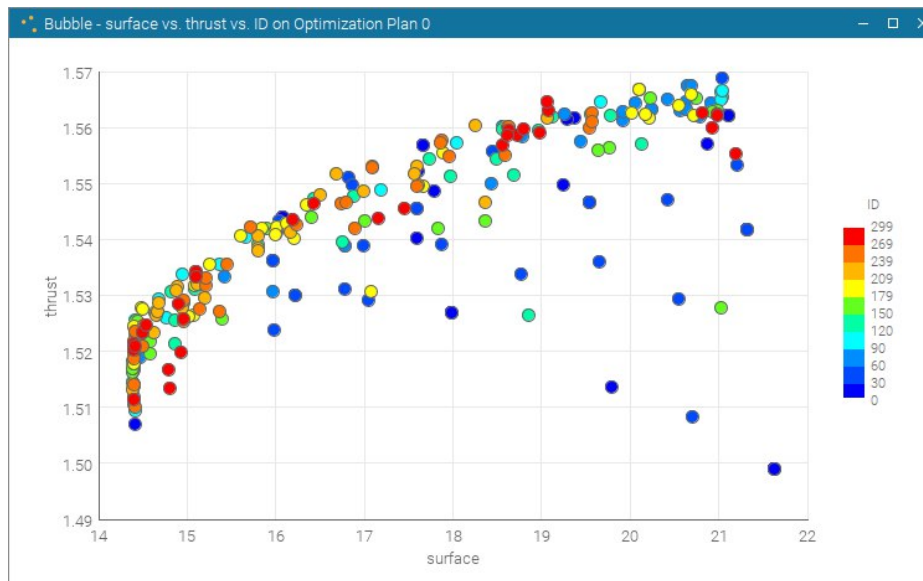


Figura 6.9: Spazio delle soluzioni

Se si è interessati alle soluzioni migliori e quindi ad isolare il fronte è possibile applicare una funzione di filtro e rappresentare solamente il fronte di Pareto.

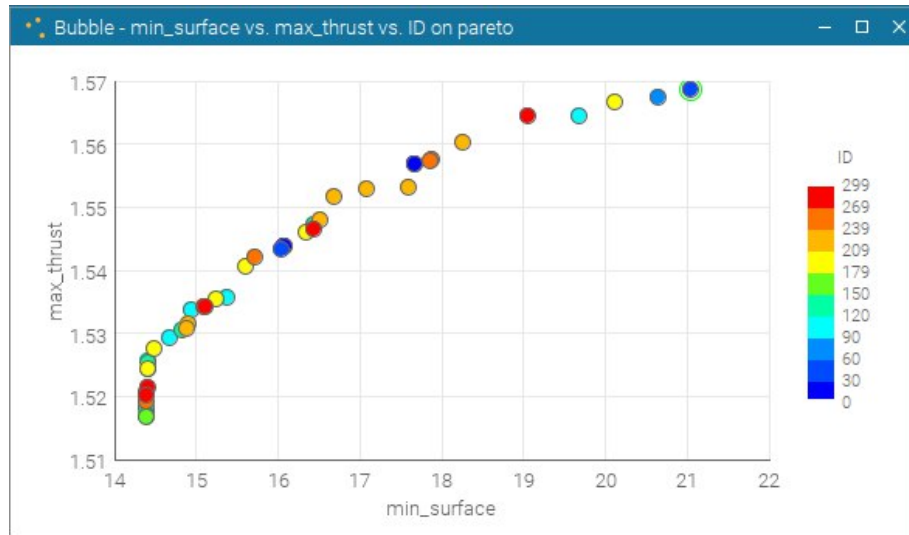


Figura 6.10: Fronte di Pareto

Il fronte non è del tutto pieno e quindi delle simulazioni ulteriori sarebbero state indicate per esplorare un numero maggiore di possibili design. Per scegliere tra i design del fronte quale sia il migliore per il trade off attraverso un parallel coordinates chart prendendo in considerazione tutte le variabili di input che quelle di output.

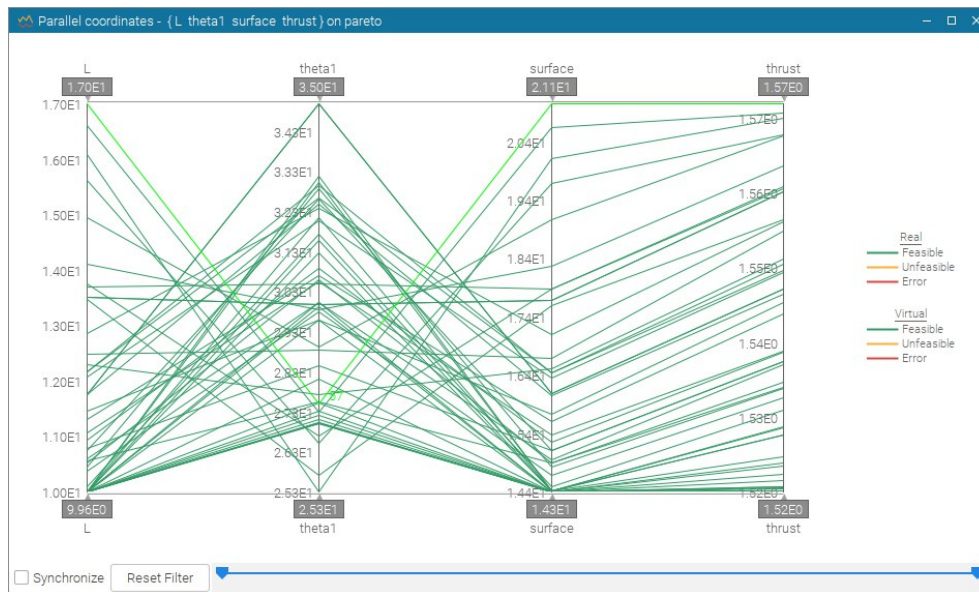


Figura 6.11: Parallel coordinates chart - Pareto designs

Ognuna delle polilinee rappresentate in figura 6.11 rappresenta un possibile design e questa interseca le linee verticali i valori delle variabili corrispondenti a quello specifico design. Attraverso questo chart è possibile operare dei filtri specifici per ogni variabile in base alle caratteristiche ricercate. Ad esempio se si volessero limitare i design a quelli caratterizzati da una massa contenuta e tra questi quelli dotati di una spinta alta, limitando il range delle

variabili di output è possibile individuare i design che soddisfano i requisiti come in figura 6.12.



Figura 6.12: Parallel coordinates chart - Pareto designs filtrati